

A SURVEY OF REINFORCEMENT LEARNING FOR ECONOMICS

PRANJAL RAWAT, GEORGETOWN UNIVERSITY

MARCH 2026

Abstract

This survey (re)introduces reinforcement learning methods to economists. The curse of dimensionality limits how far exact dynamic programming can be effectively applied, forcing us to rely on suitably “small” problems or our ability to convert “big” problems into smaller ones. While this reduction has been sufficient for many classical applications, a growing class of economic models resists such reduction. Reinforcement learning algorithms offer a natural, sample-based extension of dynamic programming, extending tractability to problems with high-dimensional states, continuous actions, and strategic interactions. I review the theory connecting classical planning to modern learning algorithms and demonstrate their mechanics through simulated examples in pricing, inventory control, strategic games, and preference elicitation. I also examine the practical vulnerabilities of these algorithms, noting their brittleness, sample inefficiency, sensitivity to hyperparameters, and the absence of global convergence guarantees outside of tabular settings. The successes of reinforcement learning remain strictly bounded by these constraints, as well as a reliance on accurate simulators. That said, when guided by economic structure, reinforcement learning provides a flexible and innovative framework. It stands as an imperfect, but promising, addition to the computational economist’s toolkit. A companion survey (Rust and Rawat, 2026b) covers the inverse problem of inferring preferences from observed behavior.

KEYWORDS: REINFORCEMENT LEARNING, ECONOMICS, STRUCTURAL ESTIMATION, INVERSE REINFORCEMENT LEARNING, MULTI-AGENT, BANDITS, RLHF

1 Introduction

This survey (re)introduces reinforcement learning to economists. I review the theoretical connections between dynamic programming and reinforcement learning, demonstrating how value iteration, Q-learning, and policy gradient methods are common solution methods to the same class of optimization problems. I then examine applications across several domains, including control problems including pricing and inventory management; structural economic models with high-dimensional state spaces; strategic games in which multi-agent algorithms compute equilibria under imperfect information; bandit problems in which economic structure yields tighter regret bounds; and preference learning. The exposition combines formal theory, practical applications and computational illustration.

Both dynamic programming and reinforcement learning solve the Bellman equation; they differ in the information requirements and the way in which the solution is refined. First, dynamic programming requires knowledge of the transition in the environment and the reward function which allows the reduction of the average Bellman error, reinforcement learning estimates value functions only from sampled transitions (observed sets of state, action, reward, next-state) which only allows reduction of the sampled Bellman error *at that state-action pair*. This allows us to improve policies in domains where it is easier to build a simulator than specify the model of the environment and rewards e.g. board games, physics simulators for robots. Second, dynamic programming makes a “breadth-first” (across all states and actions) update of the solution at each sweep, while reinforcement learning makes a “incremental” (only for the current state and action) update; this greatly reduces the computational burden and enhances scalability.

Reduction of average Bellman errors gives dynamic programming a geometric rate of convergence to the optimal solution, while the incremental updates and reduction of sampled Bellman errors, when combined with “sufficient exploration” of the state-action space, gives reinforcement learning only sublinear convergence guarantees. This is however, quite sufficient in practise, the scalability attained by only sampling transitions and making incremental updates more than makes up for the slower rates of convergence (and brittleness).

These approximation methods sacrifice theoretical guarantees. RL algorithms lack the convergence assurances of exact dynamic programming. They exhibit sensitivity to hyperparameters and initialization. They can converge to suboptimal policies without diagnostic indication. This survey presents reinforcement learning as a computationally flexible framework while acknowledging its methodological limitations.

This survey focuses on less-surveyed intersections between reinforcement learning and economics, including the shared theoretical foundations, structural estimation, strategic interaction, bandit problems with economic structure, preference learning, and causal inference. Algorithmic collusion, in which independent pricing algorithms learn to sustain supra-competitive prices (Calvano et al., 2020), is treated in a companion thesis chapter (Rawat, 2026) and omitted here. Reinforcement learning and deep learning methods for solving macroeconomic models with heterogeneous agents constitute a growing literature with dedicated methodological treatments (Atashbar and Shi, 2022), (Maliar et al., 2021), and (Fernández-Villaverde et al., 2024). Portfolio optimization, optimal execution, and asset pricing via reinforcement learning form a large body of work surveyed comprehensively elsewhere (Hambly et al., 2023). The inverse problem of inferring preferences from observed behavior using inverse reinforcement learning and structural estimation is treated in a companion survey (Rust and Rawat, 2026).

1.1 A Note on Language and Concepts

Reinforcement learning and economics developed their formalisms independently, and many technical terms that appear in both literatures carry different meanings. A reader trained in econometrics who encounters “bootstrapping” in an RL paper will think of Efron’s resampling

procedure; the RL author means something unrelated. The word “model” in structural econometrics refers to a complete specification of preferences, technology, and equilibrium; in RL it refers narrowly to a learned approximation of transition probabilities and rewards. This subsection catalogs these divergences and establishes a cross-disciplinary dictionary that the rest of the survey relies on.

The RL Problem Taxonomy

Reinforcement learning organizes its core problems along several axes that do not map directly onto econometric categories. The first distinction is between *prediction* and *control*. In RL, prediction refers to estimating the value function $V^\pi(s)$ or $Q^\pi(s, a)$ for a given, fixed policy π ; this is the problem of policy evaluation. Control refers to finding the policy π^* that maximizes expected discounted return; this is the problem of policy optimization. The econometric usage is different. Prediction in econometrics typically means forecasting an outcome variable Y given covariates X , and control refers to manipulation of instruments or policy variables to achieve a desired outcome in a physical or economic system. The RL sense of “control” is closer to what economists call “optimization” or “mechanism design.”

The second axis is *online* versus *offline*. Online RL learns while interacting with the environment, collecting new data as a consequence of its own actions. Offline RL (also called batch RL) learns exclusively from a fixed dataset of previously collected transitions, with no ability to gather additional samples. Economists will recognize the offline setting as closer to standard empirical work, where the dataset is given and the analyst cannot run new experiments. The online setting corresponds more closely to adaptive experimental design or sequential decision problems in which the decision-maker’s current choice affects future information.

The third axis is *on-policy* versus *off-policy* learning. On-policy methods evaluate and improve the same policy that generates the data. Off-policy methods learn about a target policy π from data generated by a different behavioral policy μ . This distinction is central to the causal inference chapter of this survey (Section 12), where off-policy evaluation is precisely the counterfactual policy evaluation problem that arises in program evaluation and treatment effect estimation.

The fourth axis is *model-based* versus *model-free*. A model-based RL algorithm explicitly learns or is given the transition kernel $P(s'|s, a)$ and reward function $r(s, a)$, then plans using these learned dynamics. A model-free algorithm learns value functions or policies directly from sampled transitions without ever constructing an explicit dynamics model. This terminology is a persistent source of confusion. In economics, “model-based” connotes structural estimation, the specification of a complete economic model with behavioral assumptions, equilibrium conditions, and identified parameters. An RL algorithm can be “model-based” in the RL sense (it learns P and r) while being entirely “reduced-form” in the economics sense (it makes no behavioral or equilibrium assumptions). Conversely, a structural econometric model solved by simulation methods is “model-based” in the economics sense but may use “model-free” RL algorithms internally to approximate the value function.

False Friends

Several terms deserve individual treatment because their RL and economics meanings diverge sharply enough to cause genuine misunderstanding.

Bootstrapping in statistics and econometrics refers to Efron’s resampling method for estimating the sampling distribution of an estimator by drawing with replacement from the observed data (Efron, 1979). In reinforcement learning, bootstrapping means updating a value estimate using another value estimate rather than a complete realized return. When a temporal difference algorithm sets $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$, the target $r_{t+1} + \gamma V(s_{t+1})$ depends on the current estimate $V(s_{t+1})$, which is itself uncertain. The term was introduced by

Sutton (1988) by analogy with the colloquial sense of “pulling oneself up by one’s bootstraps,” not with Efron’s procedure.

Model in structural econometrics is a complete specification of the economic environment, typically including preferences (a utility function), technology (a production function or constraint set), beliefs (an information structure), and an equilibrium concept. Estimating a structural model means recovering the primitives that rationalize observed behavior. In RL, “model” refers exclusively to the pair (P, r) , the transition dynamics and reward function of the MDP. A “model-based” RL agent that has learned $\hat{P}$ and $\hat{r}$ from data has not estimated anything resembling a structural econometric model; it has merely fit a predictive model of state transitions and rewards.

Generative model appears in both the machine learning and RL theory literatures with distinct meanings. In machine learning broadly, a generative model learns the joint distribution $P(X, Y)$ or generates synthetic data (as in GANs or diffusion models). In RL theory, a generative model is a simulator oracle that, when queried with any state-action pair (s, a) , returns a sample of the next state $s' \sim P(\cdot|s, a)$ and reward $r(s, a)$. The RL usage implies random-access simulation, the ability to query arbitrary states, which is a stronger requirement than the sequential interaction of standard online RL but weaker than knowing P analytically.

Return in finance and economics refers to the yield on an investment, typically expressed as a percentage gain. In RL, the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ is the discounted sum of future rewards from time t onward. It is the random variable whose expectation defines the value function. The RL “return” is closer to what economists call the “present discounted value” of a stream of payoffs.

Learning in the economic theory of learning (e.g., Bayesian learning, adaptive expectations) refers to agents updating beliefs about unknown parameters of their environment. In RL, learning refers to the algorithmic process of updating the parameters of a value function approximation or policy from data. An RL agent that “learns” a Q-function is not forming beliefs about an unknown distribution; it is running a stochastic approximation algorithm to find a fixed point of an operator.

Environment in economics typically refers to the economic setting, the market structure, institutional rules, or the data-generating process. In RL, the environment is a formal object, everything outside the agent, that receives actions and returns states and rewards. The RL environment subsumes the DGP, other agents, market clearing conditions, and anything else the agent cannot directly control.

Episode (synonymously, *trajectory* or *rollout*) refers to a single complete sequence of interactions from an initial state to termination, $(s_0, a_0, r_1, s_1, \dots, s_T)$. The closest econometric analogs are a single panel unit’s time series, a realization of a stochastic process, or one “history” in a dynamic discrete choice model.

Function approximation in RL refers to representing value functions or policies using parameterized function classes, such as linear combinations of basis functions, kernel methods, or neural networks. Econometricians will recognize this as sieve estimation or nonparametric series estimation, the approximation of an unknown function by projection onto a finite-dimensional basis. The convergence theory for RL with function approximation parallels the bias-variance tradeoffs in nonparametric regression.

Outcome in econometrics refers to the dependent variable Y in a regression or treatment effect framework. RL has no single analog. The reward r_t is the per-period outcome, the return G_t is the cumulative outcome, and the value function $V^\pi(s)$ is the expected cumulative outcome conditional on state. When reading across literatures, one must determine from context which RL quantity corresponds to the econometric “outcome.”

Structural Equivalences

Beyond terminological differences, several formal objects in RL and economics are mathematically identical despite independent development. Recognizing these equivalences allows economists to import intuition from familiar models.

The softmax (or Boltzmann) policy used throughout RL is the multinomial logit model of McFadden (1974a). The RL softmax policy selects actions according to

$$\pi(a | s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a' \in \mathcal{A}} \exp(Q(s, a')/\tau)}, \quad (1)$$

where $\tau > 0$ is a temperature parameter. In the discrete choice framework, $Q(s, a)$ plays the role of the deterministic component of utility $v(a | x)$, and τ is the scale parameter of the Type I extreme value (Gumbel) taste shocks ε_a . As $\tau \rightarrow 0$, the policy converges to the greedy (deterministic) policy, just as the logit choice probability concentrates on the utility-maximizing alternative as the variance of taste shocks vanishes.

The entropy regularization commonly added to RL objectives is the inclusive value (or log-sum-exp) from the discrete choice literature. The soft value function

$$V^{\text{soft}}(s) = \tau \log \sum_{a \in \mathcal{A}} \exp(Q(s, a)/\tau) \quad (2)$$

is identical to the McFadden surplus function $W(x) = \tau \log \sum_a \exp(v(a|x)/\tau) + C$, where C is Euler's constant. In the structural estimation literature, this object appears as the Emax function in dynamic discrete choice models following Rust (1987).

The advantage function $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ measures how much better action a is compared to the average action under π . In the discrete choice framework, this is precisely the choice-specific value function net of the ex-ante value function, a quantity that appears in the Hotz and Miller (1993) CCP estimator for dynamic discrete choice models.

Notation

Table 1 maps the notation used throughout this survey to the most common econometric equivalents. A Markov decision process consists of a state space $\mathcal{S}$, an action space $\mathcal{A}$, a transition kernel $P(s'|s, a)$ giving the probability of reaching state s' from state s under action a , a reward function $r(s, a)$, and a discount factor $\gamma \in [0, 1)$. A policy $\pi(a|s)$ specifies the probability of selecting action a in state s ; a deterministic policy is written $\pi(s)$. The state-value function $V^\pi(s) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | s_0 = s]$ measures expected discounted return from state s under policy π ; the action-value function $Q^\pi(s, a)$ conditions additionally on the first action. The optimal value functions V^* and Q^* correspond to the policy that maximizes expected return. Value functions are often approximated by parameterized functions such as $V(s; \mathbf{w})$ for linear approximations with weight vector $\mathbf{w}$, and $V(s; \theta)$ for general parameterizations including neural networks. Updates proceed by a learning rate $\alpha > 0$. The temporal difference error $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ measures the discrepancy between successive value estimates and drives most learning algorithms in this survey.

1.2 Organization of the Survey

The survey addresses the forward problem, that is, computing optimal policies given a known or simulated environment. Chapter 1 traces the parallel historical development of dynamic programming and reinforcement learning. Chapter 2 develops unified theory connecting planning and learning. Chapters 3 and 4 apply reinforcement learning to control problems and economic models. Chapter 5 examines strategic games. Chapter 6 addresses bandit problems. Chapter

Table 1: Notation mapping between reinforcement learning and economics.

RL Term	Symbol	Economics Equivalent	Symbol
State	$s \in \mathcal{S}$	State variable, covariate	x_t, Ω_t
Action	$a \in \mathcal{A}$	Choice, control variable	d_t, u_t
Reward	$r(s, a)$	Per-period utility, payoff	$u(x, d)$
Discount factor	γ	Discount factor	β
Policy	$\pi(a s)$	Decision rule, CCP	$P(d x)$
Value function	$V^\pi(s)$	Ex-ante value function	$\bar{V}_\theta(x)$
Q-function	$Q^\pi(s, a)$	Choice-specific value function	$v_\theta(x, d)$
Return	G_t	Present discounted value	$\sum_{k=0}^{\infty} \beta^k u_{t+k}$
Transition	$P(s' s, a)$	State transition law	$f(x_{t+1} x_t, d_t)$
Learning rate	α	Step size	α_n
TD error	δ_t	Bellman residual at sample	—

7 discusses reinforcement learning from human feedback. Chapter 8 connects reinforcement learning to causal inference. Chapter 9 concludes.

2 A Brief History of Reinforcement Learning

Reinforcement learning draws on animal psychology, game-playing programs, and optimal control theory in roughly equal measure. Thorndike’s law of effect and behaviourist trial-and-error learning provided the notion of “reinforcement” as formalized by Rescorla-Wagner. Chess and checkers programs of Shannon and Samuel from the 1950s onward gave researchers concrete problems on which to test ideas about machine learning. The Bellman-Howard-Blackwell dynamic programming framework provided the recursive structure and language.

2.1 Animal Psychology

Controlled experiments on rats, cats, and dogs inspired the “gridworld” environments still used today, and shaped how the field conceptualizes “training” agents through “reward signals”. [Thorndike \(1898\)](#) placed cats in puzzle boxes with latched doors and food visible outside. Across 15 different box configurations, the cats initially engaged in undirected behavior such as clawing at the walls, pushing against the bars, reaching through openings. The first cat to escape the simplest box required 160 seconds of random activity before accidentally pressing the latch. By the 24th trial, the same cat pressed the latch directly within 6 seconds. The learning curves showed gradual, continuous improvement rather than sudden insight. From these experiments the Law of Effect was formulated: responses (actions) followed by satisfaction (positive rewards) are “stamped in” and more likely to recur, while those followed by discomfort (negative rewards) are “stamped out.”

[Pavlov \(1927\)](#) noted that dogs salivated not only at food itself but at the sight of an empty food bowl, the sound of footsteps, and other stimuli that preceded feeding (i.e. the state). To measure these “psychic secretions” precisely, he surgically implanted fistulas (tubes allowing external collection) to collect saliva. In the canonical experiment, a metronome sounded before food delivery. After 20–40 pairings, the metronome alone elicited salivation. The response followed not from the stimulus itself but from what it “predicted”. In reinforcement learning terms, the conditioned stimulus is a state s , and the learned expectation of food is the value function $V(s)$.

[Kamin \(1969\)](#) demonstrated that learning requires more than mere co-occurrence in time. In Phase I of his blocking experiment, rats learned that a noise predicted a shock and developed a conditioned fear response to the noise alone. In Phase II, a compound stimulus of noise plus light was paired with the same shock. When the light was subsequently presented alone, no fear response occurred. The light was “blocked” (ignored) because the noise already predicted the shock perfectly. There was no prediction error (or “surprise”) to drive learning about the light. Once the noise was established as a predictor, the light added no new information and thus no new learning occurred.

[Rescorla and Wagner \(1972\)](#) formalized the blocking phenomenon as a prediction-error learning rule. Translating to RL notation:¹

$$V(s) \leftarrow V(s) + \alpha \delta, \quad \text{where } \delta = r - V(s) \quad (3)$$

This is temporal difference learning with $\gamma = 0$, without discounting of future rewards. Each stimulus s begins with $V(s) = 0$. On each trial, the organism observes the stimuli present, receives outcome $r \in \{0, 1\}$, and updates values according to δ . The model is purely predictive; there are no actions, only learned expectations about reward.² The model’s power lay in

¹The original notation used V_i for associative strength of stimulus i , α_i for stimulus salience (noticeability), β_j for learning rate, λ_j for maximum conditioning (1 if reward present, 0 otherwise), and $V_{\text{tot}} = \sum_k V_k$ for total prediction. The correspondence is: $V_i \rightarrow V(s)$, $\alpha_i \beta_j \rightarrow \alpha$, $\lambda_j \rightarrow r$, $V_{\text{tot}} \rightarrow V(s)$. See [Sutton and Barto \(1990\)](#) for details.

²The Rescorla-Wagner update is mathematically identical to the Widrow-Hoff least mean squares rule.

prediction, not just explanation. It correctly predicted overexpectation, whereby two separately conditioned stimuli combined and reinforced together each lose value because their summed prediction exceeds r .

2.2 Board Games

Chess and checkers are sequential decision problems. The board position is a state $s \in \mathcal{S}$, a legal move is an action $a \in \mathcal{A}(s)$, and the resulting position is a successor state $s' = T(s, a)$ determined by the rules of the game. The game outcome provides a terminal reward $r \in \{+1, 0, -1\}$ for win, draw, or loss. An evaluation function $f(P)$ that scores a position corresponds to a value function $V(s)$ estimating expected outcome. These games are deterministic (no chance moves in chess), fully observable (both players see the entire board), and zero-sum (one player’s gain is the other’s loss). The adversarial structure introduces a second player whose actions a' must be anticipated.

Shannon (1950) posed the fundamental question: can we program a general-purpose computer to play chess, and if so, what principles should guide the design? The challenge is computational. A chess game averages 40 moves per player with roughly 30 legal moves available at each position. Shannon estimated that exhaustive search through all possible games would require examining approximately $30^{80} \approx 10^{120}$ positions, a number exceeding the atoms in the observable universe. This is the curse of dimensionality applied to games, where state space size grows exponentially with the number of sequential decisions. Shannon calculated that brute-force enumeration would require 10^{90} years at any foreseeable computing speed. The curse intensifies with game complexity. Chess has approximately 10^{47} legal positions, shogi 10^{71} , and Go 10^{171} .³

Shannon distinguished two approaches. Type A strategies search all continuations to a fixed depth H , building a complete game tree and evaluating every leaf (“rote-learning”). Type B strategies search selectively, examining only variations deemed important by some criterion (“generalization”). Either approach requires an evaluation function $f(P)$ to score positions where search terminates. Shannon proposed linear evaluation:

$$f(P) = \sum_i w_i \phi_i(P) \quad (4)$$

with features ϕ_i for material, mobility, pawn structure, and king safety. Weights w_i were hand-tuned. The minimax principle governs adversarial search. In a two-player zero-sum game, the value of a position satisfies

$$V(s) = \max_{a \in \mathcal{A}(s)} \min_{a' \in \mathcal{A}'(s')} V(T(s, a, a')) \quad (5)$$

where the maximizing player moves first and the minimizing opponent responds optimally. This is model-based planning, since the transition function T is known exactly from the game rules. The computational problem is how to use limited search resources effectively given the exponential tree.

Both Type A and Type B strategies truncate the game tree at depth H and substitute the evaluation function for exact continuation values. This is approximate dynamic programming. The true value $V^*(s)$ satisfies a recursive equation, but computing it exactly is infeasible, so the recursion is truncated and terminal values approximated. Deeper lookahead (H -step search) builds larger trees; rollout policies extend search by simulating play to the end using a fast base policy.⁴ The evaluation function serves as a heuristic substitute for exact computation. Shannon

³State space estimates from Igami (2020).

⁴Monte Carlo tree search, developed later, samples rollouts rather than enumerating all branches, enabling deeper effective lookahead in games with large branching factors.

did not implement a chess program; the 1950 paper is theoretical, outlining the architecture that shaped fifty years of game engines.

Samuel (1959) built a checkers program for the IBM 704 that could improve through experience. The program played against itself, generating virtually unlimited training data at no cost. It parameterized the value function as a linear combination of hand-crafted features (piece advantage, mobility, king safety):

$$V(s; \mathbf{w}) = \sum_i w_i \phi_i(s) \quad (6)$$

After each move from s_t to s_{t+1} , weights were updated by temporal difference:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [V(s_{t+1}; \mathbf{w}) - V(s_t; \mathbf{w})] \nabla_{\mathbf{w}} V(s_t; \mathbf{w}) \quad (7)$$

In a 1965 match, World Champion W.F. Hellman won all four games played by mail, but was played to a draw in one game. After learning from 173,989 book moves, the program agreed with the book-recommended move (or rated only 1 move higher) 64% of the time without lookahead. With lookahead and minimaxing, it followed book moves "a much higher fraction of the time."

The linear parameterization compresses the value function from 10^{20} table entries to dozens of weights, the essential response to the curse of dimensionality. Samuel's architecture — minimax search to depth H with the learned evaluation function scoring leaves — is an early instance of rollout, where tree search simulates forward, truncating at H and substituting $V(s)$ for exact continuation values.⁵ The conceptual apparatus of modern game-playing AI — self-play, evaluation learning, tree search, and function approximation — was present in the 1950s.

2.3 Optimal Control

Bellman (1957) considered multi-stage decision processes in which a system occupies state $s \in \mathcal{S}$, the decision-maker chooses action $a \in \mathcal{A}$, the system transitions to $s' \sim P(\cdot|s, a)$, and a reward $r(s, a, s')$ accrues. The objective is to maximize cumulative reward over a finite or infinite horizon. The classical approach treats an N -stage process as a single N -dimensional optimization. Bellman calculated the consequence. A 10-stage process with 10 grid points per variable requires 10^{10} function evaluations; at one evaluation per second, 10^{10} evaluations require 2.77 million hours. He called this exponential growth the curse of dimensionality. His solution was the principle of optimality, namely that an optimal policy has the property that, whatever the initial state and initial decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision. This principle yields the Bellman equation:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right] \quad (8)$$

The equation reduces an N -dimensional problem to a sequence of N one-dimensional problems. Value iteration computes V^* by iterating $V_{k+1}(s) = \max_a [r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s')]$. The monograph applied the method to resource allocation, inventory control, bottleneck scheduling, gold mining under uncertainty, and multi-stage games.⁶

Howard (1960) observed that value iteration converges slowly for problems of indefinite duration. His alternative, policy iteration, solves for the value function of a fixed policy and then improves the policy directly. Given policy π , policy evaluation computes the gain g (average reward per period) and relative values v_i by solving the linear system $g + v_i = q_i + \sum_j p_{ij} v_j$

⁵Bertsekas (2021) interprets this as approximate dynamic programming, where offline training (learning V) combined with online planning (tree search) implements a Newton-like step for the Bellman equation.

⁶Chapter IX shows how dynamic programming derives classical variational conditions from the functional equation. The continuous-time analogue is the Hamilton-Jacobi-Bellman equation.

for $i = 1, \dots, N$, where q_i is the expected immediate reward in state i and p_{ij} is the transition probability under π . Policy improvement then selects, for each state i , the action k maximizing $q_i^k + \sum_j p_{ij}^k v_j$. Howard proved that each iteration strictly increases the gain unless the policy is already optimal, and the algorithm terminates in finitely many steps. For a problem with 50 states and 50 actions per state, exhaustive enumeration must consider $50^{50} \approx 10^{85}$ policies; policy iteration finds the optimum in a handful of iterations. Howard demonstrated the method on a toymaker’s production problem, taxicab dispatch in three city zones, and automobile replacement timing.

Blackwell (1965) established the measure-theoretic foundations for discounted dynamic programming with general state and action spaces. He proved that the Bellman operator T defined by $Tu(s) = \sup_a [r(s, a) + \gamma \int u(s') P(ds'|s, a)]$ is a contraction with modulus γ : $\|Tu - Tv\|_\infty \leq \gamma \|u - v\|_\infty$. Banach’s fixed-point theorem then guarantees a unique bounded solution V^* to the Bellman equation, with $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$ under value iteration. The central result concerns stationary policies, which use the same decision rule $f : \mathcal{S} \rightarrow \mathcal{A}$ at every period regardless of history. Blackwell proved that if the action space is finite, there exists an optimal stationary policy. For countable action spaces, ϵ -optimal stationary policies exist for every $\epsilon > 0$. These results justify the focus on memoryless policies, since optimal behavior depends only on the current state, not on the history of past states and actions. The Bellman equation, Howard’s policy iteration, and Blackwell’s existence theorems constitute the planning framework. Given complete knowledge of P and r , these methods compute optimal policies exactly. The challenge of learning without such knowledge is the central problem of reinforcement learning.

3 Reinforcement Learning Algorithms

In this section I introduce important reinforcement learning algorithms while pushing the theoretical results to the next chapter. The central problem is how do we move from estimating V_π and V_* when we do not have a model.

3.1 The Classical Synthesis

3.1.1 Monte Carlo Estimation

When $P(s'|s, a)$ and $r(s, a)$ are unknown, the obvious approach is to use Monte Carlo (MC) to approximate them. These methods estimate value functions from sampled *episodes* $(s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T)$. The realized *return* from state s_t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \quad (9)$$

First-visit MC prediction averages G_t over episodes for each state s , counting only its first occurrence per episode. Each first-visit return is an independent draw from the return distribution, so the sample mean converges almost surely to $V^\pi(s)$ by the strong law of large numbers (Sutton and Barto, 2018). An incremental update is,

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

For MC control, we can estimate action-values $Q(s, a)$ by averaging first-visit returns from each state-action pair, then improve the policy greedily: $\pi(s) = \operatorname{argmax}_a Q(s, a)$. Under exploring starts (every (s, a) pair begins an episode infinitely often), this alternation converges to Q^* .

7

⁷Tsitsiklis (2002) proved convergence even when the policy improves after every episode rather than waiting for complete evaluation. In practice, exploring starts is infeasible, so on-policy variants use ϵ -greedy exploration

3.1.2 Sutton (1988)

Monte Carlo has two limitations. The agent must wait for episode termination to compute G_t , ruling out continuing tasks. And G_t is unbiased ($\mathbb{E}[G_t | S_t = s] = V^\pi(s)$) but high-variance, because it sums random rewards over the entire *trajectory*.

? proposed temporal difference (TD) learning to fix both problems. TD(0) replaces the full return G_t with a one-step target:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \quad (10)$$

The target $r_{t+1} + \gamma V(s_{t+1})$ depends on one random reward and one random transition, so its variance is low. The cost is bias. The bootstrap target $V(s_{t+1})$ is the agent’s current estimate, not the true value. This is “*bootstrapping*”.⁸ As V improves, the bias shrinks; the low variance persists regardless. Sutton demonstrated this tradeoff on a five-state random walk where TD(0) converged faster than Monte Carlo with less data.

The general TD(λ) update interpolates between these extremes through an eligibility trace.⁹

$$V(s_t) \leftarrow V(s_t) + \alpha \delta_t e_t(s) \quad (11)$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the TD error and $e_t(s) = \gamma \lambda e_{t-1}(s) + \mathbb{1}\{s = s_t\}$ is the eligibility trace for state s . Setting $\lambda = 0$ yields TD(0); setting $\lambda = 1$ recovers Monte Carlo returns. Intermediate λ trades off variance against bias.¹⁰

3.1.3 Watkins (1989)

TD(0) learns value functions $V(s)$, but converting these to actions (the control problem) still requires knowing transition probabilities. Given $V(s')$ for all successor states, the agent needs to know which action leads to which successor.

Watkins and Dayan (1992), formalizing Watkins’s 1989 PhD thesis, provided the solution. Instead of learning $V(s')$ learn $Q(s, a)$ (the action value function or the “quality” of actions function) directly, the expected return from taking action a in state s and then behaving optimally. The optimal policy is then $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$, requiring no model to act. The Bellman optimality equation provides the fixed-point condition:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (12)$$

Q-learning achieves this via the update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (13)$$

Q-learning converges to Q^* under standard regularity conditions (Section 4.2). The maximization over a' makes Q-learning *off-policy*:¹¹ the update target uses the greedy action at the

instead. These converge to Q^* provided the exploration schedule satisfies the greedy-in-the-limit with infinite exploration (GLIE) condition: every state-action pair is visited infinitely often, and $\varepsilon_t \rightarrow 0$ so the policy converges to greedy in the limit.

⁸Bootstrapping means updating value estimates using other value estimates rather than observed returns.

⁹The eligibility trace records which states were recently visited, allowing credit assignment to propagate backward in time. States visited more recently receive stronger updates when the TD error is observed.

¹⁰Dayan (1992) proved convergence of TD(λ) for general λ in the tabular case; Jaakkola et al. (1994) gave a unified stochastic approximation proof covering both TD and Q-learning; Tsitsiklis and Van Roy (1997) extended the analysis to linear function approximation.

¹¹Off-policy learning means the agent learns about a policy different from the one generating data. Q-learning learns about the greedy policy $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$ while collecting data with an exploratory policy. This exploratory policy needs only to be sufficiently “exploratory” and admits a wide range of policies; including a

next state regardless of the action actually taken. Therefore the agent can follow an ε -greedy exploration strategy or even a fully random policy, while learning about the optimal policy directly.

3.1.4 Williams (1992)

Value-based methods learn an action-value function and derive a policy from it. Williams (1992) derived an alternative, policy gradients, that optimize the policy directly by gradient ascent on expected return

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right] \quad (14)$$

where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}$ is the discounted return from time t . The log-derivative trick allows the gradient to be estimated from sampled trajectories without differentiating through the environment dynamics.

Consider a robot arm that must apply a continuous torque $a \in \mathbb{R}$ to reach a target angle. Q-learning requires computing $\max_a Q(s, a)$ at every update, which becomes a nested optimization problem¹² when the action space is continuous. A policy gradient method sidesteps the issue. Parameterize the policy as a Gaussian $\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s))$,¹³ sample an action, observe the return, and update θ by REINFORCE. No maximization over actions is required. Virtually all continuous-control results in reinforcement learning descend from the policy gradient framework rather than from Q-learning for this reason.

3.1.5 Tesauro (1994)

Tesauro (1994)’s TD-Gammon demonstrated that temporal difference learning with neural network function approximation could achieve expert-level play in a domain with approximately 10^{20} legal positions¹⁴.

Backgammon was far beyond tabular methods, yet TD-Gammon trained a feedforward neural network to estimate the probability of winning from any board position. A hidden layer of 80 sigmoid units fed a single sigmoid output.

$$\hat{V}(s) = \sigma(\mathbf{w}^{\top} \sigma(W\mathbf{x}(s) + \mathbf{b}) + c) \quad (15)$$

where σ is the logistic sigmoid, W is the input-to-hidden weight matrix, and $\mathbf{w}$ is the hidden-to-output weight vector. The network was trained by self-play using TD(λ) with $\lambda = 0.7$. After each move from position s_t to s_{t+1} , the weights θ ¹⁵ were updated by

$$\theta \leftarrow \theta + \alpha [\hat{V}(s_{t+1}) - \hat{V}(s_t)] \mathbf{e}_t \quad (16)$$

where the eligibility trace $\mathbf{e}_t = \sum_{k=1}^t \lambda^{t-k} \nabla_{\theta} \hat{V}(s_k)$ accumulates exponentially decayed gradients of past predictions.¹⁶ At game’s end, $\hat{V}(s_{t+1})$ is replaced by the outcome $z \in \{0, 1\}$.

fully random policy.

¹²In continuous action spaces, $\max_a Q(s, a)$ has no closed-form solution in general and must be solved numerically at every Bellman update. Discretizing a d -dimensional action space on a grid of m points per dimension costs $O(m^d)$ evaluations per update step.

¹³The Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ has density $(2\pi\sigma^2)^{-1/2} \exp(-(a - \mu)^2/2\sigma^2)$. Here $\mu_{\theta}(s)$ and $\sigma_{\theta}(s)$ are neural network outputs parameterizing the policy mean and standard deviation.

¹⁴The state $\mathbf{x}(s)$ was a 198-dimensional binary encoding of the raw board (four units per board point per player indicating checker counts, plus bar and borne-off counts). The output was a four-component vector estimating probabilities of each game outcome (White/Black $\times$ normal win/gammon), and the move maximizing expected outcome among all legal moves was selected at each step.

¹⁵ θ denotes the full vector of network weights and biases, generalizing the scalar θ used for policy parameters in earlier sections.

¹⁶In the neural network case, the eligibility trace $\mathbf{e}_t \in \mathbb{R}^{|\theta|}$ is a vector accumulating exponentially-weighted

Self-play proceeds as follows. A single neural network $\hat{V}$ serves as the evaluation function for both players. At each turn, the current player rolls the dice, which determines the set of legal moves. For each legal move, the network evaluates the resulting board position $\hat{V}(s')$. The player selects the move that maximizes $\hat{V}(s')$ from its own perspective. The opponent then does the same using the identical network. After each move from position s_t to s_{t+1} , the weights are updated by the TD(λ) rule using the error $\hat{V}(s_{t+1}) - \hat{V}(s_t)$, which pushes earlier predictions toward later ones. When the game ends, the terminal signal $z \in \{0, 1\}$ replaces $\hat{V}(s_{t+1})$, grounding the entire chain of predictions in the actual outcome. As the network improves, it generates stronger play on both sides, which produces harder training games, which in turn drives further improvement. The dice rolls ensure that the network encounters a wide variety of board positions without requiring an explicit exploration mechanism¹⁷.

3.1.6 SARSA (1994)

Q-learning learns the optimal action-value function regardless of the policy generating experience. This off-policy property is useful but introduces complications when combined with function approximation. Rummery and Niranjan (1994) introduced SARSA as an *on-policy*¹⁸ alternative that learns the value of the policy actually being followed. The name derives from the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ used in each update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (17)$$

The key difference from Q-learning is that SARSA bootstraps from the action a_{t+1} actually taken at the next state, rather than the greedy action $\argmax_{a'} Q(s_{t+1}, a')$. This makes the algorithm on-policy, since the target depends on the behavior policy generating the data.

If the agent follows an ε -greedy policy, SARSA converges to $Q^{\varepsilon\text{-greedy}}$, not Q^* ¹⁹ policies and standard step-size conditions (Section 4.2). This distinction matters when exploration is costly. Consider the cliff-walking problem, where an agent must traverse a gridworld with a cliff along one edge. The optimal path runs along the cliff edge (shortest route), but the ε -greedy policy occasionally falls off. Q-learning learns the optimal path because it evaluates the greedy policy; the agent falls off during learning but the Q-values reflect the optimal route. SARSA learns a safer path further from the cliff because it evaluates the actual exploratory policy; it accounts for the fact that exploration sometimes leads to catastrophic states.

3.1.7 Baird (1995)

Baird (1995) constructed a six-state star MDP demonstrating divergence of Q-learning with linear function approximation. The MDP has five outer states that all transition to a single inner state under the target policy. The off-policy behavior samples states uniformly. With linear function approximation, the weights grow without bound under repeated Q-learning updates. The source of instability is the interaction of three components, namely bootstrapping (updating from estimated values rather than observed returns), off-policy learning (training on data from a different policy than the target), and function approximation (representing the value

gradients, extending the scalar state-based trace to parameter space.

¹⁷TD-Gammon 1.0, trained on 300,000 self-play games, played at a strong intermediate level. Version 1.0 trained on 300,000 self-play games with 80 hidden units and no lookahead search; version 2.1 trained on 1,500,000 games with 2-ply search, reducing its average loss rate against world-class human players from -0.25 to -0.02 points per game. Version 2.1 achieved near-parity with Bill Robertie, a former world champion. The program discovered novel positional strategies subsequently adopted by the human backgammon community.

¹⁸On-policy methods learn about the policy currently generating data, in contrast to off-policy methods (see footnote on Q-learning above) which learn about a different target policy.

¹⁹SARSA converges to Q^* under GLIE (greedy in the limit with infinite exploration) A GLIE schedule explores all state-action pairs infinitely often but converges to the greedy policy asymptotically. The standard ε -greedy policy with $\varepsilon_t \rightarrow 0$ is one such schedule.

function with a parameterized model). Sutton and Barto (2018) later named this the deadly triad. Any two components can be combined safely; all three together permit divergence. The mechanism underlying this instability is analyzed in Section 5.4.

This result explains the asymmetry between Tesauro’s success and Baird’s failure. TD-Gammon used bootstrapping and function approximation but was on-policy, with training data coming from the same self-play policy whose value was being estimated. Baird’s counterexample used all three components and diverged. Baird also proposed a constructive solution, namely residual gradient algorithms that perform gradient descent on the mean-squared Bellman residual, guaranteeing convergence at the cost of a different fixed point.

3.1.8 Actor-Critic Methods (2000)

The idea of maintaining both a policy (*actor*) and a value function (*critic*) dates to Barto et al. (1983), who used a two-component system to solve the pole-balancing task. The actor selected actions; the critic predicted future reinforcement and generated training signals for the actor. This architecture addressed a limitation of pure policy gradient methods. REINFORCE estimates gradients from Monte Carlo returns, which are unbiased but have high variance because returns depend on the full trajectory of stochastic transitions and rewards. TD methods use bootstrapped targets $r + \gamma V(s')$, which have lower variance but introduce bias when V is approximate.

Actor-critic methods combine both ideas. The critic learns a value function $V(s)$ by TD updates.

$$V(s_t) \leftarrow V(s_t) + \alpha_c \delta_t, \quad \delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \quad (18)$$

The actor updates the policy using the TD error as a sample of the advantage.

$$\theta \leftarrow \theta + \alpha_a \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \delta_t \quad (19)$$

The TD error δ_t estimates $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$, the advantage of action a_t over the average action.²⁰ Positive advantages indicate the action was better than expected; negative advantages indicate it was worse.

Konda and Tsitsiklis (2000) provided the first convergence proof for actor-critic algorithms with function approximation, showing convergence to a stationary point under two-timescale learning rates ($\alpha_c \gg \alpha_a$) and a compatibility condition on the critic architecture (Section 4.5).

3.1.9 Natural Policy Gradient (2001)

Standard gradient descent treats all parameter directions equally, but policy parameters define probability distributions whose natural geometry is not Euclidean. Kakade (2001) introduced the natural policy gradient, which measures progress in distribution space rather than parameter space. The update uses the Fisher information matrix $F(\theta)$:

$$F(\theta) = \mathbb{E}_{s \sim d^{\pi}, a \sim \pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s)^{\top} \right] \quad (20)$$

The natural gradient is

$$\tilde{\nabla}_{\theta} J(\theta) = F(\theta)^{-1} \nabla_{\theta} J(\theta) \quad (21)$$

This direction is invariant to reparameterization of the policy. In the tabular softmax²¹ case, a single natural gradient step with unit step size recovers one step of exact policy iteration (Section 4.4).

²⁰That δ_t is an unbiased estimate of the advantage follows from the policy gradient theorem (Sutton et al., 2000).

²¹The softmax function maps a vector $\mathbf{z}$ to a probability distribution: $\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$. A softmax policy parameterizes action probabilities as $\pi_{\theta}(a | s) = \text{softmax}(\theta_{s,a})$.

The computational bottleneck is inverting $F(\theta) \in \mathbb{R}^{d \times d}$. Practical implementations use conjugate gradient methods to solve $F(\theta)x = \nabla_{\theta} J(\theta)$ without forming F explicitly. This approach was later scaled to deep neural networks by TRPO and PPO.

3.2 The Deep Learning Era

3.2.1 Deep Q-Networks (2015)

The two decades after TD-Gammon saw limited success in applying neural networks to reinforcement learning. Attempts to replicate Tesauro’s approach in chess and Go did not achieve comparable results, though progress continued in robotics and other domains. Mnih et al. (2015) changed this by training a single convolutional neural network²² to play 49 Atari 2600 games directly from pixel inputs ($210 \times 160 \times 3$) and a scalar score, using no game-specific features.

The architecture processed four consecutive frames through three convolutional layers and a fully connected layer. The network $Q(s, a; \theta)$ was trained to minimize the squared temporal difference error

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (22)$$

Two innovations stabilized learning. *Experience replay* (Lin, 1992) stored transitions (s, a, r, s') in a buffer $\mathcal{D}$ holding 10^6 transitions and sampled uniformly for training. This breaks the temporal correlation between consecutive updates that would otherwise cause the network to overfit to recent experience. A *target network* used a frozen copy of parameters θ^- ,²³ updated to match θ only every $C = 10,000$ steps, so the regression target does not shift with each gradient step.

DQN exceeded human-level performance on 29 of 49 games using a single architecture and hyperparameters. The same network that learned Breakout (where the agent must track a ball and position a paddle) also learned Space Invaders (where the agent must track multiple enemies and time shots). Games requiring long-horizon planning or sparse rewards, such as Montezuma’s Revenge (where the agent must navigate rooms and collect keys before receiving any reward), remained difficult.

3.2.2 TRPO and PPO (2015, 2017)

Policy gradient methods suffer from a practical instability. A single large gradient step can move the policy into a region of parameter space where performance collapses, and recovery may be slow or impossible. The objective $J(\theta)$ is not well approximated by its linear Taylor expansion when the step size is large relative to the curvature of the policy space.

Schulman et al. (2015) addressed this with Trust Region Policy Optimization (TRPO). TRPO solves the constrained optimization problem

$$\max_{\theta} L_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \leq \delta \quad (23)$$

where L is a surrogate objective based on the advantage function $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$.²⁴ The KL constraint prevents the policy from moving so far in one update that performance collapses and recovery becomes impossible.

²²A convolutional neural network applies learned spatial filters to detect local patterns in grid-structured data, commonly used for image inputs.

²³The notation θ^- denotes target network parameters, a lagged copy of θ updated periodically. This stabilizes learning by keeping the regression target fixed during gradient steps.

²⁴The Kullback-Leibler divergence $D_{\text{KL}}(p||q) = \sum_x p(x) \log(p(x)/q(x))$ measures statistical distance between distributions. The bar denotes expectation over states: $\bar{D}_{\text{KL}} = \mathbb{E}_s[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s)||\pi_{\theta}(\cdot|s))]$.

Schulman et al. (2017) proposed Proximal Policy Optimization (PPO) as a simpler alternative. PPO replaces the KL constraint with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (24)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio. The clipping removes the incentive for $r_t(\theta)$ to move outside the interval $[1 - \varepsilon, 1 + \varepsilon]$, penalizing large policy updates without explicitly computing a divergence measure.

PPO outperformed A2C, TRPO, and the cross-entropy method on continuous control benchmarks and achieved the highest average reward on 30 of 49 Atari games among the methods tested. PPO became the default policy optimization algorithm for large-scale RL applications, demonstrating that constraining the magnitude of policy updates is essential for stable optimization.

3.2.3 Soft Actor-Critic (2018)

Haarnoja et al. (2018) introduced Soft Actor-Critic (SAC), which adds entropy regularization to the actor-critic framework. The agent maximizes expected return plus an entropy bonus:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t))) \right] \quad (25)$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a) \log \pi(a)$ is the entropy and $\tau > 0$ is a temperature parameter. The entropy bonus encourages exploration by penalizing deterministic policies. The optimal policy under this objective is softmax in the Q-values: $\pi^*(a|s) \propto \exp(Q^*(s, a)/\tau)$, connecting to discrete choice models in econometrics.

SAC maintains two Q-networks (to reduce overestimation bias) and a policy network. The soft Bellman operator for the critic is:

$$(T^\pi Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V(s')], \quad V(s) = \mathbb{E}_{a \sim \pi} [Q(s, a) - \tau \log \pi(a|s)] \quad (26)$$

SAC is off-policy (using experience replay), handles continuous actions naturally, and achieves state-of-the-art sample efficiency on continuous control benchmarks. The entropy regularization provides automatic exploration without ε -greedy schedules.

3.2.4 AlphaGo Zero (2017)

The game of Go has approximately 10^{170} legal positions and a branching factor of roughly 250, far beyond the reach of brute-force search. Hand-crafted evaluation functions, which had succeeded in chess, failed here because positional concepts like influence, territory, and group viability are holistic and contextual. Monte Carlo tree search (MCTS) had achieved amateur-level play by using random simulations to estimate position values, but progress had stalled below professional strength. Silver et al. (2016) broke through by combining supervised learning from 30 million human expert positions, reinforcement learning via self-play, and MCTS with learned value and policy networks; the resulting system defeated Lee Sedol four games to one in March 2016. A year later, Silver et al. (2017) showed that none of the human data was necessary.

AlphaGo Zero uses a single neural network $f_\theta(s) = (\mathbf{p}, v)$ that takes a board position s and outputs both a policy vector $\mathbf{p}$ over legal moves and a scalar value v estimating the probability of winning.²⁵ The input representation consists of 17 binary planes on the 19×19 board,

²⁵A convolutional neural network applies small learned filters that slide across a grid input, detecting local patterns regardless of position on the board. The same filter that recognizes a capturing pattern in one corner will recognize it in another.

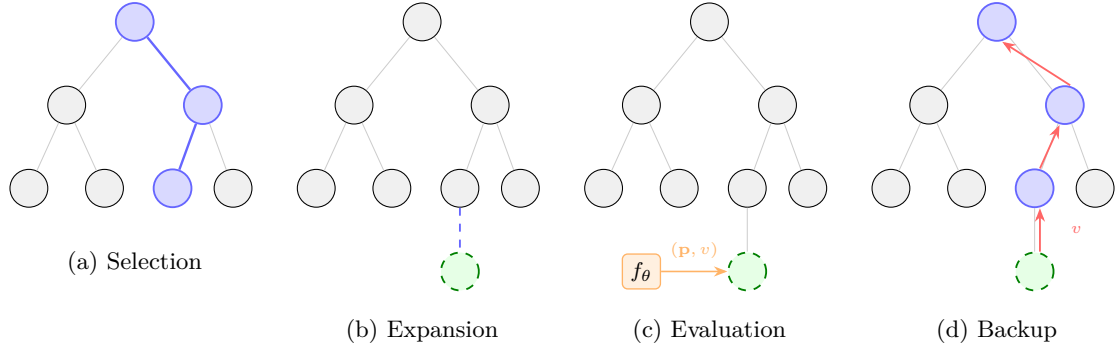


Figure 1: The four phases of a single MCTS simulation in AlphaGo Zero. (a) Selection traverses the tree from the root, choosing at each node the action maximizing a UCB-like score balancing exploitation (Q) and exploration (P/N). (b) Expansion adds a new leaf node when the traversal reaches an unexplored position. (c) The neural network f_θ evaluates the new position, producing move priors $\mathbf{p}$ and a value estimate v . (d) Backup propagates v along the traversed path, updating mean values $Q(s, a)$ and visit counts $N(s, a)$ at each edge.

encoding the raw game state without any hand-crafted features: eight planes for Black’s stone positions over the last eight moves, eight for White’s, and one indicating which color plays next.²⁶ The board’s fixed grid structure makes convolutional networks effective, since spatial patterns learned in one region transfer to others. The architecture uses residual blocks,²⁷ which allow training of very deep networks.

During play, each move is selected by running MCTS, which conducts 1,600 simulated games from the current position to estimate move quality. Each simulation proceeds in four phases, illustrated in Figure 1. In the selection phase, the algorithm starts from the current position and traverses the partially built search tree by choosing at each node the action that maximizes $Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \sqrt{\sum_b N(s, b)} / (1 + N(s, a))$, where $Q(s, a)$ is the current average value of action a , $P(s, a)$ is the prior probability from the neural network, and $N(s, a)$ is the visit count.²⁸ In the expansion and evaluation phase, when the traversal reaches a position not yet in the tree, the neural network evaluates it in a single forward pass,²⁹ producing a policy vector $\mathbf{p}$ and a value estimate v ; the policy initializes prior probabilities $P(s', a) = p_a$ for each child edge. In the backup phase, the value v propagates back up the traversed path, incrementing each edge’s visit count $N(s, a)$ and updating its mean value $Q(s, a)$. After all 1,600 simulations, the algorithm selects the move with the highest visit count at the root.

The training loop generates self-play games. At each board position s_t during a game, the program runs MCTS to produce improved move probabilities $\boldsymbol{\pi}_t$, where $\pi_t(a) \propto N(s_t, a)^{1/\tau}$ and τ is a temperature parameter controlling exploration.³⁰ A move a_t is sampled from $\boldsymbol{\pi}_t$ and played. At game end, the outcome $z \in \{-1, +1\}$ is recorded. Each position becomes a training

²⁶The history planes allow the network to detect ko situations (positions that cannot be immediately repeated) and infer the trajectory of play without any explicit feature engineering.

²⁷A residual block computes $\mathbf{x} + g(\mathbf{x})$ rather than just $g(\mathbf{x})$, where g is a learned transformation. The skip connection allows gradients to flow through very deep networks without vanishing. AlphaGo Zero’s 40-block architecture has 79 parameterized layers.

²⁸The constant c_{puct} controls exploration. Actions visited often have well-estimated Q values but a shrinking exploration bonus; rarely visited actions have uncertain values but a large bonus. This is a continuous analogue of the upper confidence bound (UCB) strategy from bandit theory.

²⁹A forward pass is one evaluation of the neural network on an input, producing outputs without any parameter updates.

³⁰Early in the game ($t \leq 30$), $\tau = 1$ so moves are sampled proportionally to visit counts, encouraging diverse openings. Later, $\tau \rightarrow 0$ and the most-visited move is selected deterministically. Dirichlet noise is also added to root priors, $P(s, a) = (1 - \epsilon)p_a + \epsilon\eta_a$ with $\eta \sim \text{Dir}(0.03)$, ensuring all legal moves can be explored despite strong network priors.

triple $(s_t, \boldsymbol{\pi}_t, z)$, and the network parameters are updated to minimize

$$\ell(\theta) = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\theta\|^2 \quad (27)$$

where the first term is a value prediction loss, the second is a policy cross-entropy loss,³¹ and the third is L_2 regularization.³² The key mechanism is a virtuous cycle. MCTS serves as a policy improvement operator, since the search probabilities $\boldsymbol{\pi}$ are stronger than the raw network outputs $\mathbf{p}$. Training the network to match $\boldsymbol{\pi}$ distills the search improvements back into the network, and the improved network in turn produces better MCTS. After 72 hours of self-play on 4 TPUs, AlphaGo Zero surpassed all previous versions, including the one that defeated Lee Sedol, and discovered novel strategies not previously seen in human play.

Go was well-suited to this architecture. Its fixed 19×19 board maps naturally to convolutional networks, its perfect information and deterministic transitions make MCTS’s tree structure exact, and the binary game outcome provides an unambiguous training signal. Igami (2020) interprets the architecture in econometric terms, where the policy network is a conditional choice probability (CCP) estimator, the value network is a conditional value function (CVF) estimator, and the system performs CCP estimation and forward simulation jointly, connecting to the approach of Hotz and Miller (1993) in dynamic discrete choice.

4 The Theory of Reinforcement Learning

4.1 The Geometry of Dynamic Programming

Value iteration (VI) and policy iteration (PI) are the workhorses of dynamic programming. VI applies the Bellman operator repeatedly until convergence; PI alternates between *policy evaluation* (solving a linear system) and *policy improvement* (taking the greedy action). PI converges faster. Why? The answer reveals a connection between dynamic programming and numerical optimization. Policy iteration is Newton’s method applied to the Bellman equation.

4.1.1 Value Iteration as Picard Iteration

Consider the Bellman optimality operator T acting on value functions.

$$(TV)(s) = \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V(s') \right\}. \quad (28)$$

This operator is nonlinear due to the max. Value iteration applies T repeatedly: $V_{k+1} = TV_k$. Since T is a γ -contraction in the supremum norm (Denardo, 1967), Banach’s fixed-point theorem guarantees $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$.³³ This is Picard iteration, with linear convergence at rate γ .³⁴ The iteration count to reduce error by a factor of δ is $k = \log(\delta)/\log(1/\gamma)$ (Bertsekas, 1996).

³¹The cross-entropy loss $-\boldsymbol{\pi}^\top \log \mathbf{p}$ measures how well the predicted distribution $\mathbf{p}$ matches the target $\boldsymbol{\pi}$; it equals zero when the distributions are identical.

³² L_2 regularization penalizes the squared magnitude of parameters, $c\|\theta\|^2$, analogous to ridge regression in econometrics.

³³This is the Contraction Mapping Theorem. The identical mathematical structure governs convergence of value function iteration in consumption-savings models, competitive equilibrium computation, and Bellman equation solution.

³⁴Picard iteration is $x_{k+1} = f(x_k)$ for finding roots of $x = f(x)$. When f is a contraction, convergence is geometric. The rate γ means each iteration reduces the error by a fixed proportion; more patient agents (higher γ) face slower convergence because the operator contracts less per step.

4.1.2 Policy Iteration as Newton’s Method

Policy iteration takes a different approach. At the current value estimate $\tilde{V}$, define the greedy policy $\tilde{\pi}(s) = \operatorname{argmax}_a \{r(s, a) + \gamma \sum_{s'} P(s'|s, a) \tilde{V}(s')\}$. The *policy evaluation* step solves the linear fixed-point equation $V = T^{\tilde{\pi}}V$ exactly, where $T^{\tilde{\pi}}$ is the policy-specific Bellman operator

$$(T^{\tilde{\pi}}V)(s) = r(s, \tilde{\pi}(s)) + \gamma \sum_{s'} P(s'|s, \tilde{\pi}(s))V(s'). \quad (29)$$

The geometric structure, formalized by [Puterman and Brumelle \(1979\)](#), is as follows: the linear operator $T^{\tilde{\pi}}$ is a supporting hyperplane to the nonlinear operator T at the current iterate.³⁵ Specifically, the operators satisfy tangency: $T^{\tilde{\pi}}\tilde{V} = T\tilde{V}$, so the linearization agrees with the nonlinear operator at the current iterate. They also satisfy support: $T^{\tilde{\pi}}V \leq TV$ for all V , meaning the linear operator lies weakly below the nonlinear one everywhere, just as a tangent line lies below a convex function. Policy evaluation solves for the fixed point of this linearization exactly. This is precisely the structure of Newton’s method; linearize the nonlinear equation at the current point, solve the linearized system, and iterate.^{36,37}

The consequence is finite termination. Since there are at most $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies and each PI step strictly improves the value function (unless the current policy is already optimal), PI reaches the exact optimum in finitely many iterations. While VI requires $k = \log(100)/\log(1/\gamma)$ iterations to reduce error by a factor of 100 ([Bertsekas, 1996](#)), PI typically converges in 5–10 iterations regardless of γ .^{38,39} At $\gamma = 0.90$, VI needs 44 iterations while PI needs only 5–8; at $\gamma = 0.95$, VI requires 90 versus 5–8; at $\gamma = 0.99$, VI needs 459 iterations while PI still converges in 5–10.

[Bertsekas \(2022b\)](#) extends this interpretation to a broad class of dynamic programming problems. The Newton structure applies whenever the Bellman operator can be written as a pointwise maximum over linear operators: $T = \max_{\pi} T^{\pi}$. This includes not only infinite horizon problems with discounting but also optimal stopping problems (job search, option exercise), average cost optimization (inventory, queueing), and minimax formulations for adversarial settings.⁴⁰ The practical implication is that algorithms with policy-improvement structure (evaluate a policy exactly or approximately, then improve) inherit Newton-like convergence behavior,

³⁵A supporting hyperplane to a convex function at x_0 is a linear function ℓ with $\ell(x_0) = f(x_0)$ and $\ell(x) \leq f(x)$ everywhere. In plainer terms: $T^{\tilde{\pi}}$ is the tangent-line approximation to T from elementary calculus, extended to function spaces. The policy operator $T^{\tilde{\pi}}$ plays this role for the Bellman operator.

³⁶The Newton interpretation of policy iteration has precursors in [Kleinman \(1968\)](#) for Riccati equations in linear-quadratic control and [Pollatschek and Avi-Itzhak \(1969\)](#) for stochastic games.

³⁷Algebraically: consider finding the root of $G(V) = V - TV = 0$. The Bellman operator T is piecewise affine, not smooth: T is affine on each region where the greedy policy is constant, with kinks at boundaries where the optimal action switches. This makes G a semismooth function in the sense of [Qi and Sun \(1993\)](#). At any iterate V_k where the greedy policy $\tilde{\pi}$ is unique (a generic condition), T is locally affine: $TV = r^{\tilde{\pi}} + \gamma P^{\tilde{\pi}}V$, so $G'(V_k) = I - \gamma P^{\tilde{\pi}}$. The Newton step $V_{k+1} = V_k - [G'(V_k)]^{-1}G(V_k) = (I - \gamma P^{\tilde{\pi}})^{-1}r^{\tilde{\pi}}$ is exactly the policy evaluation solution. At the non-smooth boundary points where two actions tie, any element of the B-subdifferential yields the same iterate because the two candidate linearizations produce the same fixed point.

³⁸[Ye \(2011\)](#) proves PI is strongly polynomial with iteration count $O\left(\frac{|\mathcal{S}||\mathcal{A}|}{1-\gamma} \log \frac{|\mathcal{S}|}{1-\gamma}\right)$, resolving a long-standing conjecture. This bound is for fixed γ ; [Fearnley \(2010\)](#) constructs examples requiring exponentially many iterations when γ is allowed to vary with $|\mathcal{S}|$.

³⁹For continuous-state problems discretized on a grid (the norm in economics), the finite-termination argument still applies to the discretized problem, but the number of grid points n enters the bound. [Santos and Rust \(2004\)](#) establish a three-tier convergence result for PI applied to discretized dynamic programs: order ≈ 1.5 globally for general interpolation schemes, quadratic convergence locally when the value function approximation is concave and piecewise linear, and superlinear convergence for general smooth interpolation. The formal error constants $C(h)$ in their quadratic bound degrade as the grid mesh $h \rightarrow 0$, but the iteration count is empirically independent of grid size.

⁴⁰These problems appear under different names such as “stochastic shortest path” for optimal stopping, “average-cost MDP” for long-run average optimization, and “model predictive control” (or receding-horizon control) for finite-horizon replanning.

while pure value-iteration methods (apply T directly) are limited to linear convergence.⁴¹

4.1.3 Simulation Study: The Brock–Mirman Economy

The Brock and Mirman (1972) optimal growth model provides a concrete demonstration. A planner chooses capital k' to maximize $\sum_{t=0}^{\infty} \beta^t \log(c_t)$ subject to the resource constraint $c_t + k_{t+1} = z_t k_t^\alpha$, where productivity $z_t \in \{0.9, 1.1\}$ follows a Markov chain with persistence 0.8. I set $\alpha = 0.36$, $\beta = 0.96$, and discretize capital on a 500-point grid covering two productivity states (1,000 state-action pairs). This model admits the closed-form policy $k'(k, z) = \alpha\beta z k^\alpha$, providing an exact benchmark.

The discretized model makes the PI–Newton equivalence concrete. Define the Bellman residual $G(V) = V - TV$ on $\mathbb{R}^n$ where $n = 1,000$ (500 capital grid points $\times$ 2 productivity states). At iterate V_k , let π_k denote the greedy policy. Since π_k is unique at V_k (generically), T is locally affine: $TV = r^{\pi_k} + \gamma P^{\pi_k} V$, so the residual becomes $G(V) = (I - \gamma P^{\pi_k})V - r^{\pi_k}$ with Jacobian $G'(V_k) = I - \gamma P^{\pi_k}$. The Newton update is

$$V_{k+1} = V_k - [G'(V_k)]^{-1} G(V_k) = (I - \gamma P^{\pi_k})^{-1} r^{\pi_k}, \quad (30)$$

which is exactly the policy evaluation step: solving $V = r^{\pi_k} + \gamma P^{\pi_k} V$ for V . Each PI iteration is one Newton step on the Bellman residual, explaining the 11-iteration convergence on a 1,000-state problem.⁴²

The VI iteration count follows from the contraction bound. Each iteration reduces the Bellman residual $\|V_{k+1} - V_k\|_\infty$ by the factor $\beta = 0.96$: a 4% reduction per step. With initial residual $\|TV_0 - V_0\|_\infty$ determined by the model primitives and tolerance $\epsilon = 10^{-10}$, the required iteration count is $k = \lceil \log(\epsilon/\|TV_0 - V_0\|_\infty) / \log \beta \rceil = 567$. At $\beta = 0.99$, the weaker contraction yields approximately four times as many iterations for the same tolerance, because $\log(0.96)/\log(0.99) \approx 4$.

Figure 2(a)–(b) provide a geometric interpretation for a scalar Bellman equation with three policies. Each policy operator $T^{\pi_i}V = r^{\pi_i} + \gamma_i V$ is affine; the Bellman operator $T = \max_i T^{\pi_i}$ is their upper envelope, a convex piecewise-linear function. Panel (a) shows VI: the staircase iterates $V_{k+1} = TV_k$ by alternating between the T curve and the 45° line, converging at the linear rate γ . Panel (b) shows PI: at each iterate, the algorithm identifies the active policy operator and solves for its fixed point on the 45° line, jumping directly to the intersection. This is a Newton step, where each T^{π_k} is a supporting hyperplane to T at V_k , and the fixed point of the linearization is the Newton iterate. The scalar picture extends to $\mathbb{R}^n$: the affine operator $T^{\pi_k}V = r^{\pi_k} + \gamma P^{\pi_k} V$ supports T at V_k , and its fixed point $(I - \gamma P^{\pi_k})^{-1} r^{\pi_k}$ is the Newton iterate from equation (30). Finite termination follows because T has finitely many affine pieces; the iteration count depends on the number of policy switches, not the state-space dimension.

Table 2 and Figure 2 report three findings. First, VI converges at rate $\beta^n = 0.96^n$ and requires 567 iterations, matching the Banach contraction bound exactly. PI converges in 11 iterations, a 50 $\times$ reduction that the Newton interpretation predicts. Second, the Manne (1960) LP formulation, $\min_V \sum_s V(s)$ subject to $V(s) \geq r(s, a) + \gamma \sum_{s'} P(s'|s, a)V(s')$, recovers the

⁴¹Blackwell (1965) proves that for discounted MDPs with finite state and action spaces, a stationary deterministic policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ exists that is optimal for all initial states simultaneously. This “uniform optimality” has three implications: (1) the search space reduces from history-dependent or stochastic policies to static maps, justifying neural networks that condition only on current state; (2) the optimal policy is independent of the initial distribution d_0 , so changing where episodes start does not require retraining; (3) PI and VI are guaranteed to converge to the same globally optimal policy regardless of initialization.

⁴²Santos and Rust (2004) is the definitive reference on PI convergence for discretized economic models of precisely this type. Their analysis of the Brock–Mirman growth model establishes that PI iteration counts are empirically independent of grid resolution (7–11 iterations across all grid sizes in Table 2), consistent with the Newton interpretation: Newton’s method converges in a number of steps determined by the nonlinearity of the operator, not the dimension of the discretization.

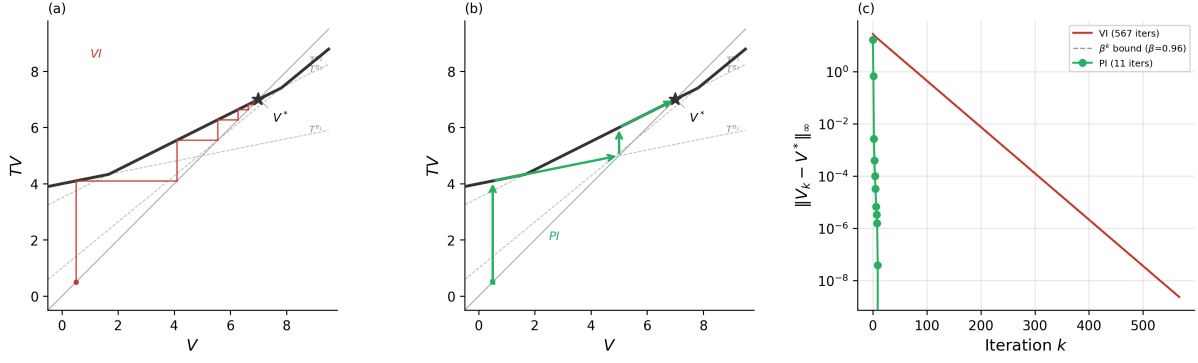


Figure 2: The Brock–Mirman economy ($\alpha = 0.36$, $\beta = 0.96$, 1,000 states). (a) Value iteration on a scalar Bellman equation. The staircase iterates $V_{k+1} = TV_k$, converging at the linear rate γ . (b) Policy iteration as Newton’s method. Each step solves for the fixed point of the active policy operator T^{π_k} , jumping to the tangent line’s intersection with the diagonal. (c) Sup-norm error $\|V_k - V^*\|_\infty$ for the discretized model; VI requires 567 iterations, PI converges in 11.

Table 2: Brock–Mirman Economy: VI vs PI vs LP

Regime	Method	Iterations	Time (s)	$\ V - V^*\ _\infty$
1. Contraction	VI	567	32.32	9.7e-11
	PI	11	0.73	0.0e+00
2. LP dual	VI	—	0.005	—
	LP	—	0.015	2.3e-09
3. Rate ($n_k=10$)	VI	567	0.003	—
	PI	7	0.000	—
3. Rate ($n_k=200$)	VI	567	2.152	—
	PI	10	0.047	—

same value function to machine precision ($\|V_{LP} - V_{VI}\|_\infty < 10^{-8}$), with the dual occupation measures $\mu(s, a)$ concentrated at the stochastic steady state. Third, PI wall-clock time scales favorably: at $n_k = 200$ grid points, PI finishes in 0.057 seconds versus VI’s 2.4 seconds, because the $O(n^3)$ per-iteration cost of policy evaluation is offset by 7–10 total iterations versus 567.

4.2 Value Learning Methods

4.2.1 Stochastic Approximation Foundations

When P is unknown, a single sampled transition (s, a, r, s') can replace the expectation $\mathbb{E}_{s' \sim P(\cdot|s,a)}[V(s')]$. The mathematical foundation is stochastic approximation, developed by Robbins (1952).⁴³ Consider the problem of finding x^* such that $g(x^*) = 0$, where g cannot be evaluated directly but one can observe noisy samples $g(x) + \epsilon$. The Robbins-Monro iteration is:

$$x_{t+1} = x_t - \alpha_t [g(x_t) + \epsilon_t], \quad (31)$$

where ϵ_t is zero-mean noise. Under two conditions on the step sizes, this converges to x^* with probability one. The conditions are $\sum_{t=0}^{\infty} \alpha_t = \infty$ (sufficient exploration, ensuring that learning never ceases) and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ (diminishing noise, ensuring the variance of cumulative updates

⁴³The modern theory of stochastic approximation, including convergence rates and the ODE (ordinary differential equation) method for analyzing iterates, is developed in Kushner and Clark (1978) and Borkar and Meyn (2000). The ODE method shows that the expected trajectory of the stochastic iterates tracks the solution of a deterministic differential equation $\dot{x} = -g(x)$, providing stability conditions via Lyapunov theory.

is finite). The canonical choice $\alpha_t = 1/(t + 1)$ satisfies both conditions.

4.2.2 Q-Learning and SARSA

Q-learning (Watkins and Dayan, 1992) is Robbins-Monro applied to the Bellman equation for action-value functions.⁴⁴ Define the Q-factor Bellman operator:

$$(FQ)(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a'). \quad (32)$$

The Q-learning update, upon observing transition (s_t, a_t, r_t, s_{t+1}) , is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \quad (33)$$

The logic of Q-learning is best understood as a Monte Carlo approximation of the Bellman contraction. The true Bellman operator involves an integral over the transition distribution, $(FQ)(s, a) = r + \gamma \int \max_{a'} Q(s', a') dP(s'|s, a)$. Since P is unknown, this integral cannot be computed analytically. However, a sample transition (s, a, r, s') acts as a single-point Monte Carlo estimate of this integral. The Q-learning update is simply an exponential moving average (with weight α_t) between the current estimate and this noisy Monte Carlo target. Because F is a γ -contraction in the supremum norm, the expected update drives the estimate toward the fixed point Q^* , provided the noise in the Monte Carlo sample averages out over time (which the Robbins-Monro conditions ensure) (Tsitsiklis, 1994).⁴⁵

Convergence requires two conditions. Exploration (visiting all state-action pairs infinitely often) ensures identification; every state-action pair must be sampled to learn its value. The Robbins-Monro step-size conditions ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$) balance tracking versus noise suppression. Watkins and Dayan (1992) and Jaakkola et al. (1994) formalize these; Tsitsiklis (1994) provides the general framework.

The choice of step-size schedule has quantitative consequences: Even-Dar and Mansour (2003) show that polynomial schedules $\alpha_t = 1/t^\omega$ with $\omega \in (1/2, 1)$ achieve convergence rate $O(1/t^{1-\omega})$, creating an explicit tradeoff between speed and stability. Recent work by Li et al. (2024a) establishes that Q-learning with variance-reduced updates achieves minimax-optimal sample complexity $\tilde{O}(|\mathcal{S}||\mathcal{A}|/(1-\gamma)^3\epsilon^2)$, matching information-theoretic lower bounds.⁴⁶ Model-free learning is possible. The optimal value function can be found without ever estimating the transition probabilities.⁴⁷

SARSA provides an on-policy variant.⁴⁸ Instead of taking the maximum over next actions,

⁴⁴The “Q” in Q-learning stands for “quality,” following Watkins and Dayan (1992), who used $Q(s, a)$ to denote the quality (expected return) of taking action a in state s . The term “Q-factor” is used interchangeably with “action-value function” throughout the RL literature.

⁴⁵Q-learning can be viewed as root-finding for the expected Bellman residual: the goal is to find parameters such that $\mathbb{E}_{(s,a,r,s')}[\delta_t] = 0$, where $\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ is the temporal difference error. The update is a stochastic gradient step on the mean-squared Bellman error, but with a critical distinction: the gradient is “semi-gradient” because the target $\max_{a'} Q(s', a')$ is treated as a fixed constant rather than a function of the parameters being updated. This simplifies computation but disconnects the update from true gradient descent, requiring the specific stability conditions of Tsitsiklis (1994).

⁴⁶Vanilla Q-learning (without variance reduction) has tight complexity $\tilde{O}(|\mathcal{S}||\mathcal{A}|/(1-\gamma)^4\epsilon^2)$, worse by a factor of $1/(1-\gamma)$ due to maximization bias inflating variance. The optimal cubic rate requires variance-reduced updates (Wainwright, 2019; Sidford et al., 2018). By contrast, naive model-based RL (estimate $\hat{P}$ from samples, then solve by planning) achieves the optimal $(1-\gamma)^{-3}$ rate with no special tricks (Agarwal et al., 2020b), illustrating the statistical cost of discarding transition structure.

⁴⁷Model-free methods are essential when (a) the environment is a physical system with dynamics too complex to write down, or (b) the agent learns directly from interaction. Note that “model-free” does not mean “atheoretical.” The agent does not store $P(s'|s, a)$ explicitly, but the Q-function serves as an implicit model encoding long-run consequences.

⁴⁸SARSA is named for the quintuple $(S_t, A_t, R_t, S_{t+1}, A_{t+1})$ used in each update (Rummery and Niranjan,

SARSA uses the action actually taken:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]. \quad (34)$$

This solves for the value function of the behavior policy π rather than the optimal policy. Singh et al. (2000) prove convergence under the same step-size conditions, provided the behavior policy converges to a stationary distribution. Q-learning is noisy value iteration on Q-factors; SARSA is noisy *policy evaluation*.⁴⁹ The Robbins-Monro conditions ensure that the noise averages out faster than the signal decays, allowing asymptotic convergence despite using only single-sample estimates.

4.2.3 Fitted Value Iteration and Fitted Q-Iteration

Tabular Q-learning maintains a separate entry for every state-action pair. When $|\mathcal{S}|$ is large or the state space is continuous, as in most economic applications, this is infeasible. *Fitted Q-Iteration* (FQI) (Ernst et al., 2005) replaces the tabular update with a supervised regression step: given a batch of transitions, fit a function approximator to the Bellman targets. This subsection covers the batch formulation and its finite-sample error theory.

Definition D1 (Fitted Q-Iteration). *Let $\mathcal{F}$ be a function class mapping $\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. Initialize $Q_0 \equiv 0$. At each iteration $k = 0, 1, \dots, K - 1$:*

1. *Draw N transitions (s_i, a_i, r_i, s'_i) from a generative model.*
2. *Construct regression targets $y_i^{(k)} = r_i + \gamma \max_{a'} Q_k(s'_i, a')$.*
3. *Set $Q_{k+1} \leftarrow \arg \min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N (f(s_i, a_i) - y_i^{(k)})^2$.*

Return the greedy policy $\pi_K(s) = \arg \max_a Q_K(s, a)$.

The regression step replaces the exact Bellman application with a projection onto $\mathcal{F}$: $Q_{k+1} = \Pi_{\mathcal{F}} \mathcal{T} Q_k$, where $\mathcal{T}$ is the Bellman optimality operator and $\Pi_{\mathcal{F}}$ is the L^2 -projection under the sample distribution. Fitted Value Iteration (FVI) (Munos and Szepesvári, 2008) applies the same idea to the value function directly: $V_{k+1} = \Pi_{\mathcal{F}} \mathcal{T}^* V_k$, where $(\mathcal{T}^* V)(s) = \max_a \{r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')\}$.

Linear case. With feature matrix $\Phi \in \mathbb{R}^{|\mathcal{S}| \times d}$ (rows $\phi(s)^\top$) and per-action weight vectors $\theta_a \in \mathbb{R}^d$, each FQI regression step for action a reduces to the normal equations:

$$\theta_a^{(k+1)} = (\Phi^\top \Phi)^{-1} \Phi^\top y_a^{(k)}, \quad (35)$$

where $y_a^{(k)}(s) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} \phi(s')^\top \theta_{a'}^{(k)}$. Computation is $O(d^2 |\mathcal{S}| + d^3)$ per action per iteration. The FVI update takes the same form with a single weight vector θ_V :

$$\theta_V^{(k+1)} = (\Phi^\top \Phi)^{-1} \Phi^\top V_{\text{target}}^{(k)}, \quad (36)$$

where $V_{\text{target}}^{(k)}(s) = \max_a \{r(s, a) + \gamma \sum_{s'} P(s'|s, a) \phi(s')^\top \theta_V^{(k)}\}$.

Error propagation. Define the inherent Bellman approximation error

$$\varepsilon_{\text{approx}} = \inf_{f \in \mathcal{F}} \|\mathcal{T}f - f\|_{p, \mu}, \quad (37)$$

1994). Convergence requires the GLIE (Greedy in the Limit with Infinite Exploration) condition: the behavior policy must explore all actions infinitely often while converging to a greedy policy. ε -greedy with $\varepsilon_t \rightarrow 0$ satisfies this.

⁴⁹Bhandari et al. (2021) provide finite-time analysis of TD learning, showing that the convergence rate depends on the mixing time of the Markov chain under the behavior policy. Faster mixing (less serial correlation in the state sequence) yields faster convergence.

the smallest residual achievable when the Bellman operator maps any element of $\mathcal{F}$ back to itself. Munos and Szepesvári (2008) show that after K iterations with N i.i.d. samples per iteration,

$$\|V_K - V^*\|_{p,\rho} \leq C_{\rho,\mu} \left[\gamma^K \|V_0 - V^*\|_{p,\rho} + \frac{\varepsilon_{\text{approx}}}{(1-\gamma)^2} + O\left(\frac{1}{\sqrt{N}}\right) \right], \quad (38)$$

where $C_{\rho,\mu}$ is a *concentrability coefficient* bounding the ratio of future-state distributions under the evaluation distribution ρ relative to the data distribution μ .⁵⁰ Three terms drive the error: the geometric decay γ^K (initialization bias), the approximation error $(1-\gamma)^{-2}\varepsilon_{\text{approx}}$ (bias from function class), and the estimation error $O(1/\sqrt{N})$ (variance from finite samples). When $\mathcal{F}$ contains V^* exactly, $\varepsilon_{\text{approx}} = 0$ and the bound recovers exact convergence as $K \rightarrow \infty$. The $(1-\gamma)^{-2}$ amplification, one factor of $(1-\gamma)^{-1}$ more than in tabular Q-learning, reflects error accumulation across approximate DP steps: each regression step introduces bias, and this bias compounds over K iterations.

FVI versus FQI under linear approximation. Both algorithms solve projected Bellman equations. When $V^* \in \text{span}(\Phi)$ exactly, FVI converges to V^* in a single projected iteration, since the normal equations (36) recover θ_V^* satisfying $\Phi\theta_V^* = V^*$. For FQI, the per-action Q-functions satisfy $Q^*(s, a^*(s)) = V^*(s)$ at the optimal action $a^*(s)$, so when $Q^*(\cdot, a)$ is also representable in $\text{span}(\Phi)$ for each a , FQI recovers consistent value estimates $\phi(s)^\top \theta_{a^*(s)}^* = \phi(s)^\top \theta_V^*$ at convergence. In practice, with a finite-dimensional feature class, $Q^*(\cdot, a)$ typically has a more complex shape than V^* , as the Q-function varies with both state and the specific level of investment, so FVI achieves lower approximation error than FQI under the same linear basis, as the simulation below illustrates.

4.2.4 Simulation Study: Fitted Methods on the Brock–Mirman Economy

The Brock–Mirman stochastic growth model (Brock and Mirman, 1972) provides a test case where both the optimal value function and the approximation error are analytically tractable. The economy has capital grid $\{k_1, \dots, k_{50}\}$ and two productivity states $z \in \{0.9, 1.1\}$, giving $|\mathcal{S}| = 100$ states and $|\mathcal{A}| = 50$ actions. The optimal value function has the closed-form log-linear structure $V^*(k, z) = A_z + B \log k$ with $B = \alpha/(1-\alpha\beta) \approx 0.550$. We use basis functions $\phi(k, z) = [1, \log k, k/\bar{k}, (k/\bar{k})^2, (k/\bar{k})^3] \otimes [\mathbb{1}_{z=z_\ell}, \mathbb{1}_{z=z_h}] \in \mathbb{R}^{10}$, which include the $\log k$ term and thus contain V^* in $\text{span}(\Phi)$ up to residual $\|\Pi_\Phi V^* - V^*\|_\infty = 0.0002$. Both FVI and FQI use the full transition matrix (no sampling noise), isolating approximation error from estimation error.

Table 3 reports feature weights and convergence metrics. FVI converges in 341 iterations to $\|V_{\text{FVI}} - V^*\|_\infty = 0.001$, with the $\log k$ coefficient $\hat{\theta}_V^{\log k} = 0.551$ matching the theoretical $B = 0.550$. FQI converges in 339 iterations but achieves $\|V_{\text{FQI}} - V^*\|_\infty = 1.65$; the per-action normal equations fit $Q^*(\cdot, a)$ globally over all feasible states, most of which have action a as sub-optimal, so the learned weights do not capture the value structure at the optimum. The convergence curves in Figure 3 show both methods reducing error monotonically, confirming that the projected Bellman iteration is stable under the log-polynomial basis.

⁵⁰The concentrability coefficient $C_{\rho,\mu}$ measures how well the data distribution μ covers future states reachable under optimal policies from ρ . When μ is the state-action distribution of the optimal policy itself, $C_{\rho,\mu} = 1$. Distribution mismatch, common when batch data comes from a sub-optimal behavior policy, inflates $C_{\rho,\mu}$ and worsens the bound. Antos et al. (2008) extend these results to continuous action spaces and single-trajectory data.

Basis feature	FVI $\hat{\theta}_V$	FQI $\hat{\theta}_{a^*(z_\ell)}$	FQI $\hat{\theta}_{a^*(z_h)}$
$z = 0.9, k^0$	-25.1796	39.0394	2.2630
$z = 0.9, \log k$	0.5512	17.5316	7.9759
$z = 0.9, k/\bar{k}$	-0.0087	-105.1561	-39.5636
$z = 0.9, (k/\bar{k})^2$	0.0103	100.4176	34.6760
$z = 0.9, (k/\bar{k})^3$	-0.0046	-39.4090	-12.8715
$z = 1.1, k^0$	-24.4575	-8.4938	-1.7315
$z = 1.1, \log k$	0.5508	4.4731	6.2144
$z = 1.1, k/\bar{k}$	-0.0062	-24.5224	-37.0297
$z = 1.1, (k/\bar{k})^2$	0.0075	24.0981	37.1908
$z = 1.1, (k/\bar{k})^3$	-0.0034	-9.7268	-15.1914

Table 3: Feature weights for FVI ($\hat{\theta}_V$) and FQI ($\hat{\theta}_{a^*}$ at the modal optimal action for each productivity level). The FVI $\log k$ coefficients (0.551) recover the theoretical value $B = \alpha/(1 - \alpha\beta) = 0.550$. FQI per-action weights are dominated by fitting $Q^*(\cdot, a)$ over all feasible states rather than at the optimum, yielding large oscillatory coefficients.

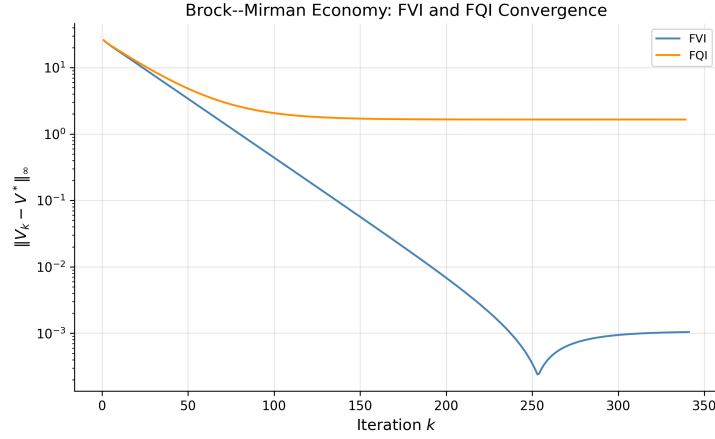


Figure 3: Convergence of FVI and FQI on the Brock–Mirman economy. Both methods reduce $\|V_k - V^*\|_\infty$ monotonically. FVI achieves final error 0.001 (near the best-in-class error 0.0002 for this basis), while FQI stalls at 1.65 due to per-action approximation mismatch.

4.2.5 Rollout, Lookahead, and AlphaZero

Two constructions bridge the Newton interpretation of Section 4.1 to practical algorithms. Given a base policy μ with value function V^μ , the *rollout* policy selects

$$\mu_R(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}} \left\{ r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\mu(s') \right\}. \quad (39)$$

This is one step of policy iteration starting from μ : the policy improvement theorem guarantees $V^{\mu_R}(s) \geq V^\mu(s)$ for all s , with strict inequality unless μ is already optimal (Bertsekas, 2021).⁵¹ Given an arbitrary approximate value function $\tilde{V}$ (not necessarily the value of any policy), the *one-step lookahead* policy selects

$$\tilde{\pi}(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}} \left\{ r(s, a) + \gamma \sum_{s'} P(s'|s, a) \tilde{V}(s') \right\}. \quad (40)$$

⁵¹Bertsekas uses cost-minimization notation throughout his work, writing $\min$ where standard RL uses $\max$ and defining value as accumulated cost rather than reward. I translate to the reward-maximization convention used elsewhere in this chapter; the mathematics are equivalent with reversed inequalities.

When $\tilde{V} = V^\mu$, lookahead and rollout coincide. The distinction matters because rollout inherits the monotone improvement guarantee (it starts from the value of a policy), while lookahead from an arbitrary $\tilde{V}$ has no such monotonicity. The Newton interpretation from Section 4.1 explains why lookahead nevertheless helps: both constructions solve the linearized Bellman equation at the current iterate.⁵²

An ℓ -step lookahead extends this by applying $(\ell - 1)$ steps of value iteration before the final greedy selection. The first $(\ell - 1)$ steps are ordinary Bellman contractions, each shrinking the approximation error by a factor of γ . Only the final step, the greedy policy improvement, constitutes the Newton step.⁵³ The resulting error bound is

$$\|V^{\tilde{\pi}} - V^*\|_\infty \leq \gamma^\ell \|\tilde{V} - V^*\|_\infty, \quad (41)$$

where the γ^ℓ factor reflects ℓ total contractions (Bertsekas, 2022a, Prop. 2.3.1). Deep lookahead compensates for poor approximation through repeated contraction, not through repeated Newton steps.

Recall the AlphaGo Zero system from Section 3.2.4, where a neural network $f_\theta(s) = (\mathbf{p}, v)$ outputs a prior policy $\mathbf{p}$ and a value estimate v for any board position s . During play, the network does not act alone: Monte Carlo Tree Search runs simulated games from the current position, using v to evaluate leaf nodes and $\mathbf{p}$ to guide which branches to explore.⁵⁴ The network provides $\tilde{V}$; MCTS applies multi-step lookahead through selective tree expansion. Table 4 makes the correspondence explicit.

Step	Policy Iteration	AlphaZero
Initialize	Arbitrary π_0	Random network f_θ
Evaluate	Solve $V = T^{\pi_k} V$ exactly	Network value head $v \approx V^{\pi_k}$
Improve	$\pi_{k+1} = \operatorname{argmax}_a \{r + \gamma P V^{\pi_k}\}$	MCTS simulations from v
Iterate	Repeat until π is stationary	Retrain f_θ on self-play outcomes

Table 4: Policy iteration and AlphaZero follow the same evaluate-improve loop. The network provides approximate policy evaluation; MCTS provides approximate policy improvement via selective tree search.

The gap between network-only play and network-plus-search is the contraction factor γ^H , where H is the effective search depth. The network provides a rough starting point $\tilde{V}$; MCTS applies the Bellman operator through deep lookahead, shrinking the approximation error by γ per level of search.⁵⁵

4.3 The Central Challenge: The Deadly Triad

The preceding results assume tabular representations, namely, a separate value for each state-action pair. In practice, state spaces are too large for lookup tables (Go has 10^{170} states; most economic models have continuous state variables). Practitioners must therefore combine three ingredients: *function approximation* (to handle large state spaces), bootstrapping (to learn from single transitions rather than complete episodes), and off-policy learning (to learn about the

⁵²Rollout requires a simulator (generative model) that can be queried from arbitrary states, a stronger assumption than the trajectory-based access of Q-learning and SARSA.

⁵³Bertsekas (2021) states this explicitly: “whatever follows the first step of the lookahead is preparation for the Newton step.” The preceding value iteration steps have linear convergence at rate γ ; only the terminal improvement step has superlinear character.

⁵⁴AlphaGo Zero uses 1,600 MCTS simulations per move for Go; the generalized AlphaZero algorithm uses 800 simulations per move across chess, shogi, and Go (Silver et al., 2018, Table S3).

⁵⁵MCTS adds UCB exploration and selective tree expansion beyond the literal lookahead framework. The Newton interpretation explains why lookahead helps at all, namely, the final greedy selection over the search tree is a policy improvement step. It does not explain the specific mechanisms (upper confidence bounds, progressive widening) that make MCTS computationally efficient.

optimal policy while exploring, or to reuse old data). Each is desirable in isolation. Their interaction, known as the *deadly triad* (Sutton and Barto, 2018, Ch. 11), is the central open problem in reinforcement learning theory.

4.3.1 The Projected Bellman Operator

With function approximation $V(s) \approx \phi(s)^\top \theta$, the parameter vector θ is shared across states. Updating θ to improve the value estimate at one state simultaneously changes the estimate at every other state. The algorithm can no longer apply the Bellman operator T^π to each state independently; instead, it applies T^π to compute a target, then *projects* the result back onto the function space (the span of the features ϕ). The composed operator is ΠT^π , the *projected Bellman operator*, where Π denotes this projection (Tsitsiklis and Van Roy, 1997). Convergence of the approximate iteration $\theta_{k+1} = \Pi T^\pi \theta_k$ requires ΠT^π to be a contraction.

In the on-policy setting, Π minimizes squared error weighted by d^π , the stationary distribution of the policy being evaluated, so $\Pi V = \arg \min_{\hat{V} \in \text{span}(\Phi)} \|V - \hat{V}\|_{d^\pi}^2$, where $\|V\|_{d^\pi}^2 = \sum_s d^\pi(s) V(s)^2$. The Bellman operator T^π is a γ -contraction in the same d^π -norm. Because both operators use the same norm, the projection Π is *orthogonal*, meaning the residual $V - \Pi V$ is perpendicular to the approximation subspace. The Pythagorean theorem then gives $\|\Pi V\|_{d^\pi}^2 + \|V - \Pi V\|_{d^\pi}^2 = \|V\|_{d^\pi}^2$, so $\|\Pi V\|_{d^\pi} \leq \|V\|_{d^\pi}$.⁵⁶ The projection cannot expand distances. The composition therefore contracts:

$$\|\Pi T^\pi V_1 - \Pi T^\pi V_2\|_{d^\pi} \leq \underbrace{\|\Pi\|}_{\leq 1} \cdot \underbrace{\|T^\pi V_1 - T^\pi V_2\|_{d^\pi}}_{\leq \gamma \|V_1 - V_2\|_{d^\pi}} < \|V_1 - V_2\|_{d^\pi}. \quad (42)$$

A unique fixed point $\Phi\theta^*$ exists, and TD(λ) converges to it with probability one. The resulting approximation error satisfies $\|\Phi\theta^* - V^\pi\|_{d^\pi} \leq \frac{1-\lambda\gamma}{\sqrt{1-\gamma^2}} \|\Pi V^\pi - V^\pi\|_{d^\pi}$, bounding the TD solution’s error by a multiple of the best possible approximation error (Tsitsiklis and Van Roy, 1997, Theorem 1).

4.3.2 Why Off-Policy Learning Diverges

In the off-policy setting, samples come from a behavior distribution $\mu \neq d^\pi$. The projection now minimizes error under μ , but the Bellman operator T^π still contracts in the d^π -norm. The two operators measure distance in different norms. The projection is no longer orthogonal in the d^π -norm; it is *oblique*. Unlike orthogonal projections, oblique projections can expand distances, with $\|\Pi_\mu\|_{d^\pi}$ exceeding 1 in the worst case. If $\|\Pi_\mu\|_{d^\pi} > 1/\gamma$, the expansion from projection overwhelms the γ -contraction from the Bellman operator, and the fixed-point iteration diverges.

This divergence is not overfitting. Overfitting occurs when the approximator memorizes training data at the expense of generalization; collecting more data helps. Divergence means the parameter vector θ grows without bound, producing arbitrarily large value estimates that bear no relation to the true values. More data does not help; the algorithm itself is unstable. The distinction matters because the remedies are entirely different. Regularization and early stopping address overfitting, while the deadly triad requires structural changes to the algorithm.

Baird (1995) constructed a six-state star MDP that makes this failure concrete. All rewards are zero, so the true value is $V^*(s) = 0$ for every state. A lookup table learns this immediately. The MDP has a star topology: states 1 through 5 each transition to state 6, and state 6 transitions to itself. Linear function approximation uses a shared weight w_1 across all states plus a state-specific weight, with $V(s) = 2w_1 + w_s$ for $s \in \{1, \dots, 5\}$ and $V(6) = 2w_1 - w_6$.

⁵⁶The same argument holds in $\mathbb{R}^n$. Projecting a vector onto a subspace never makes it longer. This is the geometric content of the Cauchy-Schwarz inequality. In the function-approximation setting, the “vector” is a value function, the “subspace” is the span of features, and “length” is the d^π -weighted L^2 norm.

Training samples all transitions equally often (uniform distribution, not d^π). The dynamics are as follows.⁵⁷ When $V(6)$ is large and positive, the TD target $\gamma V(6)$ exceeds $V(s)$ for states 1 through 5, producing positive TD errors that push w_1 upward. At state 6, the TD target is $\gamma V(6) < V(6)$, producing a negative TD error that pushes w_1 downward. But states 1 through 5 are each visited as often as state 6, so w_1 receives five upward pushes for every one downward push. The shared weight diverges to $+\infty$. The on-policy distribution would concentrate mass on state 6 (the absorbing state), counterbalancing the upward pressure; uniform sampling destroys this balance.⁵⁸

Each element of the triad is individually necessary for divergence. Without function approximation (tabular), the projection is the identity and Q-learning’s contraction applies directly. Without bootstrapping (Monte Carlo returns), targets are independent of current value estimates and the problem reduces to supervised regression. Without off-policy learning, samples come from d^π , the projection is orthogonal, and the Tsitsiklis-Van Roy convergence guarantee holds.

4.3.3 Resolutions

Three classes of algorithms restore convergence, each neutralizing a different component of the triad.

Target networks weaken bootstrapping. Instead of updating toward $r + \gamma Q(s'; \theta)$, where the target moves with each parameter update, DQN (Mnih et al., 2015) updates toward $r + \gamma Q(s'; \theta^-)$, where θ^- is a slowly-updated copy of the parameters.⁵⁹ The regression target becomes quasi-static, converting the coupled fixed-point problem into a sequence of supervised learning problems. Zhang et al. (2021) prove that this two-timescale scheme converges to a regularized TD fixed point with linear function approximation. Fellows et al. (2023) show that target networks recondition the Jacobian of the TD update: the spectral radius of the composed update operator depends on the target network update frequency k , and for sufficiently large k the spectral radius drops below 1 even in off-policy settings with nonlinear function approximation.

Gradient TD methods fix the projection mismatch. Sutton et al. (2009) reformulate the projected Bellman error as a saddle-point problem $\min_\theta \max_y L(\theta, y)$, yielding algorithms (GTD, GTD2, TDC) that perform true stochastic gradient descent on the mean-squared projected Bellman error.⁶⁰ These methods converge off-policy with linear function approximation because they eliminate the semi-gradient approximation that causes the norm mismatch.

Regularization shrinks the projection operator. Lim and Lee (2024) add an ℓ_2 penalty $-\eta\theta$ to the Q-learning update. This changes the projection from $\Pi = X(X^\top DX)^{-1}X^\top D$ to $\Pi_\eta = X(X^\top DX + \eta I)^{-1}X^\top D$. As the regularization strength η increases, the projection “shrinks” toward the origin. For sufficiently large η , $\gamma\|\Pi_\eta\| < 1$, restoring the contraction

⁵⁷Baird (1995) also presents an MDP variant with two actions per state, demonstrating that Q-learning diverges under the same mechanism. The key insight is identical: shared parameters create cross-state coupling that uniform sampling cannot counterbalance.

⁵⁸The gradient of the mean-squared Bellman error $\|Q - TQ\|^2$ would correct for this imbalance, but computing an unbiased gradient requires two independent next-state samples from the same (s, a) , since $\nabla \mathbb{E}[(Q - \mathbb{E}[r + \gamma V(s')])^2]$ involves the product $\mathbb{E}[\cdot] \cdot \nabla \mathbb{E}[\cdot]$. A single sample yields a biased estimate ($\mathbb{E}[XY] \neq \mathbb{E}[X]\mathbb{E}[Y]$). This “double-sampling” requirement is impractical in most environments (Baird, 1995). Practitioners instead use semi-gradient TD, which treats the bootstrap target as a constant and finds the projected fixed point $\Pi T^\pi \theta^*$ rather than the true Bellman fixed point. It is this semi-gradient structure that makes off-policy TD inherently vulnerable to the projection mismatch.

⁵⁹Experience replay (Lin, 1992) complements target networks by breaking temporal correlation in the training data. The two mechanisms address different sources of instability: target networks stabilize the bootstrap target, while replay stabilizes the sampling distribution.

⁶⁰The saddle-point formulation introduces auxiliary variables y of the same dimension as θ , doubling the parameter count and requiring a second learning rate. Bhandari et al. (2021) provide finite-time convergence rates for these two-timescale algorithms.

property. The algorithm converges to a biased but stable fixed point, with the bias controlled by η .

4.4 Policy Learning Methods

Value-based methods find fixed points of the Bellman operator. Policy-based methods parameterize the policy directly as $\pi_\theta(a|s)$ and maximize expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^{\infty} \gamma^t R_t]$ by gradient ascent. This formulation sidesteps the Bellman equation entirely and frames reinforcement learning as constrained optimization.

4.4.1 The Policy Gradient Theorem

The policy gradient theorem, proved independently by Williams (1992) for the episodic case and Sutton et al. (2000) for the general discounted setting, provides a tractable expression for the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) Q^{\pi_\theta}(s, a)], \quad (43)$$

where $d^{\pi_\theta}(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s | \pi_\theta)$ is the discounted state visitation distribution. The fundamental econometric challenge in optimizing $J(\theta)$ is that this distribution depends on θ through the environment's dynamics. A naive derivative would require $\nabla_\theta d^{\pi_\theta}(s)$, which implies differentiating the unknown transition matrix $P(s'|s, a)$.

The policy gradient theorem sidesteps this entirely via a likelihood ratio (or score function) trick.⁶¹ The theorem transforms a sensitivity analysis problem (how does the system evolve?) into a simpler expectation problem (what is the correlation between the score $\nabla \log \pi$ and the value Q ?). The gradient can be written as an expectation under the current policy, weighted by action-values, without requiring $\nabla_\theta d^{\pi_\theta}$. The transition dynamics $P(s'|s, a)$ do not appear; the gradient is estimable via sample averages from trajectories alone.

4.4.2 REINFORCE and Variance Reduction

REINFORCE (Williams, 1992) is the simplest policy gradient algorithm. Sample a trajectory $(s_0, a_0, r_0, s_1, \dots)$, compute the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ from each time step, and update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t. \quad (44)$$

This is an unbiased estimator of $\nabla_\theta J(\theta)$, but its variance is high because a single trajectory provides a noisy estimate of Q^{π_θ} . Despite high variance, REINFORCE converges to a globally optimal policy in the tabular setting.⁶²

4.4.3 Natural Policy Gradient and Gradient Domination

Standard gradient descent treats all parameter directions equally. But small changes in θ can cause large changes in the policy distribution π_θ . The natural policy gradient (Kakade, 2001), building on the natural gradient framework of Amari (1998), accounts for this curvature by preconditioning with the Fisher information matrix.⁶³ The relationship between NPG and

⁶¹The score function $\nabla_\theta \log \pi_\theta(a|s)$ is the same mathematical object that appears in the Cramér-Rao bound and the score test in maximum likelihood estimation. The policy gradient is a covariance, $\nabla J = \text{Cov}_{d^{\pi_\theta} \times \pi_\theta}(\nabla \log \pi_\theta, Q^{\pi_\theta})$, measuring how sensitive the log-likelihood of the policy is to parameter changes, weighted by action quality.

⁶²The baseline $b(s)$ subtracted from G_t reduces variance while preserving unbiasedness, since $\mathbb{E}[\nabla_\theta \log \pi_\theta(a|s)b(s)] = 0$ for any baseline independent of a .

⁶³The Fisher information $F(\theta) = \mathbb{E}[\nabla \log p \nabla \log p^\top]$ measures curvature of the log-likelihood and appears in the Cramér-Rao bound. Here it measures how policy distributions change with parameters.

standard PG parallels that between GLS and OLS in econometrics: both account for the covariance structure of the estimating equations, yielding efficiency gains when the covariance is non-spherical.

$$\tilde{\nabla}_{\theta} J(\theta) = F(\theta)^{-1} \nabla_{\theta} J(\theta), \quad F(\theta) = \mathbb{E}_{s,a} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s)^{\top} \right]. \quad (45)$$

Why does NPG recover policy iteration? Standard gradient ascent is sensitive to parameterization: it takes the steepest step in Euclidean parameter space, where units depend on how the policy is parameterized. NPG takes the steepest step in distribution space (measured by KL-divergence), which is invariant to reparameterization. In the tabular case, Kakade (2001, Theorem 2) proves that this geometric correction aligns the gradient exactly with the greedy policy $\tilde{\pi}$ from policy iteration. With step size 1 (and exact estimation), NPG performs one full Newton step; with smaller step sizes, it performs damped Newton updates. This explains its rapid convergence: NPG approximates the quadratic convergence of finding a fixed point rather than the linear convergence of hill-climbing.

The RL objective $J(\theta)$ is non-convex in θ . For economists trained to distrust gradient methods on non-convex objectives, the natural concern is convergence to spurious local optima. For softmax policies, this concern is unfounded. The landscape is “benign” in a precise sense. Agarwal et al. (2021a) prove that $J(\theta)$ satisfies a *gradient domination* condition (also called Polyak-Łojasiewicz, or PL). The PL condition has the same functional form as the strong convexity condition for guaranteeing linear convergence of gradient descent, but it applies to non-convex functions: whenever $\|\nabla J(\theta)\|$ is small, the policy must be near-optimal. Formally, the sub-optimality $J(\pi^*) - J(\pi_{\theta})$ is bounded by a constant times $\|\nabla J(\theta)\|^2$. The implication is immediate: any point where the gradient vanishes is globally optimal. The non-convex landscape has no false peaks, no spurious local maxima. Gradient ascent cannot get trapped.

Mei et al. (2020) sharpen this result for softmax parameterization, proving explicit convergence rates.⁶⁴

The Natural Policy Gradient achieves more: *dimension-free* convergence. Standard gradient ascent on $J(\theta)$ has a convergence rate that depends on the smoothness constant, which scales with $|\mathcal{S}|$. NPG circumvents this by preconditioning with the Fisher information matrix $F(\theta)^{-1}$. The mechanism is that the state-visitation distribution $d^{\pi_{\theta}}(s)$ appears in both $\nabla J(\theta)$ and $F(\theta)$; when computing $F^{-1} \nabla J$, these terms cancel analytically. The resulting update rule is equivalent to soft policy iteration and converges at rate $O(1/(1 - \gamma)^2 \epsilon)$, independent of $|\mathcal{S}|$ and $|\mathcal{A}|$ (Xiao, 2022).

4.4.4 Trust Region Methods

NPG requires computing and inverting the Fisher information matrix $F(\theta)$, which scales as $O(d^2)$ in parameters and is impractical for neural networks. TRPO (Schulman et al., 2015) approximates the natural gradient using conjugate gradient methods⁶⁵ without forming F ex-

⁶⁴However, “no spurious local optima” does not imply “easy optimization.” The landscape is dominated by vast plateaus (saddle points) where gradients vanish. Without sufficient exploration, the probability of visiting relevant states decays exponentially with the horizon, rendering the gradient exponentially small. Global convergence requires the starting distribution to have adequate coverage relative to the optimal policy’s visitation distribution, formalized as the “distribution mismatch coefficient” by Agarwal et al. (2021a). The Natural Policy Gradient addresses this by preconditioning with the Fisher Information Matrix, making the update direction covariant: invariant to invertible linear transformations of the parameter space. This standardizes units across parameters, preventing stalling on plateaus caused by poor parameter scaling. Li et al. (2022) make this quantitatively precise: vanilla softmax policy gradient requires iterations doubly exponential in the effective horizon $1/(1 - \gamma)$ because score functions are exponentially small in directions corresponding to suboptimal actions.

⁶⁵Conjugate gradient is an iterative method for solving linear systems $Ax = b$ without forming A explicitly, requiring only matrix-vector products Av . With k iterations it costs $O(kd)$ versus $O(d^3)$ for direct inversion, making it feasible for neural networks with millions of parameters.

plicitly, and enforces trust regions via line search.⁶⁶ Shani et al. (2020) prove convergence for adaptive trust region methods that adjust the constraint radius dynamically. PPO (Schulman et al., 2017) simplifies further by replacing the hard KL constraint with a clipped surrogate objective, trading theoretical guarantees for computational simplicity.

4.5 Hybrid Methods

REINFORCE estimates policy gradients from sample returns (unbiased, high variance); TD methods use bootstrapped targets $r + \gamma V(s')$ (lower variance, biased when V is approximate). Actor-critic methods combine both. The critic estimates the value function, the actor updates the policy using the critic’s estimates.

4.5.1 Actor-Critic Architecture and Two-Timescale Convergence

The theoretical foundation is two-timescale stochastic approximation (Konda and Tsitsiklis, 2000), building on the two-timescale ODE convergence theory of Borkar (1997).⁶⁷ Run two concurrent learning processes:

$$\text{Critic (fast): } \theta_{t+1} = \theta_t + \alpha_t^{(c)} \delta_t \nabla_{\theta} \hat{V}(s_t; \theta_t), \quad (46)$$

$$\text{Actor (slow): } \omega_{t+1} = \omega_t + \alpha_t^{(a)} \delta_t \nabla_{\omega} \log \pi_{\omega}(a_t | s_t), \quad (47)$$

where $\delta_t = r_t + \gamma \hat{V}(s_{t+1}; \theta_t) - \hat{V}(s_t; \theta_t)$ is the TD error. The critic updates the value function parameters θ ; the actor updates the policy parameters ω .

Convergence requires the critic to learn faster than the actor.

$$\lim_{t \rightarrow \infty} \frac{\alpha_t^{(a)}}{\alpha_t^{(c)}} = 0, \quad \text{with both satisfying Robbins-Monro conditions.} \quad (48)$$

Under this separation, the actor sees a quasi-stationary critic: from the actor’s perspective, the critic provides approximately correct value estimates at each step.⁶⁸ The actor’s updates are then approximately unbiased policy gradient steps. Konda and Tsitsiklis (2000) prove convergence to a stationary point of $J(\omega)$ (i.e., $\nabla J(\omega) \rightarrow 0$).

Convergence requires a structural condition on the critic. The critic’s feature vectors must span the actor’s score functions $\nabla_{\omega} \log \pi_{\omega}(a | s)$, so that the critic’s approximation error lies orthogonal to the policy gradient direction. Under this compatibility condition, the critic’s projection error does not bias the actor’s gradient estimates.⁶⁹

⁶⁶Trust region methods build on the performance difference lemma: for any two policies π and π' , $J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi'}} [\sum_a \pi'(a | s) A^{\pi}(s, a)]$, where $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ is the advantage function. This identity bounds how much policy improvement is possible and motivates constraining updates to regions where advantage estimates remain accurate (Kakade, 2002).

⁶⁷The original analysis of Konda and Tsitsiklis (2000) uses the average-cost formulation with TD error $\delta_t = c(X_t, U_t) - \Lambda + V(X_{t+1}) - V(X_t)$, where Λ is the average cost. The discounted variant presented here follows by replacing the average-cost baseline with $\gamma V(s')$. A related but distinct ODE stability framework for single-timescale stochastic approximation, including Q-learning and TD learning, appears in Borkar and Meyn (2000).

⁶⁸The two-timescale structure is analogous to nested optimization in structural estimation, where an inner loop solves for equilibrium given parameters and an outer loop searches over parameters. The critic’s inner loop (policy evaluation) must converge before the actor’s outer loop (policy improvement) takes a step. Wu et al. (2020) provide finite-time convergence rates ($\tilde{O}(\epsilon^{-2.5})$ sample complexity) for two-timescale actor-critic with linear approximation, and Tian et al. (2023) establish analogous rates for single-timescale actor-critic with multi-layer neural networks.

⁶⁹The compatible function approximation theorem first appears in Sutton et al. (2000) and is the key structural requirement in Konda and Tsitsiklis (2000). It constrains the critic architecture to be “compatible” with the actor parameterization, the same condition that makes the natural policy gradient equal to the critic’s weight vector in Kakade (2002).

A2C (Advantage Actor-Critic) is the synchronous variant: collect a batch of transitions, compute TD errors, and update both networks. A3C (Mnih et al., 2016) parallelizes this across multiple workers updating a shared parameter server asynchronously.⁷⁰

4.5.2 Entropy Regularization and Soft Actor-Critic

SAC (Soft Actor-Critic) (Haarnoja et al., 2018) extends the actor-critic framework with entropy regularization, building on the soft Q-learning algorithm of Haarnoja et al. (2017). The agent maximizes the entropy-augmented objective:

$$J_\tau(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t (R_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t))) \right], \quad (49)$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a) \log \pi(a)$ is the entropy and $\tau > 0$ is the temperature parameter. Geist et al. (2019) later provided the unifying theoretical framework, showing that entropy regularization converts the Bellman optimality operator’s non-smooth hard max into a smooth log-sum-exp, and that the resulting soft Bellman operator remains a γ -contraction, preserving the convergence guarantees of standard dynamic programming.⁷¹

Entropy regularization also addresses the deadly triad directly. By maintaining policy stochasticity, the behavior policy used for data collection remains close to the target policy being optimized. This reduces the distribution mismatch between the stationary distribution under the behavior policy and the update targets, mitigating the off-policy instability leg of the triad. Cen et al. (2022) formalize a second benefit: entropy regularization accelerates convergence of the natural policy gradient from $O(1/\epsilon)$ to $O(\log(1/\epsilon))$, providing a precise sense in which smoothing the policy landscape aids optimization. The actor-critic structure separates identification from optimization: the critic solves a regression problem (estimate V^π from data), while the actor solves an optimization problem (improve π using the estimated values).

4.5.3 Error Amplification Under Approximate Value Functions

Two questions remain important: how does approximation error propagate to policy quality, and how does computational complexity scale with problem size? Singh and Yee (1994) bound the policy degradation from value function errors (an independent derivation appears in Bertsekas and Tsitsiklis 1996, Proposition 6.1). If $\hat{V}$ approximates V^* with error $\|\hat{V} - V^*\|_\infty \leq \epsilon$, and $\hat{\pi}$ is the greedy policy with respect to $\hat{V}$, then:

$$\|V^* - V^{\hat{\pi}}\|_\infty \leq \frac{2\gamma}{1-\gamma} \epsilon. \quad (50)$$

At $\gamma = 0.99$, the amplification factor is $2 \cdot 0.99/0.01 = 198$. A 1% error in value function approximation yields at most 198% error in policy value.⁷² This bound is pessimistic but finite:

⁷⁰Parallel workers decorrelate gradient estimates by exploring different parts of the state space simultaneously, removing the need for an experience replay buffer. Rigorous convergence theory for A3C’s lock-free asynchronous parameter updates remains an open problem; existing analyses of asynchronous stochastic approximation (Qu and Wierman, 2020) address classical asynchrony (different state-action pairs updated at different times on a single trajectory), not the parallel-worker gradient setting.

⁷¹The optimal policy under entropy regularization is $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, which is precisely the McFadden (1974a) logit choice probability with systematic utility $Q^*(s,a)$ and scale parameter τ . The entropy-regularized value function is the log-sum-exp of Q-values, corresponding to the inclusive value (log-sum) operator in nested logit models. This equivalence between the soft-control framework and dynamic discrete choice models with EV1 taste shocks is developed formally in Rust and Rawat (2026), Appendix A.

⁷²The Singh-Yee bound is worst-case and not tight in general; massoud Farahmand et al. (2010) derive tighter bounds under smoothness assumptions on the MDP. The amplification factor $2\gamma/(1-\gamma)$ is a sensitivity analysis: it quantifies how errors in the “inputs” (value estimates) propagate to “outputs” (policy quality), analogous to errors-in-variables bias in regression. The discount factor γ controls sensitivity; more patient agents face larger

approximate value functions do not cause unbounded policy degradation.

4.5.4 Sample Complexity of Planning

Classical dynamic programming complexity scales with the state space size $|\mathcal{S}|$. For problems like Go, where $|\mathcal{S}| \approx 10^{170}$, exact computation is impossible. Kearns et al. (2002) prove that with access to a generative model⁷³ (a simulator that samples transitions from any state-action pair), near-optimal planning is possible with *no dependence on* $|\mathcal{S}|$. The cost is exponential dependence on the effective horizon $H = \log(R_{\max}/(\epsilon(1-\gamma)))/\log(1/\gamma)$: the sparse sampling algorithm requires $O((|\mathcal{A}|/\epsilon)^H)$ simulator calls.⁷⁴ For γ near 1, $H \approx (1-\gamma)^{-1} \log(R_{\max}/\epsilon)$, so the method is practical only for short effective horizons or moderate discount factors. The key insight is the tradeoff: classical DP scales linearly in $|\mathcal{S}|$ but polynomially in H ; sparse sampling eliminates state-space dependence at the cost of exponential horizon dependence. This explains why MCTS succeeds in large state spaces with bounded lookahead.

The minimax-optimal sample complexity for planning with a generative model, when queries to arbitrary state-action pairs are permitted, is $\Theta(|\mathcal{S}||\mathcal{A}|/((1-\gamma)^3\epsilon^2))$ (Azar et al., 2013). This bound scales linearly in $|\mathcal{S}|$ but polynomially in $1/(1-\gamma)$, the opposite regime from sparse sampling. Agarwal et al. (2020a) show that the plug-in model-based approach (learn $\hat{P}$ from samples, then plan with $\hat{P}$) achieves this minimax rate, establishing that model-based RL is statistically optimal. Li et al. (2024b) further tighten this result by breaking the $|\mathcal{S}||\mathcal{A}|/(1-\gamma)^2$ sample-size barrier, showing that the minimax rate is achievable with total sample size as low as $|\mathcal{S}||\mathcal{A}|/(1-\gamma)$.⁷⁵

4.6 Fundamental Tradeoffs

The choice between methods involves distinct tradeoffs rooted in DP structure. Value-based methods target Q^* via the Bellman contraction (Szepesvári, 2010). In the tabular case convergence is guaranteed, but function approximation introduces the deadly triad. Policy-based methods optimize π_θ directly. Modern theory (Agarwal et al., 2021a) establishes global convergence for softmax policies, with high variance as the practical weakness rather than local traps. Actor-critic methods combine both (Konda and Tsitsiklis, 2000), using the critic for low-variance value estimates while the actor inherits policy gradient’s global convergence. Each family traces to DP foundations.

Four additional trade-offs pervade reinforcement learning. First, *exploration versus exploitation*: should the agent act on its current best estimate or gather information to improve future decisions? Lai and Robbins (1985) establish the fundamental lower bound: any consistent policy must incur regret at least logarithmic in the number of periods. Naive exploration (ϵ -greedy) requires samples exponential in the horizon; strategic exploration (UCB, optimism in the face

amplification.

⁷³A “generative model” in RL is a simulator that, given any state-action pair (s, a) , returns a sampled next state $s' \sim P(\cdot|s, a)$ and reward r . This is unrelated to “generative models” in machine learning (GANs, diffusion models) or “generative processes” in Bayesian econometrics. The distinction matters: planning with a generative model is strictly easier than learning from a single trajectory, because the agent can query arbitrary states rather than following a sequential path.

⁷⁴The sparse sampling algorithm builds a random tree of depth H from the current state, sampling C successor states per action at each node, then estimates values by averaging leaf rewards back up the tree. Its running time is $O((C|\mathcal{A}|)^H)$, which is exponential in H but entirely independent of $|\mathcal{S}|$. Kearns et al. (2002) also prove a lower bound of $\Omega(2^H)$ generative model calls for any planning algorithm, so exponential horizon dependence is unavoidable in the worst case.

⁷⁵Recent extensions push these results beyond standard MDPs. Clavier et al. (2024) study the robust MDP setting where the agent must plan under model uncertainty with sa-rectangular or s-rectangular uncertainty sets, establishing minimax rates for robust policy optimization. Wang et al. (2025) extend the analysis to risk-sensitive objectives under the iterated CVaR criterion.

of uncertainty) reduces this to polynomial (Auer et al., 2002), formalizing the value of targeted experimentation.⁷⁶

Second, *model-based versus model-free*: model-based methods learn a transition model $\hat{P}(s'|s, a)$ and plan with it (Sutton, 1990); model-free methods learn value functions or policies directly from transitions. The Dyna architecture (Sutton, 1990) bridges these by generating simulated experience from the learned model to supplement real transitions. Model-based methods are sample-efficient (each transition updates the entire model, which improves value estimates for all states) but suffer asymptotic bias if the model class is misspecified; model-free methods are asymptotically unbiased but sample-inefficient, using each transition for a single gradient step.⁷⁷

Third, *on-policy versus off-policy* (Sutton and Barto, 2018, Ch. 5–7): on-policy methods (SARSA, REINFORCE) learn from data generated by the current policy, ensuring stability but discarding past experience; off-policy methods (Q-learning, DQN) reuse stored experience via replay buffers, gaining sample efficiency but risking the instabilities of the deadly triad.

Fourth, *bias versus variance in advantage estimation*: REINFORCE uses the full Monte Carlo return (unbiased but high variance); actor-critic methods (Konda and Tsitsiklis, 2000) use bootstrapped TD targets (low variance but biased by the critic’s approximation error). Generalized Advantage Estimation in PPO (Schulman et al., 2015) interpolates between these extremes via a parameter $\lambda \in [0, 1]$, where $\lambda = 1$ recovers Monte Carlo returns and $\lambda = 0$ recovers one-step TD. The value function baseline is the variance-minimizing choice, motivating the actor-critic architecture as a bias-variance compromise.

4.7 Conclusion

The central insight is that RL algorithms are not mysterious. They are asymptotic approximations to classical dynamic programming operators, justified by the mathematics of contractions, stochastic approximation, and gradient domination. Value iteration becomes Q-learning (Watkins and Dayan, 1992; Tsitsiklis, 1994) when expectations are replaced by single samples. Policy iteration becomes the natural policy gradient (Kakade, 2001; Agarwal et al., 2021a) when the greedy improvement step is approximated by gradient ascent, and NPG recovers PI exactly in the tabular case. The stochastic approximation framework, from the foundational work of Robbins (1952) through the ODE method of Borkar and Meyn (2000), guarantees that under appropriate step-size conditions, noisy iterates converge to the same fixed points as their deterministic counterparts. Reinforcement learning is not a departure from dynamic programming but an extension of it. Tabular RL and RL with linear function approximation is well understood. However, deep RL is not as well understood. It seems to work for some cases but fails in others, and we do not have a firm handle on why. But in some sense, we do know that when it works it works due to the underlying Bellman principle. For these reasons convergence in RL remains an open issue and successes in RL remain brittle and case-specific.

5 The Empirics of Deep Reinforcement Learning

5.1 The Moving Target Problem

In supervised learning, the loss function is a *fixed function* of the training data and model parameters. Deep reinforcement learning does not enjoy this property. Each gradient step moves

⁷⁶The exploration-exploitation tradeoff is the subject of the bandits chapter, where the multi-armed bandit framework provides the sharpest analysis. In full MDPs, exploration is harder because the agent must learn not just reward distributions but also transition dynamics, compounding the information requirement.

⁷⁷Model misspecification in RL is the analog of omitted variable bias in econometrics: if the learned model omits relevant state variables or misspecifies the functional form of transitions, the resulting policy is biased regardless of sample size. Moerland et al. (2023) provide a comprehensive survey of model-based RL, analyzing the model-bias versus sample-efficiency tradeoff across method families.

both the current value estimates and the targets simultaneously, creating a “nonstationary” optimization landscape. The target network heuristic introduced by Mnih et al. (2015) slows target drift by periodically freezing a copy of the network, but does not eliminate it. Therefore Bellman residual is a poor proxy for the accuracy of the value function.

Fujimoto et al. (2022) formalize this observation. Let Q^π denote the true value function for policy π and let Q denote an approximation. Define the Bellman error $\varepsilon(s, a) = Q(s, a) - (r + \gamma \mathbb{E}_{s', a'}[Q(s', a')])$ and the value error $\Delta(s, a) = Q(s, a) - Q^\pi(s, a)$. The Bellman equation is uniquely satisfied by Q^π over all state-action pairs, so zero Bellman error implies zero value error when the entire MDP is considered. But Fujimoto et al. (2022) prove that this equivalence breaks in two ways in practice. First, the Bellman error can be small due to error cancellation: if both sides of the Bellman equation are biased in the same direction, their difference is small even when both are far from the truth. Second, with a finite dataset, the Bellman equation can be satisfied exactly by infinitely many suboptimal solutions, because the constraint is only enforced at observed state-action pairs. A network can reach near-zero training loss while the value function remains arbitrarily inaccurate.

The dual failure mode appears in policy gradient methods. Ilyas et al. (2020) examine the surrogate objective used in Proximal Policy Optimization (Schulman et al., 2017). The PPO clipping mechanism is designed to keep policy updates within a trust region by bounding the probability ratio $\pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$. Ilyas et al. (2020) find that even when the surrogate objective improves monotonically over training, the episode return can plateau or decline. The gradient of the surrogate is poorly aligned with the gradient of the true return, particularly in later training phases where the policy has diverged from the behavior policy used to collect the replay data. The loss metric that practitioners monitor throughout training is measuring something other than what they care about.

Section 5.8 demonstrates this disconnect empirically: a DQN agent trained on CartPole-v1 exhibits near-zero TD loss in some training phases while episode return stagnates, and elevated TD loss in other phases where performance is stable.

5.2 The Reproducibility Crisis

Henderson et al. (2018) trained five leading policy gradient algorithms (PPO, TRPO, DDPG, TD3, SAC) on six MuJoCo benchmark environments, holding all hyperparameters fixed and varying only the random seed. The resulting learning curves from different seeds were non-overlapping: a seed that performed well under one algorithm performed comparably to a different algorithm’s best seeds, making cross-algorithm comparison unreliable. Differences as large as 2,000 points in final episode return arose from seed variation alone.

The statistical consequences are severe. Agarwal et al. (2021b) analyze the Atari 100k benchmark and show that comparing point estimates (mean or median final score) from 5 training runs per task has Type I error rates exceeding 50% for realistic effect sizes. At such sample sizes, a random noise injection that does nothing beneficial will appear to “improve” performance in half of all comparisons. The standard practice in published deep RL results from 2015 to 2020 was to report 3–5 seeds per task, a regime where statistical conclusions are essentially random.

Agarwal et al. (2021b) propose the *interquartile mean* (IQM) as a replacement for mean and median when comparing algorithms. The IQM discards the top and bottom 25% of runs before averaging, reducing sensitivity to outlier seeds. They also introduce performance profiles, which plot the fraction of tasks and seeds where an algorithm achieves performance above a threshold τ , as τ varies from 0 to the maximum. Performance profiles reveal the full shape of the score distribution rather than collapsing it to a single statistic. Using these tools and stratified bootstrap confidence intervals, Agarwal et al. (2021b) find that several widely-cited algorithmic improvements on Atari 100k vanish or reverse when statistical uncertainty is accounted for.

The fragility extends to hyperparameters. Eimer et al. (2023) conduct a systematic study of hyperparameter sensitivity across 6 algorithms and 17 environments, finding that default hyperparameters from published papers perform competitively in the specific environments used in those papers but generalize poorly across environments. Patterson et al. (2024) synthesize these findings into an empirical design handbook, recommending at minimum 10 seeds per configuration, IQM-based comparisons, and preregistration of hyperparameter search protocols.

5.3 Systematic Value Overestimation

Q-learning uses the Bellman optimality update

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a'), \quad (51)$$

where the maximum is taken over the estimated Q-values of all actions at the successor state. Thrun and Schwartz (1993) identify a positive bias intrinsic to this update: if the Q-value estimates contain noise with mean zero, the maximum over noisy estimates is biased upward by Jensen’s inequality. An agent that uses a single network for both action selection ($\arg \max$) and value estimation ($\max Q$) systematically overestimates the values of every state it visits, biasing the Bellman target upward at every update step. The bias compounds through bootstrapping: overestimated targets produce overestimated updates, which produce further overestimated targets.

van Hasselt (2010) propose double Q-learning as a remedy: maintain two independent Q-networks Q_A and Q_B . Use Q_A to select the greedy action at s' , but use Q_B to evaluate that action. Because the two networks are trained on different data, their errors are approximately independent, and the positive bias largely cancels. van Hasselt et al. (2016b) implement this as Double DQN, using the online network for action selection and the periodically-frozen target network for evaluation. On 49 Atari games, Double DQN reduces overestimation by a factor of 3–5 and improves median performance by 20% relative to DQN.

Fujimoto et al. (2018) observe that Double DQN’s correction is incomplete in continuous-action settings, where the target network and online network remain correlated through shared updates. They propose Clipped Double Q-learning: compute two Q-value estimates Q_1, Q_2 with separate networks trained on the same data, and use $y = r + \gamma \min(Q_1(s', a'), Q_2(s', a'))$ as the Bellman target. The minimum operator introduces pessimistic underestimation, which is conservative but avoids the explosive positive bias. Ciosek et al. (2019) note that clipped double Q can cause excessive pessimism under high uncertainty, proposing optimistic actor-critic as a counterweight.

5.4 The Deadly Triad in Practice

Sutton and Barto (2018, Ch. 11) identify the *deadly triad*: divergence is possible whenever an algorithm combines function approximation, bootstrapping, and off-policy learning. ? provide the canonical counterexample—Baird’s star—in which linear function approximation with Q-learning diverges to infinity on a simple 7-state MDP with a deterministic policy, despite stable-looking updates at each step. The mechanism is that off-policy data breaks the contraction property of the projected Bellman operator, and bootstrapping propagates errors forward through the value function approximation rather than against the direction of data flow.

DQN’s stabilizations—experience replay, a target network, reward clipping, and gradient clipping—successfully prevent outright divergence in practice. But van Hasselt et al. (2018) find that *soft divergence*, defined as a temporary spike in value estimates by more than 10% followed by recovery, occurs in the majority of DQN training runs across 57 Atari games. Larger networks diverge more frequently than smaller ones, counter to the usual intuition that more expressive models should generalize better. The deadly triad does not announce itself as a

training failure: the value estimates may spike and recover, leaving no visible trace in the loss curve while corrupting the policy.

Kumar et al. (2021) describe a continuous manifestation of the deadly triad: *implicit underparameterization*. Neural networks trained with bootstrapped TD objectives progressively lose their effective rank, with fewer and fewer neurons contributing distinct directions in the representation. This rank collapse is silent—training loss continues to decrease—but the network’s ability to represent new information degrades over time, a form of capacity loss distinct from but related to plasticity loss.

5.5 Plasticity Loss and Primacy Bias

A network that trains well at step $t = 100$ may be incapable of learning at step $t = 100,000$, even if the data quality at the later step is higher. Lyle et al. (2022) call this *capacity loss*: the network’s ability to update its own weights degrades progressively during training, measured by the fraction of effective parameters and the network’s ability to fit new random labels. Lyle et al. (2023) extend this to *plasticity loss*, identifying dead ReLU neurons, weight norm growth, and feature rank collapse as three distinct mechanisms, not all of which co-occur.

Nikishin et al. (2022) identify *primacy bias* as a specific cause. Because the replay buffer is filled incrementally, early transitions are oversampled relative to later ones: they appear in every minibatch from the moment they are collected, while transitions collected at step t only begin to influence training at step t . The result is that the network over-fits to early environment transitions, corrupting representations that govern behavior throughout the remainder of training. Nikishin et al. (2022) show that 100 initial “priming” steps of excessive gradient updates—without any environmental reset—degrade a SAC agent’s performance for hundreds of thousands of subsequent steps.

Sokar et al. (2023) measure the fraction of dormant neurons—units with near-zero activation across the entire replay buffer—as training proceeds. Dormant neurons accumulate monotonically during training with high replay ratios, and their fraction correlates with performance degradation across 10 Atari games. Dohare et al. (2024) study the phenomenon in long-horizon continual learning tasks and find that standard deep networks lose all plasticity within a few million gradient steps, a result published in *Nature* and confirmed across architectures.

The proposed remedies fall into three categories. Periodic resets reinitialize the last layers of the network while retaining the replay buffer, allowing the agent to forget overfit representations without discarding experience (Nikishin et al., 2022; D’Oro et al., 2023). Continual backpropagation replaces neurons with near-zero utility at each gradient step rather than waiting for a global reset (Dohare et al., 2024). Architectural interventions—layer normalization (Lyle et al., 2025), orthogonal initialization, spectral normalization—reduce the rate at which plasticity is lost by stabilizing gradient magnitudes and preventing weight norm growth.

5.6 Implementation Dominates Algorithmic Innovation

Engstrom et al. (2020) conduct a controlled ablation of PPO’s components on MuJoCo continuous control. PPO is typically credited with outperforming TRPO (Schulman et al., 2015) because of its clipped surrogate objective, which constrains policy updates without requiring second-order information. Engstrom et al. (2020) find that PPO with the clipping mechanism disabled—but with observation normalization, reward normalization, value function clipping, global gradient norm clipping, orthogonal weight initialization, and Adam optimizer enabled—performs indistinguishably from the full algorithm. TRPO (Schulman et al., 2015) with the same code-level additions matches PPO’s performance. The performance difference between algorithms is explained by implementation details, not the core algorithmic innovations.

Andrychowicz et al. (2021) extend this analysis using a 250,000-agent study across 5 environments and 250 hyperparameter configurations. They find that three implementation choices

account for most of the variance in performance: observation normalization (normalizing inputs to zero mean, unit variance), orthogonal weight initialization in the policy network, and learning rate annealing. The discount factor γ had near-zero marginal effect in their study; the clipping parameter ϵ had a small positive effect on two of five environments.

Huang et al. (2022) catalog 37 implementation details required to reproduce PPO’s published results on Atari, including reward clipping to $[-1, 1]$, frame stacking to 4 frames, a specific episode termination convention, and a numerically stable normalization of the advantage estimate. Omitting any of these details produces results that differ materially from the published benchmark. The same analysis applies to most published deep RL results: the algorithm description in the paper is necessary but not sufficient to reproduce the result.

The most consequential implementation detail is the distinction between termination and truncation (Pardo et al., 2018). In reinforcement learning, an episode can end for two distinct reasons: *termination*, where the environment reaches a natural absorbing state (the pole falls in CartPole, the robot falls in locomotion tasks), and *truncation*, where the episode is cut short by an external time limit. At a termination, the value of the successor state is zero: $V(s_{\text{term}}) = 0$. At a truncation, the episode is merely paused and the successor state has non-zero value: $V(s_{\text{trunc}}) \neq 0$. Treating truncated transitions as terminated substitutes zero for a non-zero bootstrap value at every time limit boundary, corrupting every Bellman update in the vicinity of episode boundaries. Pardo et al. (2018) show that this conflation degrades performance by 20–40% on standard MuJoCo benchmarks. The Gymnasium API (Towers et al., 2024) enforces the distinction by returning separate `terminated` and `truncated` flags, but most pre-2022 codebases conflate them in the `done` flag.

5.7 Replay Buffer Pathologies and Reward Scaling

Experience replay (Lin, 1992) decouples the data collection and learning processes, allowing a single transition to be used for multiple gradient updates. The replay ratio—the number of gradient updates per environment step—governs the trade-off between sample efficiency and data staleness. Zhang and Sutton (2017) show that increasing the replay ratio beyond a modest threshold degrades performance: at high replay ratios, the distribution shift between the current policy and the behavior policy that generated the stored data grows large enough to violate the off-policy assumptions of Q-learning. The relationship is non-monotone and environment-dependent, making replay buffer size a sensitive hyperparameter with no universal default.

Schaul et al. (2016) propose prioritized experience replay (PER), which samples transitions with probability proportional to the magnitude of their TD error, on the argument that high-error transitions are the most informative. Fedus et al. (2020) revisit PER on large-scale Atari experiments and find that uniform sampling from a large enough buffer matches or outperforms PER, while being simpler to implement and tune.

Reward scaling introduces a separate class of failure modes. Standard DQN (Mnih et al., 2015) clips rewards to $[-1, +1]$ across all environments to stabilize training. van Hasselt et al. (2016a) observe that reward clipping changes the objective: clipped rewards make all positive events equivalent regardless of magnitude, so the agent learns to maximize the frequency of positive events rather than their cumulative value. This substitution can produce policies that are locally rational under the clipped reward but qualitatively suboptimal under the true reward. van Hasselt et al. (2016a) propose PopArt (Preserving Outputs Precisely, while Adaptively Rescaling Targets), which normalizes targets to have unit variance while adjusting the output layer so that the policy remains invariant to the normalization. PopArt allows consistent learning across reward scales spanning several orders of magnitude without reward clipping.

Skalse et al. (2022) provide a formal analysis of reward hacking: the phenomenon where optimizing an imperfect proxy reward $\tilde{R}$ leads to poor performance under the true reward R . They define a proxy as *unhackable* if increasing expected proxy return cannot decrease expected true return. Their main result states that for the set of all stochastic policies, two

reward functions are unhackable only if one of them is constant. Any non-constant proxy reward can in principle be hacked. For deterministic policies and finite policy sets, non-trivial unhackable pairs exist, but the conditions are stringent. The practical implication is that any reward function designed by a human is vulnerable to exploitation by a sufficiently capable optimizer.

5.8 Simulation Study: The TD Loss and Episode Return Disconnect

The theoretical arguments in Section 5.1 predict that the TD loss should be a poor proxy for policy quality in deep RL. We demonstrate this empirically by training DQN on the CartPole-v1 control task and comparing the trajectory of TD loss against episode return.

Design. A DQN agent with two hidden layers of 128 units each is trained for 60,000 environment steps on CartPole-v1 (Gymnasium). Training uses a replay buffer of 10,000 transitions, minibatch size 64, $\gamma = 0.99$, ϵ -greedy exploration decaying from 1.0 to 0.01, and a hard target network update every 100 steps. We log the mean-squared Bellman error (TD loss) per minibatch and the rolling 20-episode average return at intervals of 500 steps, across 3 random seeds. As a supervised analogue, we train logistic regression on a linearly-separable 2-dimensional dataset using cross-entropy loss, logging both the loss and classification accuracy.

Results. The supervised analogue shows the expected tight coupling: cross-entropy loss decreases monotonically as accuracy increases from 50% to 92.6%, with Pearson correlation near -0.99 between loss and error rate. The DQN results show the opposite pattern. The Pearson correlation between TD loss and episode return is 0.17 across the full training run (Table 5). In the first half of training, TD loss rises while episode return also rises (early correlation 0.06): the network is learning to solve the task while simultaneously generating larger Bellman residuals. In the second half, TD loss fluctuates without tracking return (late correlation -0.11): seeds that achieve higher episode returns show marginally higher TD loss, inverting the supervising-learning intuition.

Method	Final TD loss	Final return (rolling avg.)	Pearson r (loss, return)	Correlation interpretation
Logistic regression (CE)	0.166	92.6% acc.	-0.99	Tight coupling
DQN (seed 42)	1.398	375.5		
DQN (seed 123)	0.700	436.2		
DQN (seed 777)	0.956	414.3		
DQN (mean, 3 seeds)	1.018	408.7	$+0.17$	Near-zero coupling

Table 5: TD loss and episode return for DQN on CartPole-v1, with supervised analogue (logistic regression on a 2D separable dataset). In supervised learning, loss and accuracy are tightly coupled (correlation -0.99). In DQN training, the correlation between TD loss and episode return is $+0.17$ over the full run and -0.11 in the second half of training, confirming the theoretical prediction of Fujimoto et al. (2022).

5.9 A Practitioner’s Checklist

The findings reviewed in this section distill into a set of implementation requirements that should be treated as defaults rather than optional refinements.

1. Normalize observations to zero mean, unit variance before passing them to any neural network. This is the single implementation detail with the largest aggregate effect on performance (Andrychowicz et al., 2021).

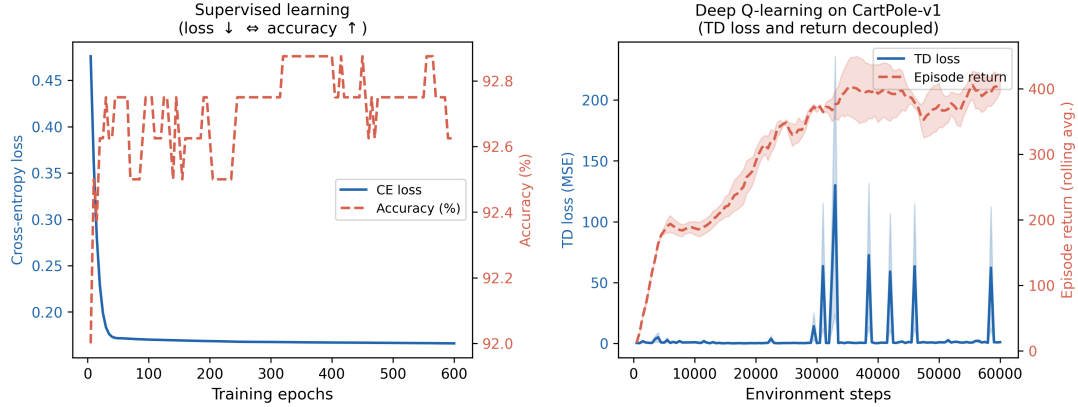


Figure 4: Left: logistic regression on a 2D toy dataset. Cross-entropy loss and accuracy move in lockstep, as expected in supervised learning. Right: DQN on CartPole-v1. TD loss and episode return (rolling 20-episode average, shaded ± 1 SE over 3 seeds) are largely decoupled throughout training. TD loss rises, falls, and oscillates independently of the return trajectory, illustrating the theoretical result of Fujimoto et al. (2022) that the Bellman residual is a poor proxy for value function accuracy.

2. Scale rewards so that episodic returns are $O(1)$, not $O(10^6)$. Clip or normalize rather than leaving raw reward magnitudes from the environment, which may span orders of magnitude across environments or over training (van Hasselt et al., 2016a).
3. Distinguish termination from truncation in every environment wrapper. Set the bootstrap target to $r + \gamma V(s')$ for truncated transitions and to r for terminated transitions. Never use a single `done` flag that conflates both (Pardo et al., 2018).
4. Track episode return and policy entropy alongside the training loss. The training loss is uninformative about policy quality; entropy collapse and stagnating return are early warning signs of plasticity loss and mode collapse.
5. Run a minimum of 10 seeds per configuration. Use the interquartile mean and stratified bootstrap confidence intervals when comparing algorithms, not point estimates from 3–5 seeds (Agarwal et al., 2021b).
6. Use PPO or SAC as the default baseline before implementing any custom algorithm. Both have well-understood hyperparameters, stable open-source implementations, and broad benchmark coverage.
7. Monitor the fraction of dormant neurons and the effective rank of the value network’s penultimate layer as training progresses (Sokar et al., 2023). Dormant fraction above 20% and sustained rank collapse are diagnostic of plasticity loss; consider periodic resets or layer normalization as interventions.

6 Reinforcement Learning Algorithms: An Illustrated Example

The preceding chapters derived convergence rates, identified failure modes, and catalogued the tradeoffs governing reinforcement learning algorithms. This section runs nine algorithms on a common environment to observe these properties empirically, with particular attention to per-state value function convergence.

6.1 Setup

The environment is a 5×5 deterministic gridworld with five actions (up, down, left, right, stay). Each non-terminal step incurs a penalty of -0.1 ; reaching the goal in the bottom-right corner

yields +10. A small state-dependent perturbation ($\epsilon = 0.001$) breaks the symmetry between equally-long paths, ensuring a unique optimal policy.⁷⁸ The discount factor is $\gamma = 0.95$ and each episode is capped at 50 steps. The optimal path requires 8 steps, yielding an optimal discounted return of 9.30.⁷⁹

With only 25 states, the value function $V^*(s)$ can be displayed in full. The grid ranges from $V^*(s_0) = 6.38$ at the top-left corner to $V^* = 10.0$ at the state adjacent to the goal, with values increasing monotonically along the optimal path. This small scale makes it feasible to track convergence at every state individually, rather than relying on aggregate metrics that can mask important differences between algorithms.

Two planning methods serve as baselines: value iteration (VI) and policy iteration (PI), both requiring the full transition model $P(s'|s, a)$. Seven model-free learning algorithms observe only sampled transitions: Q-learning, SARSA, $Q(\lambda)$, DQN, REINFORCE with value baseline, Natural Policy Gradient (NPG), and Proximal Policy Optimization (PPO). DQN replaces Q-learning’s lookup table with a two-layer neural network (64 hidden units) as function approximator, training on minibatches sampled from a replay buffer. Algorithm definitions appear in Sections 3.1–4.5; this section focuses on empirical behavior.

Table 6 reports results for all nine algorithms.

Table 6: Performance of nine algorithms on the 5×5 gridworld. “Ep to V^* conv” is the first checkpoint at which $\max_s |V(s) - V^*(s)| < 0.1$ across all 25 states; “—” indicates the algorithm never reaches this threshold. All seven learning algorithms achieve the optimal return of 9.30, but only Q-learning, $Q(\lambda)$, and DQN converge to V^* everywhere.

Algorithm	Return	Agreement (%)	V^* RMSE	Max $ V - V^* $	Ep to $V(s_0)$	Ep to V^* conv	Time (s)
<i>Planning (full model)</i>							
VI	9.30	100.0	0.00	0.00	—	9 iter	0.001
PI	9.30	100.0	0.00	0.00	—	6 iter	0.019
<i>Temporal difference</i>							
Q-Learning	9.30	100.0	0.00	0.00	340	610	22.8
SARSA	9.30	88.0	1.67	5.53	7600	—	22.4
<i>Multi-step</i>							
$Q(\lambda)$	9.30	100.0	0.00	0.00	340	550	29.6
<i>Deep RL</i>							
DQN	9.30	92.0	0.01	0.01	1600	1600	2920.6
<i>Policy gradient</i>							
REINFORCE	9.30	76.0	2.20	5.71	132000	—	139.6
NPG	9.30	44.0	4.52	9.72	200	—	117.0
PPO	9.30	52.0	1.88	5.42	30000	—	253.3

6.2 Planning Versus Learning

VI converges in 9 iterations (0.001 seconds). On this deterministic gridworld, the iteration count matches the MDP diameter: the longest shortest path from any state to the goal is 8 steps, and each Bellman backup propagates value one step outward from the *terminal state*.⁸⁰

⁷⁸Without the perturbation, any shortest path from start to goal is optimal. The perturbation subtracts $\epsilon \cdot i / N^2$ from each step reward, where i is the destination state index. Since moving down increases the state index by N while moving right increases it by 1, the agent strictly prefers right over down at every interior state.

⁷⁹Shared hyperparameters across all learning algorithms: learning rate $\alpha = 0.1$ (decay 0.99995/episode) for value-based methods, $\alpha = 0.01$ for policy gradient methods; ϵ -greedy exploration from 1.0 to 0.01 (decay 0.9995/episode); trace decay $\lambda = 0.9$; NPG actor step size $\eta = 0.2$, critic step size 0.5 (both decay 0.99995/episode). DQN uses a two-layer network with 64 hidden units, replay buffer of 50,000 transitions, batch size 64, learning rate 10^{-3} , target network update every 100 episodes, and ϵ -decay 0.9999. PPO uses clip ratio 0.2, 4 optimization epochs per batch, GAE $\lambda = 0.95$, learning rate 0.01 (decay 0.99995/episode). All learning algorithms run for 500,000 episodes with seed 42.

⁸⁰The γ^k contraction bound from Section 4.1 predicts roughly $\log(\epsilon^{-1}) / \log(\gamma^{-1}) \approx 360$ iterations for $\epsilon = 10^{-8}$, making it loose by a factor of 40 on this structured problem.

PI converges in 9 policy-improvement steps (0.018 seconds). At each step, PI solves the linear system $V^\pi = r^\pi + \gamma P^\pi V^\pi$ and propagates value information across the full state space simultaneously, rather than one step at a time as in VI.

The fastest learning method, $Q(\lambda)$, needs 20 episodes to reach 99% of the optimal return; NPG and PPO need 50; SARSA needs 80. The gap between 0.001 seconds of planning and 20–80 episodes of learning is the price of not knowing the transition model $P(s'|s, a)$.

6.3 Value Function Convergence

All seven learning algorithms achieve the optimal return of 9.30, yet only three converge to $V^*(s)$ for all 25 states (Table 6). This distinction, invisible in the return metric, is the central finding of this comparison.

Q-learning and $Q(\lambda)$ are off-policy and learn Q^* regardless of the behavior policy. Both achieve $\max_s |V(s) - V^*(s)| < 0.1$ by episode 610 and 550 respectively, with final RMSE below 0.01. $Q(\lambda)$ converges slightly faster due to eligibility traces propagating the terminal reward backward through the full 8-step trajectory within a single episode.

DQN is also off-policy and converges to V^* at all 25 states by episode 1600, with RMSE 0.006 and $\max_s |V(s) - V^*(s)| = 0.01$. These errors are comparable to the tabular methods. The cost is wall-clock time: DQN requires 2921 seconds versus 25 seconds for tabular Q-learning, a $118\times$ slowdown from neural network training overhead on a problem small enough for exact tabular representation.

SARSA, REINFORCE, NPG, and PPO are on-policy and learn the value of the exploration policy π_ϵ , not π^* . SARSA leaves 9 of 25 states unconverged at episode 500,000, with a maximum error of 5.53 concentrated at the state farthest from the goal. REINFORCE leaves 15 of 25 states unconverged (max error 5.71), NPG leaves 12 of 25 unconverged (max error 9.72), and PPO leaves 16 of 25 unconverged (max error 5.42).

Table 7 tracks $\max_s |V(s) - V^*(s)|$ at five episode checkpoints. For Q-learning and $Q(\lambda)$, the maximum error drops below 0.1 by episode 1,000 and continues falling. DQN follows the same trajectory with a slight delay. The on-policy methods show a different pattern: SARSA’s maximum error remains above 5.0 throughout training, and the policy gradient methods plateau at similar levels, confirming that the error is structural rather than a matter of insufficient training.

Table 7: Maximum per-state value error $\max_s |V(s) - V^*(s)|$ over episodes (rolling average over ± 3 snapshots). Off-policy methods converge uniformly; on-policy methods plateau at persistent errors driven by rarely-visited states.

Algorithm	Ep 100	Ep 1K	Ep 10K	Ep 100K	Ep 500K
Q-Learning	7.64	0.00	0.00	0.00	0.00
SARSA	8.94	5.90	5.53	5.53	5.53
$Q(\lambda)$	4.64	0.00	0.00	0.00	0.00
DQN	8.40	1.17	0.03	0.02	0.01
REINFORCE	8.24	6.94	6.85	6.47	5.71
NPG	9.72	9.72	9.72	9.72	9.72
PPO	8.33	7.12	6.60	6.22	5.42

Table 8 tracks the fraction of states at which each algorithm’s greedy policy matches π^* . Q-learning and $Q(\lambda)$ reach 100% policy agreement by episode 10,000. DQN achieves 92% at its best checkpoints but oscillates substantially between snapshots, a consequence of neural network weight updates shifting the argmax at states with similar Q-values. SARSA stabilizes at 88% despite never converging in value. REINFORCE, NPG, and PPO plateau at 44–76%.

Table 8: Policy agreement (% of states where $\pi(s) = \pi^*(s)$) over episodes (rolling average over ± 3 snapshots). Tabular off-policy methods reach full agreement. DQN’s average is depressed by inter-snapshot oscillation. Policy gradient methods plateau below 80%.

Algorithm	Ep 100	Ep 1K	Ep 10K	Ep 100K	Ep 500K
Q-Learning	59	91	100	100	100
SARSA	74	82	88	88	88
Q(λ)	48	94	100	100	100
DQN	52	70	61	61	56
REINFORCE	72	76	76	76	76
NPG	48	48	48	44	44
PPO	49	52	52	52	52

6.4 Optimal Returns Without Optimal Value Functions

NPG reaches 99% of optimal return at episode 50 but never converges to V^* at 12 of 25 states. The per-state error pattern reveals why: NPG’s value function is accurate along the optimal path (states near the diagonal from start to goal) but wildly inaccurate at off-path states. At state (1,4), NPG’s error is 9.72, nearly the full magnitude of V^* . The agent has learned to act optimally on the equilibrium path without learning what off-equilibrium states are worth.

This is the distinction between on-path and off-path behavior. An agent can achieve optimal performance in its observed environment, acting correctly on the equilibrium path, without having solved the complete decision problem for every contingency. In structural estimation terms, the agent’s observed behavior reveals optimal choices, but its counterfactual predictions (what would it do at states it never visits?) are unreliable. Off-policy methods sidestep this problem by learning Q^* from any data, including exploratory transitions; on-policy methods learn the value of the policy they actually follow, which is the exploration policy during training.

Figure 5 makes this visible. At 5×5 resolution, each state’s error is individually readable. Q-learning shows a wavefront of decreasing error spreading from the goal; by the final episode, the entire grid is white (error near zero). NPG, REINFORCE, and PPO show persistent dark patches at states the optimal policy rarely visits, with errors exceeding 5.0 at states in the far corners.

Figure 6 shows the corresponding policy grids. Q-learning’s arrows match π^* everywhere by the final checkpoint. NPG and PPO show incorrect arrows concentrated at the same off-path states where value errors are largest, since the agent has never needed to act at these states, so its policy there reflects the softmax initialization rather than learned behavior.

Figure 1 displays the evolution of the value function $V(s)$ for different reinforcement learning algorithms across various training steps. The figure is organized into a grid where rows represent algorithms and columns represent training steps (10%, 25%, 50%, 75%, and Final). Each heatmap shows the value function for states s_1 to s_5 . The color bar on the right indicates the value function $V(s)$ ranging from 0 to 10.

43

Policy evolution (percentage of V^* -convergence time, seed 42)

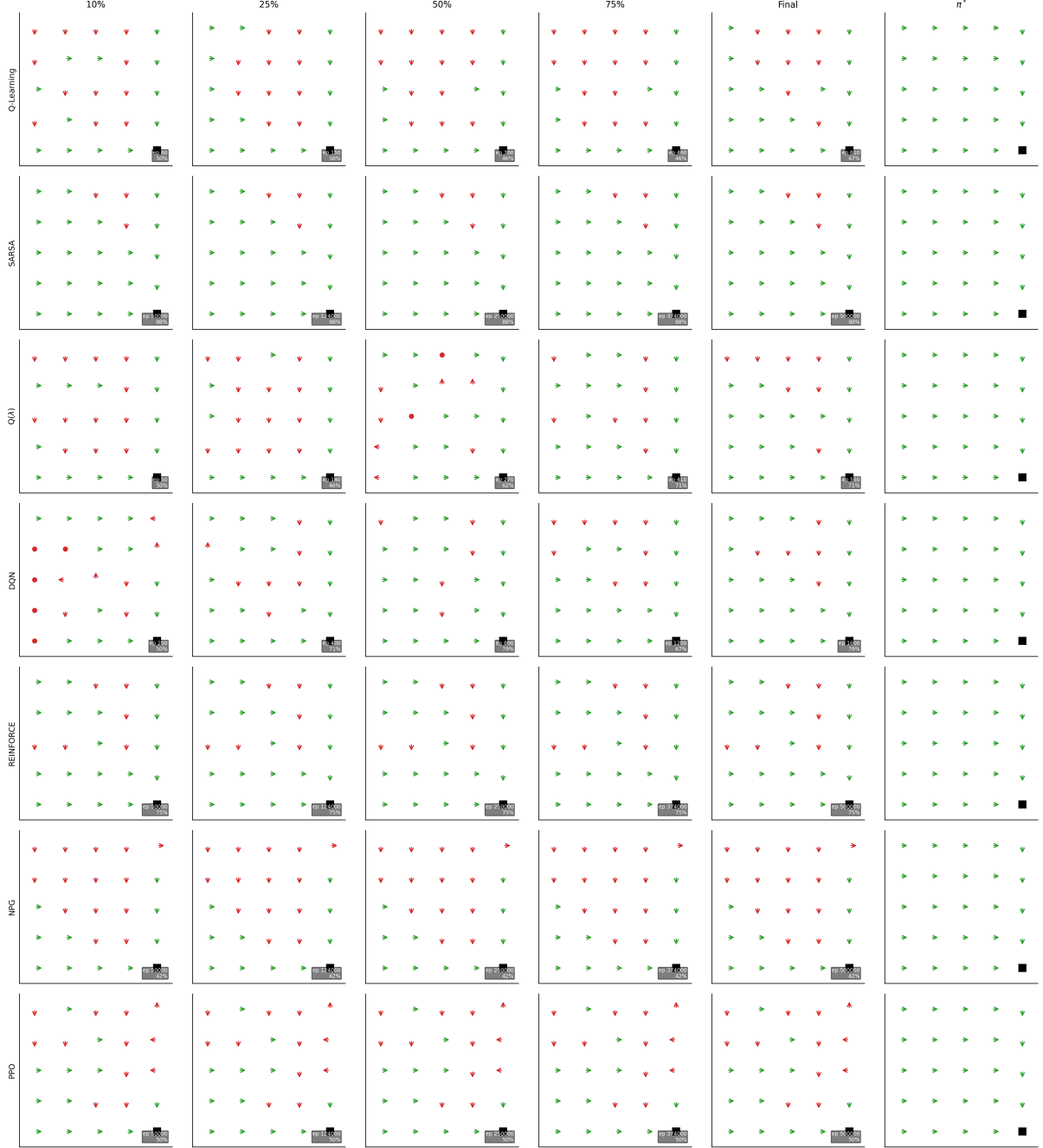


Figure 6: Policy grids at percentage-based checkpoints of V^* -convergence time T (same definition as Figure 5). Arrows indicate the greedy action at each state; dots indicate the stay action. Off-policy methods converge to π^* everywhere. On-policy methods retain incorrect actions at states far from the optimal path.

7 Reinforcement Learning for Optimal Control

Several industries now run reinforcement learning in production, not just in simulation. I focus on systems where RL operates continuously in the field, affecting millions of real decisions, such as ride-hailing dispatch at DiDi, data center cooling at Google, hotel revenue management, and financial order execution. The chapter concludes with a simulation study on the bus engine replacement problem, demonstrating that RL matches and surpasses dynamic programming on a classical benchmark.

7.1 Ride-Hailing Dispatch

Each driver-passenger assignment changes the spatial distribution of available drivers, making ride-hailing dispatch a sequential optimization problem at a scale (tens of millions of daily rides at DiDi) where exact dynamic programming is intractable.

Qin et al. (2021) formalized DiDi’s order dispatching as a semi-Markov decision process⁸¹ where each driver is an independent agent. The state of a driver consists of location (discretized into hexagonal zones) and time (bucketed into intervals). The action is the order assigned to the driver, with the option to remain idle. The reward is the trip fare. State transitions are determined by trip destinations, as completing an order transports the driver from origin to destination, changing their spatial state. The stochasticity arises from future demand, which determines the available actions at each state. The per-driver value function $V^\pi(s)$ represents the expected cumulative fare a driver can earn from state s under dispatching policy π :

$$V^\pi(s) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^{\tau_k} r_k \mid s_0 = s, \pi \right], \quad (52)$$

where τ_k is the time to complete the k -th trip and r_k is the corresponding fare. The platform’s objective is to maximize total driver income across the fleet, which decomposes into the sum of individual driver value functions.

The deployed system uses generalized policy iteration within a combinatorial optimization framework. Each dispatching window (a few seconds), the platform collects open orders and available drivers, constructs a bipartite graph, and solves a linear assignment problem. The edge weights are computed as the advantage of each driver-order match relative to the driver’s current state value.

$$w_{ij} = \hat{r}_i + \gamma^{\hat{\tau}_i} V(s'_j) - V(s_j), \quad (53)$$

where $\hat{r}_i$ is the predicted fare for order i , $\hat{\tau}_i$ is the estimated trip duration, and s'_j is the destination state. The value function is learned via temporal-difference methods from historical trip data. DiDi’s Cerebellar Value Network (CVNet; Tang et al., 2019) uses hierarchical coarse-coding⁸² with a multi-resolution hexagonal grid, enabling transfer learning across cities and robustness to data sparsity in low-traffic zones.

Production deployment across more than 20 Chinese cities demonstrated consistent improvements. Table 9 summarizes the reported gains from A/B tests using time-slice rotation, where algorithms alternate control of the platform in 3-hour blocks to avoid interference effects.

Li et al. (2019) addressed the coordination challenge using mean-field multi-agent RL. With thousands of drivers making simultaneous decisions, the full multi-agent state space is intractable. The mean-field approximation replaces individual driver states with an aggregate distribution, allowing each driver to condition on the density of nearby drivers rather than

⁸¹A semi-Markov decision process generalizes the standard MDP by allowing variable time between decisions. The discount factor γ^{τ_k} depends on the actual time elapsed τ_k rather than applying a fixed per-step discount.

⁸²Coarse-coding represents a continuous state using overlapping receptive fields at multiple resolutions. At coarser levels, each tile covers a large region, capturing broad spatial patterns; finer levels capture local detail. The value function is the sum of features activated across all resolution levels.

Table 9: DiDi dispatch deployment results from [Qin et al. \(2021\)](#) and [Tang et al. \(2019\)](#).

Metric	Improvement vs. Baseline	Scale
Total driver income	+0.5–2.0%	Tens of millions daily rides
Order response rate	+0.5–2.0%	Platform-wide
Fulfillment rate	+0.5–2.0%	Platform-wide

their exact locations. Experiments on DiDi data showed improved fleet utilization compared to single-agent baselines.

[Han et al. \(2022\)](#) reported complementary results from Lyft, where the dispatching system optimizes driver assignment and repositioning jointly. Their value decomposition architecture⁸³ assigns credit to individual driver decisions within the global objective. The production system demonstrated improvements in rider wait times and driver utilization, confirming that the RL dispatch paradigm generalizes across platforms and regulatory environments.

The percentage improvements appear modest, but at the scale of ride-hailing operations they translate to substantial aggregate effects. A 1% improvement in fulfillment rate at DiDi’s volume represents hundreds of thousands of additional completed rides per day. The central lesson from these deployments is that dispatching is not just a matching problem but a fleet positioning problem. Today’s assignments determine tomorrow’s driver distribution.

7.2 Data Center Cooling

Data centers consume enormous amounts of energy, with cooling systems accounting for 30–40% of total power usage. Google applied machine learning to autonomous HVAC control, achieving substantial efficiency gains while maintaining temperature safety constraints. The problem is a continuous control task with high-dimensional state and action spaces, strict safety requirements, and non-stationary dynamics.

[Gao \(2014\)](#) reported initial results from deploying neural network predictions to guide data center operations. The system used over 100 sensor readings, including temperatures at multiple locations, coolant flow rates, power draw, and external weather conditions, to predict Power Usage Effectiveness (PUE).⁸⁴ Operators used these predictions to adjust equipment setpoints. This assisted approach achieved a 40% reduction in cooling energy and a 15% improvement in overall PUE.

[Lazic et al. \(2018\)](#) subsequently deployed a fully autonomous system using model-predictive control with learned dynamics. The state space consists of temperatures, flow rates, and equipment states across multiple zones. Actions are setpoints for chillers, fans, and pumps. The system learns a neural network dynamics model $\hat{f}$ from operational data:

$$s_{t+1} = \hat{f}(s_t, a_t) + \epsilon_t. \quad (54)$$

At each control interval (5 minutes), the system optimizes over a horizon of future states using the learned model, subject to temperature constraints.

$$\min_{a_0, \dots, a_H} \sum_{h=0}^H c_{\text{energy}}(a_h) \quad \text{s.t.} \quad T_{\min} \leq T_{\text{zone}, i}(\hat{s}_h) \leq T_{\max} \quad \forall i, h. \quad (55)$$

⁸³Value decomposition decomposes the platform’s global matching objective into individual driver value functions, each estimable via temporal-difference learning. The global optimum is recovered by optimizing the sum of individual values.

⁸⁴PUE is the ratio of total facility energy to IT equipment energy. A PUE of 1.0 means all energy goes to computing; the industry average is approximately 1.6.

The MPC approach provides interpretability and safety guarantees that pure model-free RL cannot. Actions are projected onto the feasible set defined by temperature constraints, with uncertainty estimates from the learned model used to maintain conservative margins. Human operators can override at any time. Table 10 summarizes the reported performance.

Table 10: Google data center cooling results from Gao (2014) and Lazic et al. (2018).

Metric	Improvement	Method
Cooling energy	−40%	Neural network predictions
PUE	−15%	Assisted operation
Additional efficiency	−12%	Autonomous MPC
Temperature violations	0	Safety constraints

The system operates continuously across multiple data centers, processing millions of sensor readings per day and controlling thousands of setpoints. The learned model adapts online to seasonal variations, equipment degradation, and workload changes. The lesson is that model-based RL with explicit safety constraints is better suited to critical infrastructure than model-free approaches that require extensive exploration.

7.3 Hotel Revenue Management

Hotels face a dynamic pricing problem with perishable inventory, since rooms unsold by check-in yield zero revenue. Classical revenue management uses booking limit controls derived from dynamic programming under parametric demand assumptions. RL offers an alternative that learns pricing policies from data without specifying demand functional forms.

The problem is a finite-horizon MDP. The state (t, I, d) consists of days until check-in, remaining rooms, and a demand intensity indicator. Actions are discrete price levels. Booking arrivals follow a Poisson process with intensity depending on demand conditions; each arrival books with probability decreasing in price. The objective is to maximize total revenue.

$$V_t(I, d) = \max_{p \in \mathcal{A}} \mathbb{E} [p \cdot B + V_{t-1}(I - B, d') \mid I, d, p], \quad (56)$$

where B is the number of bookings at price p and d' is the next-period demand state.

Gallego and van Ryzin (1994) established the analytical benchmark for dynamic pricing with Poisson demand. Under their intensity control framework, the optimal price trajectory can be characterized when demand follows specific functional forms. The Gallego-van Ryzin solution provides a theoretical upper bound, and any practical pricing system should approach this performance in settings where the model assumptions hold.

Chen et al. (2023) applied deep Q-learning to hotel revenue management, training on simulated booking data under parameterized demand models. The DQN agent uses state features (normalized time, normalized inventory, demand indicators) and outputs Q-values for each price level. Table 11 compares performance against baselines on held-out test periods.

The DQN agent achieves 98.7% of the DP optimal revenue while scaling to capacities where exact DP is infeasible.⁸⁵ Training time is nearly constant in hotel capacity, whereas DP computation grows linearly. The learned policies exhibit interpretable patterns, with prices increasing as inventory depletes and as check-in approaches, with larger adjustments during high-demand periods.

⁸⁵EMSR-b (Expected Marginal Seat Revenue, variant b) is the standard heuristic in airline and hotel revenue management. It computes booking limits by comparing the expected marginal revenue of reserving a unit for a higher fare class against selling it now.

Table 11: Hotel revenue management results from [Chen et al. \(2023\)](#).

Method	Mean Revenue	vs. Fixed Price
Fixed price	2,847	–
Myopic pricing	2,962	+4.0%
EMSR-b heuristic	3,156	+10.9%
Exact DP	3,241	+13.8%
DQN	3,198	+12.3%

7.4 Financial Order Execution

Optimal execution of large equity orders requires balancing execution risk (adverse price movements while holding inventory) against market impact (price deterioration from trading aggressively). The theoretical benchmark is the Almgren-Chriss framework ([Almgren and Chriss, 2001](#)), which derives optimal deterministic schedules under linear impact assumptions. Real markets exhibit state-dependent liquidity that these assumptions miss.

[Nevmyvaka et al. \(2006\)](#) applied tabular Q-learning to execution on real limit order book data from NASDAQ stocks (AMZN, QCOM, NVDA). The state consists of time remaining, inventory remaining, bid-ask spread, and signed volume imbalance.⁸⁶ Actions are trade sizes or the decision to wait. The reward is the negative implementation shortfall,⁸⁷ the difference between execution price and a benchmark. The agent learns to condition on market microstructure signals, trading aggressively when spreads are narrow, wait when order flow predicts favorable price movement, accelerate near the deadline.

Table 12: Execution results from [Nevmyvaka et al. \(2006\)](#) on NASDAQ stocks. TWAP is the time-weighted average price baseline (equal-sized trades at uniform intervals).

Stock	RL vs. TWAP	RL vs. Almgren-Chriss	Data
AMZN	–18.2%	–14.6%	500 days LOB
QCOM	–22.1%	–19.3%	500 days LOB
NVDA	–15.8%	–12.1%	500 days LOB

The negative percentages indicate lower costs for the RL agent. Improvements of 12–19% over the Almgren-Chriss baseline are economically significant. For institutional traders executing billions of dollars annually, this represents substantial savings. Several investment banks have subsequently deployed production execution algorithms built on these ideas, using deep RL with richer state representations.

7.5 Supply Chain Inventory Management

Multi-echelon inventory systems coordinate ordering decisions across supply chain stages, where each stage’s order becomes the next stage’s incoming shipment. A retailer orders from a warehouse, which orders from a distribution center, which orders from a factory. The sequential nature creates complex dependencies, since an order placed upstream today affects downstream availability many periods later. The state space grows exponentially in the number of echelons,

⁸⁶The limit order book records all outstanding buy and sell orders at each price level. Signed volume imbalance is the difference between buy and sell order volume near the best prices, normalized by total volume; positive imbalance typically predicts short-term price increases.

⁸⁷Implementation shortfall measures the total cost of executing a trade relative to a benchmark price, typically the mid-quote at order arrival. It captures market impact, timing cost, and opportunity cost of unexecuted shares.

with K stages each having M possible inventory levels yielding M^K states. Classical inventory theory provides elegant solutions for special cases, most notably the echelon base-stock policy of [Clark and Scarf \(1960\)](#).

The MDP formulation for a serial K -echelon system is as follows. The state $s = (I_1, \dots, I_K)$ records on-hand inventory at each stage, where stage 1 faces customer demand and stage K is the most upstream. The action $q \in \{0, 1, \dots, Q_{\max}\}$ is the order quantity placed at stage K . Demand $D_t \sim F_D$ arrives at stage 1 each period; unfilled demand is backordered. Shipments flow downstream, with stage k receiving what stage $k + 1$ shipped in the previous period. The per-period cost combines holding, backorder, and ordering components.

$$c_t = h \sum_{k=1}^K I_k^+ + b \cdot (D_t - I_1)^+ + c_o \cdot q_t, \quad (57)$$

where $I_k^+ = \max(0, I_k)$ is on-hand inventory, $(x)^+ = \max(0, x)$, h is holding cost per unit, b is backorder cost per unit, and c_o is ordering cost. The objective is to minimize expected discounted total cost.

[Gijsbrechts et al. \(2022\)](#) conducted a systematic evaluation of deep RL against classical base-stock policies across lost sales, dual sourcing, and multi-echelon configurations. The classical benchmark is the echelon base-stock policy, in which each stage k maintains an echelon inventory position and orders to bring this position to a target level S_k .⁸⁸ For single-echelon systems with backorders, the optimal base-stock level is given by the newsvendor critical fractile $S^* = F_D^{-1}(b/(b + h))$. Table 13 summarizes results from their multi-echelon experiments.

Table 13: Multi-echelon inventory results from [Gijsbrechts et al. \(2022\)](#).

Echelons	DRL Cost Gap vs. Base-Stock	States	Convergence
2	+1.2%	$\sim 10^2$	Achieved
3	+6.8%	$\sim 10^3$	Achieved
4	+12.3%	$\sim 10^5$	Partial
6	—	$\sim 10^8$	Failed

The results are sobering for RL proponents. In settings where classical solutions exist and can be computed, RL adds little value and often underperforms. The base-stock policy, derived analytically for idealized settings, remains highly competitive when properly calibrated. RL struggles particularly with multi-echelon coupling, where orders at upstream stages affect costs at downstream stages many periods later, creating difficult credit assignment problems.⁸⁹ The authors report that DRL required millions of training transitions to approach performance levels that base-stock policies achieve through closed-form calculation. The value proposition for RL in inventory management lies in problems where analytical solutions are unavailable, such as non-stationary demand, complex operational constraints, or settings where the cost structure does not admit tractable decomposition.

7.6 Real-Time Bidding

Real-time bidding for display advertising presents a budget pacing problem in which an advertiser must allocate a fixed budget across a sequence of auctions to maximize conversions. [Wu](#)

⁸⁸Formally, the echelon inventory position at stage k equals on-hand inventory at k plus all inventory at downstream stages $1, \dots, k - 1$ plus in-transit inventory, minus backorders. The echelon base-stock policy of [Clark and Scarf \(1960\)](#) is optimal for serial systems with linear costs and backorders.

⁸⁹Credit assignment refers to the difficulty of determining which past actions caused a delayed reward or cost. In multi-echelon systems, an upstream ordering decision today may not affect customer-facing costs for many periods, making it difficult for RL to learn the causal connection.

et al. (2018) formalized this as an MDP where the state includes remaining budget, time elapsed, and recent win rates, with actions being bid multipliers. The RL agent learns to adapt bidding intensity to market conditions, increasing bids when competition is low and conserving budget when prices spike. Simulation experiments calibrated to real RTB data showed improvements over rule-based pacing algorithms.

7.7 Simulation Study: Bus Engine Replacement

I conclude with a simulation demonstrating that RL matches dynamic programming on a classical economics benchmark. The bus engine replacement problem, introduced by Rust (1987), models a fleet manager’s monthly decision whether to replace engines based on accumulated mileage. Replacement incurs a fixed cost but resets mileage to zero; continued operation incurs maintenance costs increasing in mileage.

I extend the single-engine problem to a fleet of N engines with a capacity constraint limiting replacements per period. The state $s = (m_1, \dots, m_N)$ records discretized mileage for each engine. Actions are subsets of engines to replace, subject to the capacity constraint. The per-period cost is $c(s, a) = \alpha \sum_i m_i + \beta |a|$, where α is the operating cost per unit mileage and β is the replacement cost.⁹⁰ Mileage evolves deterministically; replaced engines reset to $m = 0$; others increment by one bin. With $M = 6$ mileage bins, the state space is 6^N , yielding 1,296 states at $N = 4$, 7,776 at $N = 5$, 46,656 at $N = 6$. Value iteration is feasible for $N \leq 5$.

Table 14 compares dynamic programming, DQN, and heuristic baselines across fleet sizes.

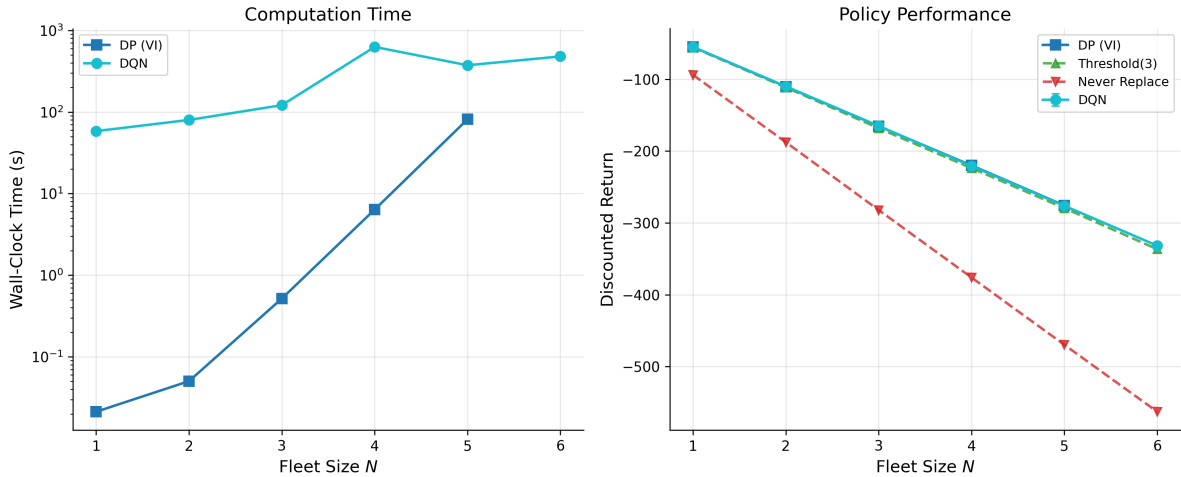


Figure 7: Bus engine replacement benchmark. Left: computation time vs. fleet size (log scale). Right: discounted return vs. fleet size for DP, DQN, and heuristic baselines.

For $N = 1$ through 5 where both methods are computable, DQN matches DP within 1% of the optimal discounted return. At $N = 6$ (46,656 states), DP is infeasible but DQN produces a policy. The threshold heuristic, which replaces engines above a mileage cutoff, provides a reasonable baseline but cannot account for capacity constraints or the joint state of multiple engines. The never-replace heuristic performs poorly due to accumulated mileage costs, confirming that the cost structure creates a non-trivial replacement decision.

⁹⁰The mileage-dependent operating cost $\alpha \sum_i m_i$ follows Rust’s original specification $c(x, \theta_1) = \theta_{11}x$, creating a non-trivial threshold replacement policy. The fleet extension uses deterministic mileage increments to isolate the combinatorial scaling challenge that arises from the joint state of multiple engines.

Table 14: Bus engine replacement results across fleet sizes.

N	$ \mathcal{S} $	DP Time	DP Return	DQN Return	Thresh(3)	Never Replace
1	6	0.02s	-55.1	-55.1 ± 0.1	-55.7	-94.0
2	36	0.05s	-110.3	-110.0 ± 0.3	-111.4	-187.7
3	216	0.52s	-165.3	-165.0 ± 0.1	-168.3	-282.1
4	1,296	6.38s	-220.0	-220.8 ± 0.1	-223.8	-376.0
5	7,776	81.33s	-275.7	-276.2 ± 0.1	-279.5	-469.7
6	46,656	—	—	-331.7 ± 1.1	-336.6	-562.8

8 Structural Estimation with Reinforcement Learning

Several recent papers have used RL training loops, namely Q-learning, temporal-difference learning, policy gradient, and actor-critic methods, to solve structural economic models at scales where conventional dynamic programming fails.⁹¹ Throughout this chapter I adopt a unified notation. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s, a)$ is the transition kernel, $r(s, a)$ is the per-period reward, and $\gamma \in [0, 1)$ is the discount factor.⁹² A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to distributions over actions. The value function under π is $V^\pi(s) = \mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s]$, and the action-value function is $Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} [V^\pi(s')]$. The optimal value function satisfies $V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$.

8.0.1 Adusumilli and Eckardt (2022): TD Learning for CCP Estimation

Adusumilli et al. (2022) adapt temporal-difference (TD) learning to estimate the recursive terms that arise in CCP-based estimation, entirely avoiding specification or estimation of transition densities.

The CCP approach requires computing two functions $h : \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}^d$ and $g : \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ that solve the recursive equations

$$h(a, s) = z(s, a) + \gamma \mathbb{E}[h(a', s') \mid a, s], \quad (58)$$

$$g(a, s) = e(a, s) + \gamma \mathbb{E}[g(a', s') \mid a, s], \quad (59)$$

where $e(a, s) = \gamma_E - \ln P(a|s)$ under logit errors (γ_E denoting the Euler constant), and the expectation is over the next-period state-action pair (s', a') given the transition kernel $P(s'|s, a)$ and the observed policy $P(a|s)$. Both h and g have a value-function structure; they satisfy a Bellman-like recursion under the observed (data-generating) policy, not the optimal policy.

This observation motivates the approach. Standard TD learning estimates value functions under a fixed policy from trajectory data, which is exactly what is needed here. Adusumilli et al. (2022) propose two methods.

The core idea is to approximate h and g with linear combinations of basis functions, then use regression on observed state-action sequences to estimate the weights. This reduces the infinite-dimensional fixed-point problem to a finite-dimensional linear system.

The first is the linear semi-gradient method.⁹³ Approximate $h(a, s) \approx \phi(a, s)^\top w$ where

⁹¹I exclude papers that use neural network function approximation without an RL training mechanism. Inverse reinforcement learning is treated in the sister survey (Rust and Rawat, 2026).

⁹²Several of the papers reviewed here use β for the discount factor, following economics convention. I translate all results to γ for consistency with the RL literature and the rest of this survey.

⁹³The method is called “semi-gradient” because it computes the gradient of only the prediction $\phi(a, s)^\top w$ with respect to w , not the full TD error including the bootstrap target $\phi(a', s')^\top w$. This avoids differentiating through the target but sacrifices guaranteed convergence in some settings.

$\phi : \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}^p$ is a vector of basis functions (e.g., polynomials in state variables) and $w \in \mathbb{R}^p$ are the weights to be estimated. The TD(0) fixed-point equation is

$$\mathbb{E} \left[\phi(a, s) (\phi(a, s) - \gamma \phi(a', s'))^\top \right] w = \mathbb{E} [\phi(a, s) z(s, a)]. \quad (60)$$

The sample analog replaces population expectations with averages over the observed panel.

$$\hat{w} = \left(\frac{1}{n(T-1)} \sum_{i=1}^n \sum_{t=1}^{T-1} \phi_{it} (\phi_{it} - \gamma \phi_{i,t+1})^\top \right)^{-1} \left(\frac{1}{n(T-1)} \sum_{i=1}^n \sum_{t=1}^{T-1} \phi_{it} z_{it} \right), \quad (61)$$

where $\phi_{it} = \phi(a_{it}, s_{it})$ and $z_{it} = z(s_{it}, a_{it})$. This requires inverting a $p \times p$ matrix, where p is the number of basis functions, making computation trivial in most settings. No transition density estimation is needed, since the method uses only observed sequences of current and next-period state-action pairs.

The second method is approximate value iteration (AVI). Rather than fixing a linear basis, AVI constructs a sequence of approximations $\hat{h}^{(k)}$ by iterating the Bellman-like operator and solving a nonparametric regression at each step. At iteration k , given the current approximation $\hat{h}^{(k-1)}$, one constructs pseudo-outcomes

$$Y_{it}^{(k)} = z_{it} + \gamma \hat{h}^{(k-1)}(a_{i,t+1}, s_{i,t+1}) \quad (62)$$

and then fits $\hat{h}^{(k)}$ by regressing $Y_{it}^{(k)}$ on (a_{it}, s_{it}) using any machine learning method, including LASSO, random forests, or neural networks. This is the first DDC estimator compatible with arbitrary ML prediction methods, enabling application to very high-dimensional state spaces.

With $\hat{h}$ and $\hat{g}$ in hand, structural parameters are recovered by pseudo-maximum likelihood estimation (PMLE). For continuous state spaces, [Adusumilli et al. \(2022\)](#) derive a locally robust correction to the PMLE criterion that accounts for the nonparametric first-stage estimation of value terms, restoring $\sqrt{n}$ -convergence of $\hat{\theta}$.⁹⁴ The PMLE score is $m(a, s; \theta, h, g) = \partial_\theta \ln \pi(a, s; \theta, h, g)$, where π is the logit choice probability with continuation value $V(a, s) = h(a, s)^\top \theta + g(a, s)$. The naive estimator solves $\mathbb{E}_n[m(a, s; \theta, \hat{h}, \hat{g})] = 0$, but with continuous states this moment condition is not orthogonal to the first-stage estimates and converges slower than $\sqrt{n}$. The locally robust moment adds a debiasing correction:

$$\zeta = m(a, s; \theta, h, g) - \lambda(a, s; \theta) \left\{ z(s, a)^\top \theta + \gamma e(a', s') + \gamma V(a', s') - V(a, s) \right\}, \quad (63)$$

where $\lambda(a, s; \theta)$ solves a backward recursion that propagates the influence of estimation error through the dynamic structure.⁹⁵ The corrected estimator $\hat{\theta}_{LR}$ solves $\mathbb{E}_n[\zeta_n] = 0$ and is computationally no harder than the naive PMLE, since the correction is constant in θ .

Theorem 1 ([Adusumilli et al. \(2022\)](#), Theorem 1). *Under regularity conditions, the linear semi-gradient estimator $\hat{h}$ satisfies $\|\hat{h} - h\|_2 = O_P(n^{-1/2}(T-1)^{-1/2})$, where $\|\cdot\|_2$ denotes the $L^2(P)$ norm.*⁹⁶

Theorem 2 ([Adusumilli et al. \(2022\)](#), Theorem 5). *Under regularity conditions on the ML method used in AVI, the locally robust PMLE estimator $\hat{\theta}_{LR}$ satisfies $\sqrt{n}(\hat{\theta}_{LR} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Sigma)$ for an explicit variance Σ , even when the state space is continuous.*

⁹⁴An estimator has $\sqrt{n}$ -convergence if its error shrinks at rate $n^{-1/2}$ where n is the sample size. This is the standard parametric rate; slower rates (e.g., $n^{-1/4}$) indicate efficiency loss from nonparametric first stages.

⁹⁵The term in braces is the temporal-difference error of the continuation value V . The adjoint λ weights this TD error by its marginal impact on the PMLE score. See Online Appendix B.3 of [Adusumilli et al. \(2022\)](#) for the derivation.

⁹⁶The $L^2(P)$ norm is $\|f\|_2 = (\int f(x)^2 dP(x))^{1/2}$, measuring average squared deviation under the probability measure P . This is the natural norm for mean-squared-error analysis.

Monte Carlo experiments on a dynamic firm entry model with seven structural parameters and five continuous state variables show that the TD-based estimators achieve a 4- to 11-fold reduction in mean squared error compared to CCP estimators using state-space discretization.

For dynamic discrete games, the method extends naturally. Standard CCP-based estimation of games requires integrating out other players' actions, which becomes intractable with many players or continuous states. TD learning avoids this entirely, since it works directly with the joint empirical distribution of states and their successors. The “integrating out” is done implicitly within sample expectations.

The limitation is the linear semi-gradient's dependence on the choice of basis functions, though cross-validation can guide this selection. AVI with flexible ML methods relaxes this constraint at the cost of slightly higher small-sample bias, which the locally robust correction addresses.

8.0.2 Hu and Yang (2025): Policy Gradient for DDC Estimation

Hu and Yang (2025) combine policy gradient methods with the Simulated Method of Moments (SMM) to estimate DDCs, with particular focus on models with unobserved state variables. Where the previous subsection replaced the inner fixed-point computation with TD learning, Hu and Yang replace it with policy gradient, sidestepping value function approximation entirely and working directly in the space of parametrized policies.

The outer loop is SMM.⁹⁷ Define a vector of data moments $\mathbf{M}_d$ computed from the observed panel. For candidate structural parameters θ and transition parameters ξ , simulate the model to produce simulated moments $\mathbf{M}_s(\theta, \xi)$. The estimator minimizes

$$(\hat{\theta}, \hat{\xi}) = \arg \min_{\theta, \xi} (\mathbf{M}_d - \mathbf{M}_s(\theta, \xi))^\top \mathbf{W} (\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)), \quad (64)$$

where $\mathbf{W}$ is a positive definite weight matrix. Computing $\mathbf{M}_s(\theta, \xi)$ requires solving for the optimal policy under (θ, ξ) , which is the inner-loop problem.

The inner loop parametrizes the choice probability directly as a logistic function of state variables. For a binary choice $J_t \in \{0, 1\}$, the general form is $\Pr(J_t = 1 \mid \mathbf{X}_t; \gamma(\theta)) = \text{logistic}(\mathbf{X}_t \gamma(\theta))$, where $\gamma(\theta)$ are policy parameters that depend on the structural parameters.⁹⁸ In their application to a Rust bus engine model with unobserved bus condition S_t^* , this takes the form

$$\Pr(J_t = 1 \mid X_t, S_t^*, t; \gamma) = \frac{\exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}{1 + \exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}, \quad (65)$$

where t enters the index directly to capture time-varying replacement incentives. The policy parameters are updated by REINFORCE-style gradient ascent, applying the policy gradient theorem (Sutton et al., 1999):

$$\nabla_\gamma V(\gamma) = \mathbb{E} \left[\sum_{t=0}^T \nabla_\gamma \log \pi_\gamma(J_t \mid X_t, S_t^*, t) Q^{\pi_\gamma}(X_t, S_t^*, J_t) \right], \quad (66)$$

where Q^{π_γ} is the action-value function under the current policy, estimated by Monte Carlo returns from forward-simulated trajectories.

The main contribution is handling unobserved state variables. When state variables are only partially observed, the policy in (65) is parametrized as a function of both X_t and S_t^* , and the algorithm forward-simulates trajectories of both observed and unobserved variables.

⁹⁷SMM (Gourieroux et al., 1993) estimates structural parameters by matching moments from simulated data to moments from observed data, avoiding direct evaluation of the likelihood function.

⁹⁸The linear index can be replaced by higher-order terms of $\mathbf{X}_t$ or deep neural networks; the method requires only that the gradient $\nabla_\gamma \log \pi_\gamma$ has a closed form.

Building on the nonparametric identification results of [Hu and Shum \(2012\)](#), the outer-loop SMM targets moments from five consecutive periods of observed data, which suffice to separately identify the structural parameters θ and transition parameters ξ without requiring the econometrician to observe S_t^* . No discretization of continuous unobserved states is needed; the same algorithm handles both discrete and continuous unobserved heterogeneity.

For each candidate (θ, ξ) in the outer minimization (64), the inner loop runs policy gradient until convergence, producing optimal policy parameters $\gamma^*(\theta, \xi)$. These are used to simulate data and compute $\mathbf{M}_s(\theta, \xi)$.

On an extended Rust bus engine model with a continuous unobserved bus condition following an AR(1) process, estimates of seven structural parameters are centered around their true values across 400 simulations (Table 1 in their paper). On a discrete-unobservable variant of the same model, the method matches the precision of [Arcidiacono and Miller \(2011\)](#)’s two-step EM algorithm, with computation time depending on the number of inner-loop iterations: at 5,000 iterations, RL+indirect inference runs in 0.40 minutes versus 0.66 minutes for CCP estimation; at 20,000 iterations, it rises to 1.15 minutes (Table 2 in their paper). Each outer-loop evaluation requires a full policy gradient convergence in the inner loop, so the computational advantage diminishes as more inner-loop iterations are used for precision.

8.1 Dynamic Oligopoly and Strategic Interaction

Dynamic oligopoly models combine elements of game theory and dynamic programming. Firms choose actions (prices, quantities, investment levels) strategically, anticipating both their own future states and the strategies of competitors. The state space grows combinatorially in the number of firms, and the requirement to compute best responses against other firms’ strategies adds further computational burden.

8.1.1 Asker, Fershtman, Jeon, and Pakes (2020): Q-Learning in Dynamic Procurement Auctions

[Asker et al. \(2020\)](#) develop a computational framework for analyzing dynamic procurement auctions with serially correlated asymmetric information. Their approach builds on the Experience-Based Equilibrium (EBE) concept of [Fershtman and Pakes \(2012\)](#), which computes equilibria by simulating industry trajectories and updating strategies toward best responses. EBE evaluates values only on recurrently visited states rather than the full state space, making it feasible for large state spaces. While [Fershtman and Pakes \(2012\)](#) use a stochastic approximation algorithm to update continuation values from simulated industry trajectories, [Asker et al. \(2020\)](#) add explicit value-function updates via stochastic approximation.

The model is a repeated first-price sealed-bid auction with two firms. Each firm i maintains a private inventory state $\omega_{i,t}$ (stock of unharvested timber, in their application). The state evolves endogenously, as winning an auction increases inventory while harvesting depletes it. Each firm’s private state is not observed by its competitor except at periodic revelation events.

The key computational innovation is the use of Q-learning to compute equilibrium strategies. Each firm i maintains a Q-function $Q_i : \mathcal{S}_i \times \mathcal{A}_i \rightarrow \mathbb{R}$, where $\mathcal{S}_i$ encodes firm i ’s information set (its own inventory, beliefs about the competitor’s inventory, public history) and $\mathcal{A}_i$ is its action set (participation decision and bid level). The Q-function satisfies

$$Q_i(s, a) = \mathbb{E} \left[r_i(s, a, a_{-i}) + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s', a') \mid s, a \right], \quad (67)$$

where $r_i(s, a, a_{-i})$ is firm i ’s per-period profit given the state, its own action a , and the competitor’s action a_{-i} , and the expectation is taken over the competitor’s strategy and the stochastic transitions.

Firms update Q-values using sample averaging.

$$Q_i(s, a) \leftarrow Q_i(s, a) + \frac{1}{h_k(s, a)} \left[r_i + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s', a') - Q_i(s, a) \right], \quad (68)$$

where $h_k(s, a)$ is the number of times state-action pair (s, a) has been visited.⁹⁹ The equilibrium computation proceeds iteratively, with firms simultaneously updating their Q-functions based on simulated play against each other's current strategies. Strategies are derived from Q-values using an ε -greedy rule or Boltzmann exploration.¹⁰⁰

Asker et al. (2020) add a boundary consistency condition to the EBE concept that restricts behavior at the boundary of the recurrent state class, reducing the multiplicity of equilibria. Their numerical analysis reveals that information sharing between firms can, through increased precision of beliefs about competitor states, induce firms to spend more time in states where competition is less intense. The dynamic RL-computed equilibrium yields qualitatively different predictions from both static analysis and myopic ($\gamma = 0$) benchmarks. With dynamics, information sharing decreases average bids and increases average profits, while the myopic benchmark shows negligible effects.

The limitation of this approach is the tabular representation; the Q-function is stored as a lookup table over discretized states and actions, restricting applicability to models with moderate state-space dimension.

8.1.2 Hollenbeck (2019): TD Learning for Merger Analysis with Innovation

Hollenbeck (2019) uses RL to solve a dynamic oligopoly model with endogenous mergers, entry, exit, and quality investment. The model extends the Ericson-Pakes framework to study how horizontal mergers affect innovation incentives.

The industry state is $\Omega = (\omega_1, \dots, \omega_n)$ where $\omega_i \in \{1, \dots, \omega_{\max}\}$ is firm i 's product quality. Firms produce differentiated goods and compete in prices (Bertrand competition with logit demand). In each period, firms simultaneously choose investment levels, entry/exit decisions, and potentially initiate merger negotiations.

Each firm i computes its continuation value $V_i(\Omega)$ from the industry state. Because the state space is the product of all firms' quality levels plus industry structure (number of active firms, recent mergers), exact dynamic programming is infeasible for industries with more than two or three firms. Hollenbeck (2019) instead uses temporal-difference learning to estimate values from simulated industry trajectories.

The value function update for firm i is¹⁰¹

$$V_i(\Omega) \leftarrow V_i(\Omega) + \alpha [\Pi_i(\Omega, \mathbf{a}) + \gamma V_i(\Omega') - V_i(\Omega)], \quad (69)$$

where $\Pi_i(\Omega, \mathbf{a})$ denotes firm i 's per-period profit given industry state Ω and the joint action profile $\mathbf{a}$ (investment, entry/exit, merger decisions),¹⁰² γ is the discount factor, and Ω' is the realized next-period state. The algorithm uses ε -decreasing exploration to prevent convergence to locally suboptimal equilibria.

⁹⁹This sample-averaging rule ($\alpha_k = 1/h_k$) is equivalent to maintaining the running mean of observed returns, as distinct from fixed- α Q-learning which gives exponentially decaying weight to older observations. The update is applied for all actions, including counterfactual actions not taken, using the observed state transition.

¹⁰⁰ ε -greedy selects the greedy action $\arg\max_a Q(s, a)$ with probability $1 - \varepsilon$ and a uniformly random action otherwise. Boltzmann exploration selects action a with probability $\propto \exp(Q(s, a)/\tau)$ where $\tau > 0$ is a temperature parameter.

¹⁰¹This stochastic approximation update, introduced by Pakes and McGuire (1994) for dynamic oligopoly computation, is closely related to temporal-difference learning in the RL literature. The original algorithm uses visit-count averaging ($\alpha_k = 1/k$ where k counts visits to each state) rather than a fixed learning rate.

¹⁰²I use Π_i for firm i 's per-period profit to avoid confusion with policy π .

The equilibrium computation follows the Pakes-McGuire iterative scheme.¹⁰³ The algorithm simulates long industry histories, updates each firm’s value function via (69), re-derives best-response strategies from updated values, and repeats. The RL component replaces the exact Bellman iteration step that would be infeasible over the full state space, focusing computational effort on recurrently visited states.

The central finding is that horizontal mergers, while reducing static consumer surplus in the short run, create a strong incentive for entry and investment. Firms enter with negative static profits because the prospect of a lucrative buyout justifies the initial investment. The result is substantially higher long-run innovation and consumer welfare with mergers than without. This finding reverses the standard static antitrust prediction and can only emerge in a dynamic model where firms are forward-looking.

Two related papers merit brief mention. [Lomys and Magnolfi \(2024\)](#) develop structural estimation methods for strategic settings where agents use learning algorithms (specifically regret-minimizing rules) rather than playing a fixed equilibrium. They impose an “asymptotic no-regret” condition as a minimal rationality requirement and derive identification results for payoff parameters. Their contribution is conceptual, bridging the gap between RL-based behavioral models (where agents learn) and the econometric problem (where the researcher estimates structural parameters from data generated by learning agents). Their estimation method itself is standard econometrics rather than RL. [Covarrubias \(2022\)](#) uses deep RL to study oligopolistic pricing in a New Keynesian framework, representing firms’ pricing policies as neural networks $\pi_\phi(a|s)$. The method uncovers multiple equilibria ranging from competitive to collusive pricing.¹⁰⁴

8.2 Auction Equilibria and Mechanism Design

Computing equilibrium bidding strategies in auctions is a long-standing challenge. Closed-form solutions exist only for narrow families of valuation distributions and auction formats, while numerical methods scale poorly with the number of bidders and items. Recent work has applied deep RL to learn equilibrium strategies and optimal mechanisms.

8.2.1 Brero, Eden, Gerstgrasser, Parkes, and Rheingans-Yoo (2021): RL for Sequential Price Mechanisms

[Brero et al. \(2021\)](#) use RL to design optimal sequential price mechanisms (SPMs), a class of indirect auction mechanisms where agents are approached in sequence and offered menus of items at posted prices. SPMs generalize both serial dictatorship and posted-price mechanisms and essentially characterize all strongly obviously strategyproof (SOSP) mechanisms ([Pycia and Troyan, 2023](#)).¹⁰⁵

The mechanism design problem is formulated as a partially observable Markov decision process (POMDP).¹⁰⁶ The state at round t includes the set of remaining items $\rho_t^{\text{items}} \subseteq [m]$, remaining agents $\rho_t^{\text{agents}} \subseteq [n]$, the partial allocation $\mathbf{x}_t$, and the agents’ (unobserved) valuation

¹⁰³The Pakes-McGuire algorithm ([Pakes and McGuire, 1994](#)) computes Markov perfect equilibria by iterating: (1) given current value functions, compute best-response strategies; (2) given strategies, update value functions via simulation. Convergence is not guaranteed but works well in practice.

¹⁰⁴The most prominent example of algorithmic collusion is [Calvano et al. \(2020\)](#), who showed that independent Q-learning agents in a repeated Bertrand pricing game learn to sustain supra-competitive prices and punish deviators without explicit communication. This result and its implications for competition policy are treated in the companion thesis chapter ([Rawat, 2026](#)).

¹⁰⁵A mechanism is strategyproof if truthful reporting is a dominant strategy. It is obviously strategyproof if this dominance is apparent even to boundedly rational agents; strongly obviously strategyproof (SOSP) adds that dominance holds at every information set ([Pycia and Troyan, 2023](#)).

¹⁰⁶In a POMDP, the agent cannot directly observe the full state. It maintains a belief distribution over possible states, updated via Bayes’ rule as new observations arrive. This captures the mechanism designer’s uncertainty about bidder valuations.

functions. The action $a_t = (i_t, \{p_j^t\}_{j \in \rho_{t-1}^{\text{items}}})$ specifies which agent to visit next and what prices to offer. The observation is the agent’s purchase decision, from which the mechanism can update beliefs about valuations. The reward is the objective function evaluated at the final allocation:

$$r = g(\mathbf{x}_T, \boldsymbol{\tau}_T; \mathbf{v}), \quad (70)$$

where g can be social welfare $\sum_i v_i(x_i)$, revenue $\sum_i \tau_i$, or max-min fairness $\min_i v_i(x_i)$.

A key theoretical result establishes when adaptive mechanisms outperform static ones.

Theorem 3 (Brero et al. (2021), Propositions 1–4, informal). *Each feature of adaptive mechanisms is necessary for welfare optimality, even in simple settings. Personalized prices are needed with one item and two i.i.d. agents (Proposition 1); adaptive prices are needed with two identical items and three i.i.d. agents (Proposition 2); adaptive ordering is needed with six agents whose valuations are correlated (Proposition 3); both adaptive prices and ordering are needed with four agents whose valuations are independently but non-identically distributed (Proposition 4).*

The policy maps from a sufficient statistic of the observation history to actions. Brero et al. (2021) show that this statistic can be represented compactly. The set of remaining items and agents suffices for independent valuations, while the full allocation matrix is needed for correlated valuations. They train the mechanism policy using Proximal Policy Optimization (PPO),¹⁰⁷ which handles the discrete action space (agent selection) and continuous action space (price setting) of the POMDP.

Experimental results show that the learned SPMs achieve near-optimal welfare across settings with up to 20 agents and 5 items (with similar results noted for up to 30 of each), significantly outperforming static pricing benchmarks. The improvement is largest when agent valuations are correlated, since adaptive prices allow the mechanism to infer information about remaining agents from earlier purchases.

The limitation is that the POMDP formulation requires knowledge of the prior distribution over valuations, which in practice must be estimated from data. The method also does not scale easily to very large numbers of items due to the combinatorial action space.

8.2.2 Ravindranath, Feng, Wang, Zaheer, Mehta, and Parkes (2024): Fitted Policy Iteration for Combinatorial Auctions

Ravindranath et al. (2024) address revenue-maximizing mechanism design for combinatorial auctions with multiple items and strategic bidders. Their innovation is integrating differentiable auction structure into a fitted policy iteration framework, enabling analytical gradient computation where standard RL methods struggle with high variance.

The mechanism visits agents one at a time in sequence. Each agent i , upon being visited, selects a bundle of items from those still available, given posted prices. Valuations are drawn once from distributions V_i and remain fixed throughout the mechanism. Complementarities arise from the structure of the valuation function over bundles, not from dynamic evolution. The MDP state at step t is $s_t = (i_t, S_t)$, where i_t is the current bidder and S_t is the set of remaining items.

The mechanism’s policy maps from state to a price vector over available items. The key technical contribution is making the auction clearing differentiable. In a standard auction, the allocation is an argmax over bids (non-differentiable), and payments depend discontinuously on the allocation. Ravindranath et al. (2024) replace the hard bundle selection with a softmax relaxation,¹⁰⁸ enabling analytical gradient computation through the mechanism. The actor

¹⁰⁷Proximal Policy Optimization (Schulman et al., 2017) is a policy gradient algorithm that constrains each update to a trust region, preventing large destabilizing policy changes. It is described in Chapter 2.

¹⁰⁸The softmax relaxation replaces the hard allocation $\operatorname{argmax}_i b_i$ with a soft allocation $\exp(b_i/\tau) / \sum_j \exp(b_j/\tau)$, which is differentiable and approaches the hard allocation as $\tau \rightarrow 0$.

loss is the negative expected revenue, and gradients flow directly through the softmax-relaxed allocation and payment rules. The paper explicitly avoids REINFORCE-style estimators, noting that analytical gradients overcome the sample inefficiency and high variance of score-function methods.

The method follows fitted policy iteration (Bertsekas and Tsitsiklis, 1996), alternating between evaluating the current policy (computing expected revenue) and improving it via gradient ascent on the actor loss. This differs from model-free RL approaches (PPO, SAC) that the paper uses as baselines.

Experiments on settings with additive and combinatorial valuations show that the learned mechanisms achieve up to 13% higher revenue than item-wise Myerson optimal auctions, with the largest gains in combinatorial settings where bundle complementarities make item-wise pricing suboptimal. Standard RL baselines (PPO, SAC) are also outperformed, confirming the advantage of exploiting differentiable structure.¹⁰⁹

8.3 Macroeconomic Models

Reinforcement learning and deep learning are increasingly used to solve high-dimensional macroeconomic models where grid-based dynamic programming is infeasible. Heterogeneous agent models, in which a distribution of agents with different wealth levels, productivities, or beliefs interact through markets, generate state spaces that grow with the number of agent types and asset positions. Maliar et al. (2021) demonstrate that deep neural networks can approximate policy and value functions in dynamic economic models, achieving accuracy comparable to established projection methods while scaling to problems with dozens of state variables. Fernández-Villaverde et al. (2023) solve a model with financial frictions and an endogenous wealth distribution using deep learning, obtaining global solutions to a problem where perturbation methods fail due to strong nonlinearities. Fernández-Villaverde et al. (2024) provide a systematic treatment of deep learning methods for high-dimensional dynamic programming problems in economics, covering both single-agent and equilibrium settings. Atashbar and Shi (2023) apply deep deterministic policy gradient (DDPG)¹¹⁰ to a real business cycle model, demonstrating that model-free RL can recover near-optimal consumption and investment policies without deriving optimality conditions such as the Euler equation. This is a rapidly growing area; readers seeking a comprehensive methodological treatment are directed to the cited papers and the references therein.

8.4 Optimal Policy Design

Zheng et al. (2022) introduce the AI Economist, a two-level multi-agent reinforcement learning framework for automated tax policy design. In their environment, a population of AI worker agents learn to work, trade, and build in a spatial-economic simulation, while a government RL agent simultaneously learns tax brackets that optimize a social welfare objective. The worker agents are trained with PPO to maximize individual post-tax utility, and the government agent is trained with PPO to maximize a weighted combination of equality (measured by the Gini index) and productivity (total output). The two-level structure produces a Stackelberg game between the government (leader) and the workers (followers), where the government must anticipate how tax policy changes affect worker behavior. The learned tax policies achieve equality-productivity tradeoffs that Pareto-dominate several analytical baselines, including the Saez tax formula. This framework extends the mechanism design perspective of the preceding subsection from auctions to fiscal policy, using multi-agent RL to jointly solve for optimal mechanisms and equilibrium responses.

¹⁰⁹DDPG was also tested but found to be unstable. DQN is not applicable to continuous price-setting.

¹¹⁰DDPG extends Q-learning to continuous action spaces by learning a deterministic policy network alongside a Q-function critic.

9 Reinforcement Learning in Games

Earlier chapters treated single-agent problems: one agent optimizes against a fixed environment whose transition dynamics $P(s'|s, a)$ are stationary. This chapter introduces multiple agents who adapt simultaneously. Agent i 's environment includes agents $-i$, whose policies change as they learn. The “transition” each agent faces is non-stationary, and the Bellman equation, which assumes a fixed environment, no longer directly applies. The question becomes: can adaptive agents converge to equilibrium? Shoham et al. (2007) enumerate five questions that define the multi-agent learning research agenda: (1) convergence, whether the algorithm converges to a stationary strategy in self-play; (2) rationality, whether it best-responds against stationary opponents; (3) equilibrium attainment, whether convergence yields a Nash equilibrium; (4) safety, whether it guarantees at least the Nash-value payoff against arbitrary opponents; (5) social welfare, whether it can achieve Pareto-superior outcomes when partners cooperate. No existing algorithm satisfies all five in general games; the trade-offs explored in this chapter are the field's central preoccupation.

Two paradigms emerged to address this question. Value-based methods generalize the Bellman operator to games, replacing the max in standard Q-learning with game-theoretic solution concepts (minimax, Nash). Regret-based methods minimize counterfactual regret without ever computing equilibria, instead accumulating regrets and averaging strategies over time. The two paradigms target different game classes through different mechanisms. Value-based methods operate on stochastic games with simultaneous moves and observable payoffs. CFR operates on extensive-form games with sequential moves and private information. Value-based convergence typically requires equilibrium uniqueness or specific game structure; CFR converges in all two-player zero-sum extensive-form games regardless of equilibrium multiplicity. Computing Nash equilibria is itself PPAD-complete (Daskalakis et al., 2009), so neither approach escapes the fundamental hardness, but both achieve convergence in the game classes they target.

9.1 Stochastic Games and Equilibrium Learning

9.1.1 The Stochastic Game Framework

An n -player stochastic game $\Gamma = (n, \mathcal{S}, \mathcal{A}_1, \dots, \mathcal{A}_n, P, r_1, \dots, r_n, \gamma)$ consists of a finite state space $\mathcal{S}$; finite action sets $\mathcal{A}_i$ for each player i ; a transition function $P : \mathcal{S} \times \mathcal{A}_1 \times \dots \times \mathcal{A}_n \rightarrow \Delta(\mathcal{S})$; reward functions $r_i : \mathcal{S} \times \mathcal{A}_1 \times \dots \times \mathcal{A}_n \rightarrow \mathbb{R}$; and a common discount factor $\gamma \in [0, 1)$. At each stage, all players simultaneously choose actions, receive individual rewards, and the game transitions to a new state.¹¹¹

A Markov decision process is a stochastic game with $n = 1$. A matrix game (normal-form game) is a stochastic game with $|\mathcal{S}| = 1$. The stochastic game framework unifies both. Each player i seeks a policy $\pi_i : \mathcal{S} \rightarrow \Delta(\mathcal{A}_i)$ maximizing discounted return $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_i(s_t, a_{1,t}, \dots, a_{n,t})]$.

The central difficulty is non-stationarity. When all agents learn, the environment each agent faces changes as opponents adapt. Standard Q-learning convergence (Watkins and Dayan, 1992) requires stationary transition and reward dynamics; with multiple learners, this assumption fails. Bowling and Veloso (2002) formalized two properties a learning algorithm should satisfy. It is *rational* if, when all other players converge to stationary policies, it converges to a best response; it is *convergent* if, in self-play, it converges to a stationary policy.

If all players use rational, convergent algorithms, the resulting stationary policy profile is a Nash equilibrium: each player best-responds to the others' stationary policies, and a profile of mutual best responses is, by definition, a Nash equilibrium. The challenge is achieving both properties simultaneously.

¹¹¹Shapley (1953) introduced stochastic games in 1953 for the two-player zero-sum case, proving existence of the value via a contraction argument on the Bellman operator. The general-sum extension to n players is due to Fink (1964) and Takahashi (1964).

9.1.2 Minimax-Q Learning

Littman (1994) proposed the first Q-learning algorithm for stochastic games, targeting two-player zero-sum games. The key modification replaces the max operator in the standard Q-learning backup with a minimax operator. Each agent maintains $Q_i(s, a_i, a_{-i})$ over the joint action space. The update rule is

$$Q_i(s, a_i, a_{-i}) \leftarrow (1 - \alpha) Q_i(s, a_i, a_{-i}) + \alpha [r_i + \gamma V_i(s')], \quad (71)$$

where the value backup solves a linear program:

$$V_i(s) = \max_{\pi_i \in \Delta(\mathcal{A}_i)} \min_{a_{-i} \in \mathcal{A}_{-i}} \sum_{a_i \in \mathcal{A}_i} \pi_i(a_i) Q_i(s, a_i, a_{-i}). \quad (72)$$

This is the RL analogue of Shapley’s value iteration for zero-sum stochastic games (Shapley, 1964). The agent does not merely maximize; it secures the best guaranteed payoff against the worst-case opponent. The resulting policy $\pi_i(s)$ is generally a mixed strategy, since deterministic policies are exploitable in adversarial settings.

Minimax-Q converges to the minimax Q-values under Robbins-Monro learning rates ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$) and infinite exploration of all state-action tuples.¹¹² However, the algorithm sacrifices rationality: it plays the equilibrium strategy even against exploitable opponents. In the soccer game of Littman (1994), minimax-Q trained against a hand-built opponent won 53.7% of games, while standard Q-learning won only 26.1%. Against an adversarially trained challenger, Q-learning won 0% of games because its deterministic policy was fully predictable; minimax-Q won 37.5%, maintaining robustness through mixed strategies.

9.1.3 Nash-Q Learning

Hu and Wellman (2003) extended the framework to general-sum stochastic games, where players may have aligned, opposed, or mixed incentives. Each agent i maintains a Q-function over the joint action space $Q_i(s, a_1, \dots, a_n)$ and updates via

$$Q_i(s, \mathbf{a}) \leftarrow (1 - \alpha) Q_i(s, \mathbf{a}) + \alpha [r_i + \gamma \text{Nash}_i(Q_1(s'), \dots, Q_n(s'))], \quad (73)$$

where $\text{Nash}_i(\cdot)$ denotes player i ’s payoff under a Nash equilibrium of the stage game defined by the current Q-values $(Q_1(s'), \dots, Q_n(s'))$. At each backup, the algorithm treats the Q-values as payoff matrices, computes a Nash equilibrium of this matrix game, and uses the equilibrium payoffs for the value estimate. This bridges dynamic programming and game theory: the stage-game Nash computation is equivalent to solving a static game at each Bellman backup.

Theorem 4 (Hu and Wellman (2003)). *Nash-Q converges to Nash Q-values under Robbins-Monro learning rates and infinite exploration, provided every stage game encountered during learning has either (a) a global optimal point (all agents receive their highest payoff at the same joint action) or (b) a unique saddle-point Nash equilibrium.*

These conditions are restrictive. When stage games have multiple Nash equilibria, agents may select different equilibria for their backups, causing divergence. In the experiments of Hu and Wellman (2003), a grid game with a unique equilibrium Q-function converged in 100% of trials under Nash-Q, compared to 20% under independent Q-learning. A grid game with three equilibrium Q-functions converged in only 68–90% of trials, depending on the equilibrium

¹¹²The convergence proof extends the contraction argument for standard Q-learning. Because the zero-sum minimax operator is a contraction with modulus γ under the ℓ^∞ norm, the stochastic approximation converges to the fixed point. See Littman (1994) and the general treatment in Szepesvári and Littman (1999).

selection rule. Nash-Q reduces to standard Q-learning in the single-agent case, since the “Nash equilibrium” of a one-player game is simply the payoff-maximizing action.¹¹³

9.1.4 The Convergence Problem

Table 15 summarizes the trade-offs across algorithms. No single algorithm achieves both rationality and convergence in general games. Independent Q-learning is rational (it best-responds to stationary opponents) but not convergent in self-play; its deterministic policy cycles in games requiring mixed strategies. Minimax-Q is convergent (guaranteed to find minimax values) but not rational; it plays equilibrium even against exploitable opponents. Nash-Q’s convergence is conditional on the restrictive equilibrium uniqueness assumptions of Theorem 1.

Table 15: Multi-agent Q-learning algorithms: convergence and information requirements

Algorithm	Rational	Convergent	Game class	Information
Indep. Q-learning	Yes	No	Any	Own reward
Minimax-Q	No	Yes	Zero-sum	Joint actions
Nash-Q	Cond.	Cond.	General-sum	All rewards
WoLF-PHC	Yes	Yes (2×2)	General-sum	Own reward

WoLF-PHC (Win or Learn Fast, Policy Hill-Climbing) of [Bowling and Veloso \(2002\)](#) achieves both properties in restricted settings. The algorithm maintains a policy $\pi_i(a|s)$ and a running average policy $\bar{\pi}_i(a|s)$, and updates Q-values over own actions as in standard Q-learning. The policy moves toward the greedy action at a variable rate:

$$\delta = \begin{cases} \delta_l & \text{if } \sum_a \pi_i(a|s) Q_i(s, a) < \sum_a \bar{\pi}_i(a|s) Q_i(s, a) \quad (\text{losing}) \\ \delta_w & \text{otherwise} \quad (\text{winning}) \end{cases} \quad (74)$$

with $\delta_l > \delta_w$. When the current policy underperforms the historical average (losing), the agent adapts quickly; when outperforming (winning), it adapts slowly to avoid destabilizing the opponent. [Bowling and Veloso \(2002\)](#) proved that WoLF-IGA (the infinitesimal gradient ascent variant) converges to Nash equilibrium in all two-player, two-action games. The trajectory traces piecewise ellipses around the equilibrium, shrinking by a factor of $\ell^4 < 1$ per orbit, where $\ell = \sqrt{\delta_w/\delta_l}$. WoLF-PHC requires only own-reward observations, the same information as independent Q-learning.

Two further algorithms deserve mention. Friend-or-Foe Q-learning ([Littman, 2001](#)) decomposes agents as cooperative (friend) or adversarial (foe), using max for friends and minimax for foes; it always converges but requires knowing the relationship type a priori.¹¹⁴ The evolutionary perspective of [Börgers and Sarin \(1997\)](#) provides a deeper lens: reinforcement learning dynamics in matrix games converge to the replicator equation from evolutionary game theory. The cycling of Q-learning in games such as matching pennies is structurally identical to the cycling of biological evolution in Rock-Paper-Scissors games. For economists, this connection is natural: the replicator dynamics are the standard model of boundedly rational adaptation in populations.

¹¹³Nash-Q requires each agent to observe all other agents’ rewards and to maintain Q-values over the joint action space $\mathcal{A}_1 \times \dots \times \mathcal{A}_n$. Storage is $O(n|S| \prod_i |\mathcal{A}_i|)$, exponential in the number of players.

¹¹⁴Correlated-Q learning ([Greenwald and Hall, 2003](#)) generalizes both Nash-Q and Minimax-Q by using correlated equilibrium, a probability distribution over joint actions enforced by a correlating device. The set of correlated equilibria contains all Nash equilibria. Correlated-Q converges under conditions analogous to Nash-Q.

9.1.5 Simulation Study: Cournot and Bertrand Duopoly

Two canonical games from industrial organization serve as benchmarks. In Cournot duopoly, two firms choose quantities $q_i \in \{0, 1, \dots, 9\}$ with inverse demand $P(Q) = 10 - Q$ and marginal cost $c = 1$; the unique Nash equilibrium is $q^* = 3$ with profit $\pi^* = 9$. In Bertrand duopoly with differentiated products, two firms choose prices $p_i \in \{0, 1, \dots, 9\}$ with demand $d_i = 10 - 2p_i + p_j$ and marginal cost $c = 1$; the continuous Nash equilibrium is $p^* \approx 4.33$, which discretizes to $p^* = 4$ with profit $\pi^* = 18$. Both games have unique Nash equilibria in pure strategies. Three algorithms compete: independent Q-learning (IQL), Nash-Q, and WoLF-PHC. Each configuration runs for 50,000 iterations across 20 seeds.

Table 16: Convergence to Nash equilibrium in Cournot and Bertrand duopoly

Game	Algorithm	Action	Profit	$ a - a^* $	Conv. iter
Cournot	IQL	2.95 ± 0.05	9.1 ± 0.0	0.05	1,000
	Nash-Q	2.89 ± 0.33	8.8 ± 1.3	0.17	1,000
	WoLF-PHC	3.00 ± 0.00	9.0 ± 0.0	0.00	1,000
Bertrand	IQL	4.00 ± 0.00	18.0 ± 0.0	0.33	1,000
	Nash-Q	4.00 ± 0.00	18.0 ± 0.0	0.33	1,000
	WoLF-PHC	3.95 ± 0.05	17.8 ± 0.2	0.38	1,000

Notes: Action and Profit report mean $\pm$ standard error across 20 seeds over the final 5,000 iterations. $|a - a^*|$ is the mean distance from Nash. Conv. iter is the first iteration where the smoothed average action enters a 0.5-neighborhood of Nash.

Table 16 and Figure 8 report the results. All three algorithms converge to the Nash equilibrium in both games. In Cournot, WoLF-PHC achieves exact convergence ($q = 3.00$), IQL reaches $q = 2.95$, and Nash-Q reaches $q = 2.89$ with higher variance across seeds. In Bertrand, all three converge to the discretized Nash price $p = 4$. Convergence is rapid, occurring within the first 5,000 iterations. The key observation is that even IQL, which lacks any game-theoretic computation, finds the Nash equilibrium in these well-structured games; Nash-Q’s explicit equilibrium computation does not provide faster convergence when the game has a unique, strict Nash equilibrium in pure strategies. The advantage of game-aware methods emerges in games with mixed-strategy equilibria, where IQL’s deterministic policy limit prevents convergence.

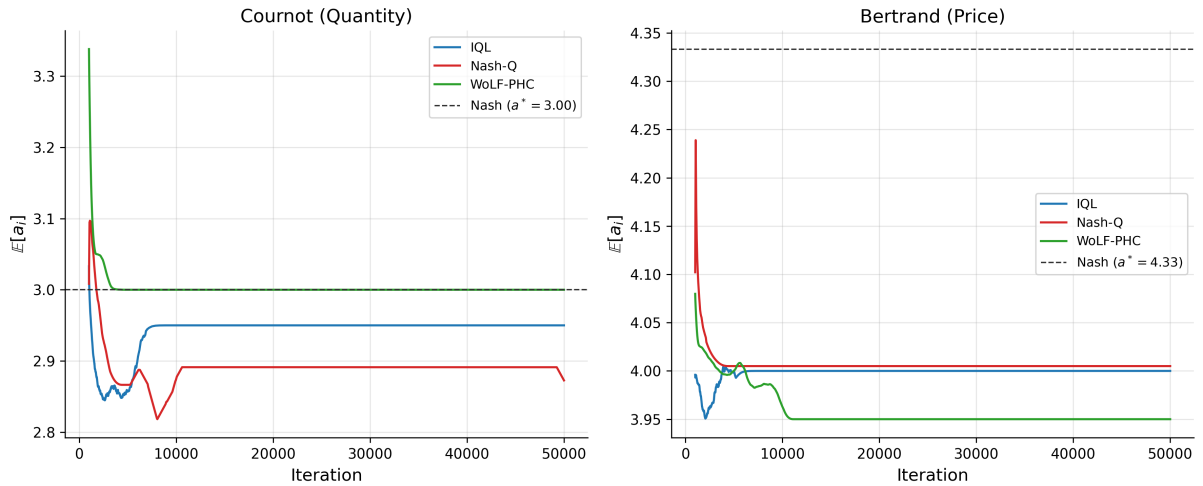


Figure 8: Convergence of expected actions to Nash equilibrium. Left: Cournot duopoly ($q^* = 3$). Right: Bertrand duopoly ($p^* \approx 4.33$, discretized to 4). Smoothed over 1,000-iteration windows, averaged across 20 seeds.

9.2 Counterfactual Regret Minimization

The algorithms above operate on stochastic games with simultaneous moves and observable payoffs. Extensive-form games, which model sequential decisions under imperfect information (bargaining, auctions, poker), require a different approach. CFR bypasses the equilibrium selection problem entirely: instead of computing Nash equilibria at each step, it minimizes cumulative regret and lets the time-averaged strategy converge to equilibrium.

An extensive-form game consists of a game tree with information sets $\mathcal{I}_i$ partitioning player i 's decision nodes. An information set groups nodes where the player cannot distinguish between them due to hidden opponent actions or private information. A behavioral strategy $\sigma_i : \mathcal{I}_i \rightarrow \Delta(\mathcal{A})$ assigns action probabilities at each information set.

The key concept in CFR is the *counterfactual value*, which measures how much better player i would have done by taking action a at information set I , assuming she played to reach I but her opponent played according to σ . Formally,

$$v_i^\sigma(I, a) = \sum_{h \in I} \pi_{-i}^\sigma(h) \sum_{z \supseteq ha} \pi^\sigma(ha, z) u_i(z) \quad (75)$$

where $\pi_{-i}^\sigma(h)$ is the probability opponents reach h , and $\pi^\sigma(ha, z)$ is the probability of reaching terminal z from ha . The counterfactual formulation weights by opponent reach $\pi_{-i}^\sigma(h)$ rather than joint reach $\pi^\sigma(h)$. This ensures non-zero updates even at rarely-visited information sets, avoiding the vanishing gradient problem that affects direct policy gradient methods in extensive-form games. Cumulative regret for action a is $R^T(I, a) = \sum_{t=1}^T [v^{\sigma^t}(I, a) - v^{\sigma^t}(I)]$. CFR updates via *regret matching*:¹¹⁵

$$\sigma^{T+1}(I, a) = \frac{[R^T(I, a)]^+}{\sum_{a'} [R^T(I, a')]^+}, \quad [x]^+ = \max(x, 0). \quad (76)$$

The average strategy $\bar{\sigma}^T$ converges to ε -Nash with $\varepsilon = O(|\mathcal{I}| \sqrt{|\mathcal{A}| \Delta} / \sqrt{T})$, where Δ is the range of payoffs (Zinkevich et al., 2008).¹¹⁶

CFR+ (Tammelin, 2014) replaces standard regret matching with Regret Matching+, which truncates cumulative regrets to zero after each update rather than only at action selection. The update becomes $R^{T+1}(I, a) = \max(R^T(I, a) + r^{T+1}(I, a), 0)$, where $r^{T+1}(I, a) = v^{\sigma^{T+1}}(I, a) - v^{\sigma^T}(I, a)$ is the instantaneous counterfactual regret. CFR+ also weights iteration t by t when computing the average strategy, giving later iterations, which reflect strategies closer to equilibrium, more influence. While vanilla CFR converges at $O(1/\sqrt{T})$, CFR+ empirically converges at $O(1/T)$, roughly squaring the number of useful digits of exploitability per unit of compute.¹¹⁷ CFR+ enabled Bowling et al. (2015) to essentially solve heads-up limit Texas hold'em, a game with 3.16×10^{17} states, the first non-trivial imperfect-information game played competitively by humans to be essentially solved. Their program Cepheus achieved exploitability below 1 mbb/g (milli-big-blind per game), small enough that a human lifetime of play would not suffice to distinguish it from a Nash equilibrium strategy.

9.3 Neural Extensions

Tabular CFR stores regrets at every information set, infeasible when $|\mathcal{I}| > 10^{14}$. Two neural approaches scale to large games.

¹¹⁵Regret matching is an action selection rule where the probability of choosing action a is proportional to the cumulative regret for not having chosen a in the past (truncated at zero). Actions with high regret receive higher probability; actions with negative regret (having performed worse than average) receive zero probability.

¹¹⁶An ε -Nash equilibrium is a strategy profile where no player can improve her expected payoff by more than ε through unilateral deviation. As $\varepsilon \rightarrow 0$, this converges to an exact Nash equilibrium.

¹¹⁷The $O(1/T)$ rate for CFR+ is empirically observed but not yet proven in full generality. Tammelin (2014) conjectured this rate based on extensive experiments across poker variants.

9.3.1 Deep CFR

Brown et al. (2019) approximate cumulative regrets with a neural network $V_\theta : \mathcal{I} \times \mathcal{A} \rightarrow \mathbb{R}$ trained on sampled (information set, iteration, regret) tuples:

$$L_V(\theta) = \mathbb{E}_{(I, t', r) \sim \mathcal{M}} [t' \cdot (V_\theta(I, a) - r)^2]. \quad (77)$$

Weighting by iteration index t' gives more recent regret estimates higher importance, since later iterations reflect strategies closer to equilibrium. A separate network Π_ϕ approximates the average strategy, trained via weighted MSE on strategy samples with the same iteration weighting. The current strategy derives from regret matching, with $\sigma(I, a) \propto [V_\theta(I, a)]^+$.

9.3.2 Neural Fictitious Self-Play

NFSP (Heinrich and Silver, 2016) combines *fictitious play*¹¹⁸ (Brown, 1951) with deep Q-learning.¹¹⁹ Each player maintains two networks: a best-response network Q_θ trained via DQN, and an average strategy network $\bar{\pi}_\phi$ trained via supervised learning on the best-response actions:

$$L_{\text{RL}}(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q_{\theta^-}(I', a') - Q_\theta(I, a) \right)^2 \right], \quad L_{\text{SL}}(\phi) = \mathbb{E} [-\log \bar{\pi}_\phi(a|I)]. \quad (78)$$

During play, agents follow $\bar{\pi}_\phi$ with probability $1 - \eta$ and Q_θ with probability η .

9.3.3 Poker Results

These methods achieved superhuman performance in poker. Deep CFR attained *exploitability* of 37 mbb/g (milli-big-blinds per game)¹²⁰ in heads-up flop hold'em (Brown et al., 2019). Libratus defeated top human professionals in heads-up no-limit hold'em using nested subgame solving with CFR (Brown and Sandholm, 2018). Pluribus extended to six-player no-limit hold'em (Brown and Sandholm, 2019). The game tree of heads-up no-limit hold'em contains $\sim 10^{161}$ states; these results demonstrate that CFR-based methods scale to economically relevant game sizes.

9.4 The Coase Conjecture

The durable goods monopoly is a canonical extensive-form bargaining problem with private information: the seller does not know the buyer's valuation. CFR solves it without any knowledge of the equilibrium structure, benchmarking against the closed-form predictions of Gul et al. (1986).

Coase (1972) conjectured that a durable goods monopolist¹²¹ loses market power when buyers are patient. Unable to commit to future prices, the seller competes with her future self, eroding rents. As the inter-offer interval shrinks ($\delta \rightarrow 1$), price collapses to marginal cost and

¹¹⁸Fictitious play (Brown, 1951) is a classical learning rule where each player best-responds to the empirical distribution of opponents' past actions. Under certain conditions (e.g., two-player zero-sum games), the time-average strategies converge to Nash equilibrium.

¹¹⁹Deep Q-Network (DQN) (Mnih et al., 2015) approximates the Q-function with a neural network, using experience replay and a target network to stabilize training. The target network θ^- in the loss function is a delayed copy of the main network, updated periodically.

¹²⁰Exploitability measures how far a strategy is from Nash equilibrium, defined as the maximum expected gain an adversary could achieve by best-responding. Zero exploitability means the strategy is unexploitable (Nash). In poker, exploitability is measured in milli-big-blinds per game (mbb/g).

¹²¹A durable good provides utility over multiple periods (e.g., a car, appliance, or software license) rather than being consumed immediately. Unlike non-durable goods, durable goods create intertemporal competition, as the seller at time t competes with her own future self at $t + 1$.

all surplus is extracted by buyers. Gul et al. (1986) formalized this for the gap case, where the seller’s cost is strictly below all buyer valuations, guaranteeing trade and a unique stationary equilibrium.¹²²

9.4.1 Model

A seller with zero cost faces a buyer with private valuation $v \in \{v_L, v_H\}$, where $\Pr(v = v_H) = \pi$. In each period $t = 1, 2, \dots$, the seller posts price p_t ; the buyer accepts or rejects. Upon acceptance at t , the seller receives $\delta^{t-1}p_t$ and the buyer receives $\delta^{t-1}(v - p_t)$, where $\delta \in (0, 1)$ is the common discount factor. The game ends upon acceptance or after T periods. Parameters: $v_L = 100$, $v_H = 200$, $T = 2$ periods.

9.4.2 Equilibrium Analysis

The screening price $P^*(\delta)$ makes the high-type buyer indifferent between accepting now and waiting.

$$v_H - P^* = \delta(v_H - v_L) \implies P^*(\delta) = v_H - \delta(v_H - v_L). \quad (79)$$

The seller’s optimal strategy depends on π . Let $\Pi_{\text{screen}} = \pi P^* + (1 - \pi)\delta v_L$ and $\Pi_{\text{pool}} = v_L$. The seller screens if $\Pi_{\text{screen}} > \Pi_{\text{pool}}$, yielding threshold

$$\pi^* = \frac{v_L(1 - \delta)}{P^* - \delta v_L} = \frac{1}{2} \quad \text{when } v_H = 2v_L. \quad (80)$$

For $\pi < \pi^*$, the seller pools (offers v_L immediately). For $\pi > \pi^*$, the seller screens (offers P^* , then v_L if rejected).¹²³ The Coase conjecture manifests as $\delta \rightarrow 1$, where $P^*(\delta) \rightarrow v_L$ and the seller cannot extract surplus from high types.

9.4.3 Computational Results

I model the bargaining game as an extensive-form game and apply CFR. The game tree has two information sets for the seller (indexed by rejection history) and two for the buyer (indexed by private type). CFR runs for 5,000 iterations per parameter configuration; NashConv (sum of exploitabilities) measures convergence.

Table 17 reports results from varying π at fixed $\delta = 0.5$. CFR recovers the phase transition at $\pi^* = 0.5$: below $\pi = 0.5$, the seller pools with probability approaching 1; above $\pi = 0.5$, the seller screens with probability approaching 1. The transition is sharp. A δ -sweep at $\pi = 0.7$ confirms the screening price formula $P^*(\delta) = 200 - 100\delta$ with zero error across $\delta \in [0.1, 0.9]$; at $\delta \approx 0.75$, the seller switches to pooling as patient buyers erode the screening premium, consistent with the Coase conjecture.¹²⁴ CFR recovered the phase transition, pricing formula, and Coase dynamics without any knowledge of the economic structure.

¹²²Stokey (1981) showed that a monopolist facing rational consumers cannot price discriminate intertemporally; Bulow (1982) proved that in the no-gap case, the monopolist prefers renting to selling. The no-gap case admits multiple equilibria with more complex dynamics.

¹²³In a screening equilibrium, the seller uses price to separate buyer types: high-value buyers accept immediately at a high price, while low-value buyers reject and receive a lower offer. In a pooling equilibrium, the seller offers a single price that all types accept. The gap case ($0 = c < v_L$) guarantees trade in equilibrium.

¹²⁴Four stress tests were conducted: (1) awkward primes $(v_L, v_H) = (37, 83)$, converging to $P^* = 55$ (theory: 55.4); (2) information leak with a single seller information set at the root; (3) grid shift with 150 removed, recovering 145 (99.4%); (4) 3-period game, finding $P_1 = 120$ versus theoretical 136. The 3-period result reflects a tie-breaking equilibrium: at $P_1 = 136$ the high buyer is exactly indifferent, so the seller prefers $P_1 = 120$ (revenue 108 versus 86.4 if rejected).

Table 17: CFR Equilibrium vs. Theory: π -sweep at $\delta = 0.5$

π	P^*	P(Screen)	Theory	NashConv	Eq. Type	Status
0.10	150	0.000	0.0	5.0258	Pooling	✓
0.15	150	0.000	0.0	7.5249	Pooling	✓
0.20	150	0.000	0.0	10.0239	Pooling	✓
0.25	150	0.000	0.0	12.5229	Pooling	✓
0.30	150	0.000	0.0	15.0223	Pooling	✓
0.35	150	0.000	0.0	17.5216	Pooling	✓
0.40	150	0.000	0.0	20.0213	Pooling	✓
0.45	150	0.000	0.0	22.5213	Pooling	✓
0.50	150	0.001	0.5	25.0221	Indifferent	✓
0.55	150	0.002	1.0	27.5256	Screening	✓
0.60	150	0.506	1.0	30.0066	Screening	✓
0.65	150	0.999	1.0	26.2553	Screening	✓
0.70	150	1.000	1.0	22.5027	Screening	✓
0.75	150	1.000	1.0	18.7531	Screening	✓
0.80	150	1.000	1.0	15.0067	Screening	✓
0.85	150	1.000	1.0	11.2576	Screening	✓
0.90	150	1.000	1.0	7.5089	Screening	✓

Notes: P(Screen) is the probability the seller offers $P^* = 150$ in period 1. Theory column gives the predicted probability (0 for pooling, 1 for screening). NashConv is the sum of exploitabilities. 5,000 CFR iterations per configuration.

9.5 Discussion

This chapter presented two computational approaches to Nash equilibrium in games. Stochastic-game Q-learning replaces the Bellman max with a game-theoretic operator; CFR accumulates counterfactual regrets and lets the time-averaged strategy converge. The Cournot and Bertrand simulations (Section 9.1.5) show convergence in simultaneous-move games; the Coase simulation shows convergence in an extensive-form bargaining game with private information. The two mechanisms, Bellman backup versus regret minimization, reflect a genuine divide in the field corresponding to the game classes each approach targets.

10 Real-Time Reinforcement Learning and Dynamic Pricing

In this chapter we focus on “online” or “real-time” reinforcement learning where an agent learns “live” or “in production” and not in a “offline” or simulation environment. This setting makes *exploration* a real cost, and the agent is forced to balance exploration vs *exploitation*. This setting is particularly relevant for economic applications, where the agent is typically a firm learning about demand and the cost of exploration is lost revenue. I focus on the problem of dynamic pricing, where a seller learns to set prices for products with unknown demand. This problem has been studied extensively in the economics and machine learning literatures, and it provides a rich testbed for understanding how structural assumptions about demand affect algorithmic learning.

Rothschild (1974) posed pricing under demand uncertainty as a two-armed bandit problem. His insight was that a *myopic* seller can get stuck at a suboptimal price forever, because exploiting the currently best-looking price generates no information about alternatives. This chapter focuses on papers that develop these types of explore-exploit models and tries to quantify the value of structural modeling in online dynamic pricing.

A seller faces T customers in sequence, setting a price for each and observing only whether

the customer bought. The seller does not observe the customer’s willingness to pay. *Regret* measures the total revenue gap between the seller’s pricing policy and the best fixed price in hindsight. If the optimal fixed price earns revenue r^* per customer, a policy with regret $R(T)$ earns approximately $Tr^* - R(T)$ in total. The central question in dynamic pricing is how fast $R(T)$ can shrink, relative to T , as the seller accumulates purchase data.

This chapter surveys the theoretical and empirical literature on bandit-based dynamic pricing. I begin with foundational results that establish the baseline regret rates under progressively stronger assumptions on demand. I then examine more recent research, each illustrating a distinct economic insight. A comparison table collects these results and I conclude with a simulation study replicating the Misra et al. (2019) pricing experiment.

10.1 Foundations

10.1.1 No Structure on Demand

Kleinberg and Leighton (2003) study the simplest version of the problem. Customers arrive with valuations drawn i.i.d. from an unknown distribution on $[0, 1]$, and the seller posts a price from a continuous set. The demand curve $D(p) = \Pr(v \geq p)$ is unknown; the only feedback is whether each customer bought. The seller’s goal is to minimize regret against the single price p^* that maximizes $p \cdot D(p)$. Kleinberg and Leighton prove that the minimax regret is $\Theta(\sqrt{T})$.¹²⁵ In concrete terms, after 10,000 customers the seller loses roughly 100 customers’ worth of revenue to the uncertainty in demand. No algorithm can do better without imposing structure on D . For adversarial valuations (chosen by a worst-case opponent rather than drawn from a fixed distribution), the minimax regret rises to $\Theta(T^{2/3})$, achieved by the Exp3 algorithm on a discretized price grid. I focus on the stochastic setting throughout this chapter.

10.1.2 Parametric Demand

Broder and Rusmevichientong (2012) consider a parametric demand model $d(p; z) = \Pr(V \geq p)$, where $z \in \mathbb{R}^n$ is an unknown parameter governing the demand curve. The seller observes binary purchase decisions and updates a maximum likelihood estimate of z . Under standard regularity conditions (bounded demand, unique optimal price, smooth revenue function), the minimax regret is again $\Theta(\sqrt{T})$.¹²⁶ Parametric structure alone does not break the $\sqrt{T}$ barrier.

The picture changes under a “well-separated” condition requiring that every price in the feasible set is informative about the demand parameter. Formally, the Fisher information $I(p, z)$ is bounded below by $c_f > 0$ for all prices p and parameters z .¹²⁷ Under well-separation, a pure greedy policy (MLE-Greedy, which simply prices at $p^*(\hat{z}_t)$ in each period) achieves $O(\log T)$ regret (Theorem 4.8). After 10,000 customers, the revenue loss is only about 9 customers’ worth. When every price teaches the seller something, dedicated exploration rounds become unnecessary.

¹²⁵The upper bound, $O(\sqrt{T \log T})$, discretizes $[0, 1]$ into $K = \lceil (T / \log T)^{1/4} \rceil$ prices and runs the UCB1 algorithm of Auer et al. (2002). The lower bound, $\Omega(\sqrt{T})$, constructs a family of demand curves parameterized by the location of the optimal price $p^* \in [0.3, 0.4]$. The key tension: posting prices far from p^* is informative about demand but costly in revenue; posting prices near p^* is cheap but uninformative. Resolving this tension costs at least $\Omega(\sqrt{T})$ in cumulative revenue.

¹²⁶The lower bound (Theorem 3.1 of Broder and Rusmevichientong (2012)) constructs a linear demand family where all demand curves pass through the same point at the optimal price $p^*(z_0)$. Observing purchases at this price provides no information about z . An MLE-Cycle policy that interleaves dedicated exploration rounds with greedy pricing achieves the matching upper bound $O(\sqrt{T})$ (Theorem 3.6).

¹²⁷This means no two demand curves $d(p; z)$ and $d(p; z')$ cross on the pricing interval, so every purchase observation distinguishes the two hypotheses. In the lower-bound family of Theorem 3.1, the curves all cross at $p = 1$, which is why the $\sqrt{T}$ floor emerges.

10.1.3 High-Dimensional Features with Sparsity

Javanmard and Nazerzadeh (2019) extend the setting to products described by d -dimensional feature vectors x_t . The market value is $v_t = x_t^\top \theta_0 + z_t$, where $\theta_0 \in \mathbb{R}^d$ is unknown and z_t is i.i.d. noise from a known log-concave distribution F .¹²⁸ Only s_0 of the d coordinates of θ_0 are nonzero, but the seller does not know which ones.

Their RMLP algorithm (Regularized Maximum Likelihood Pricing) operates in episodes whose lengths double (1, 2, 4, 8, ...). At each episode boundary, the seller fits a LASSO-penalized maximum likelihood estimate of θ_0 using data from the previous episode, then prices greedily throughout the current episode. The regret is $O(s_0 \log d \cdot \log T)$ (Theorem 4.1), with a matching lower bound of $\Omega(s_0(\log d + \log T))$ (Theorem 5.1). The reason this beats $\sqrt{T}$ connects back to the Broder lower bound. In the parametric model of Section 10.1.2, all demand curves can cross at the optimal price, making that price uninformative. Here, customer features vary across periods, so the aggregate demand function at any fixed price changes in proportion to the estimation error in θ_0 . Every price is informative, and dedicated exploration rounds become unnecessary.¹²⁹ Even with hundreds of features, if only a handful matter, learning is fast. These results along with Kleinberg and Leighton (2003) and Broder and Rusmevichientong (2012) establish the baseline rates for how structural assumptions affect learning speed in dynamic pricing.

10.2 Revealed Preference and Partial Identification

Misra et al. (2019) bring economic theory into the bandit pricing framework. Their model has K discrete prices, S consumer segments (segment membership observed, but valuations unknown), and within-segment heterogeneity δ . A consumer in segment s has valuation $v_i = v_s + n_i$, where v_s is the segment midpoint and $n_i \in [-\delta, \delta]$ is idiosyncratic noise. The key structural assumption is WARP (Weak Axiom of Revealed Preference): if a consumer buys at price p , she would buy at any lower price $p' < p$. This is equivalent to downward-sloping demand.

WARP enables partial identification of each segment's valuation. Suppose the seller has offered several prices to segment s . Define $p_s^{\min} = \max\{p_k : \text{all customers purchased}\}$ and $p_s^{\max} = \min\{p_k : \text{no customer purchased}\}$. Then the segment midpoint lies in $[p_s^{\min}, p_s^{\max}]$, and within-segment heterogeneity satisfies $\hat{\delta}_s \leq (p_s^{\max} - p_s^{\min})/2$. These bounds propagate to aggregate demand: for each price p_k , the seller constructs upper and lower bounds on total profit $\pi(p_k) = p_k \cdot D(p_k)$. When the profit upper bound at some price falls below the best profit lower bound across all prices, that price is dominated and permanently eliminated from consideration.

The UCB-PI algorithm combines dominance elimination with a price-scaled exploration bonus:

$$I_{p_k}(t) = \hat{\mu}_{p_k}(t) + p_k \sqrt{\frac{2 \ln t}{N_{p_k}(t)}} \quad (81)$$

if p_k is not dominated, and $I_{p_k}(t) = 0$ otherwise, where $\hat{\mu}_{p_k}(t)$ is the average profit observed at price p_k and $N_{p_k}(t)$ is the number of trials.¹³⁰ Two innovations drive the improvement over standard UCB1. First, dominance elimination reduces the effective number of arms the

¹²⁸Log-concavity of F and $1 - F$ is satisfied by the normal, logistic, uniform, Laplace, and exponential distributions. It ensures that expected revenue $p \cdot [1 - F(p - x_t^\top \theta_0)]$ is strictly quasi-concave in p , giving a unique optimal price.

¹²⁹If some feature directions are rarely observed, the seller cannot learn all coordinates of θ_0 quickly, and regret degrades to $O(\sqrt{\log(d) \cdot T})$ (Theorem 4.2). If the noise distribution belongs to a known parametric family but its scale parameter is unknown, regret reverts to $\Omega(\sqrt{T})$ (Theorem 7.1), foreshadowing the result of Xu and Wang (2021) discussed in Section 10.3.

¹³⁰Standard UCB1 uses an exploration bonus of $\sqrt{2 \ln t / N_{p_k}(t)}$, which assumes rewards in $[0, 1]$. Since profit at price p_k is bounded by p_k , scaling the bonus by p_k tightens exploration for cheap prices that cannot contribute much profit regardless.

algorithm must explore. Second, the p_k scaling focuses exploration on prices where uncertainty actually matters for profit. Together, these yield $O(\log T)$ regret.

$$\mathbb{E}[R_T(\text{UCB-PI})] \leq \sum_{k \neq k^*} \frac{8p_k \log T}{\Delta_k} + \left(1 + \frac{\pi^2}{3}\right) \sum_{k=1}^K \Delta_k \quad (82)$$

where $\Delta_k = \mu_{k^*} - \mu_k$ is the gap between arm k and the optimal arm.¹³¹

Misra et al. (2019) calibrate the model to a field experiment at ZipRecruiter, a B2B online recruiting platform. With 7,870 customers per month, 1,000 segments, and 10 price points from \$19 to \$399, UCB-PI achieves 98% of oracle profit and produces 43% higher profits during the first month of testing compared to a learn-then-earn alternative. The algorithm has both higher mean profit and lower variance, reflecting the value of eliminating dominated prices early.¹³²

10.3 The Value of Knowing the Noise Distribution

Xu and Wang (2021) ask how much it helps to know the shape of demand uncertainty. In their model, a feature vector $x_t \in \mathbb{R}^d$ describes each sales session, the customer's valuation is $w_t = x_t^\top \theta^* + N_t$ where θ^* is unknown and N_t is zero-mean i.i.d. noise with CDF F , and the seller observes only whether the customer bought at the posted price. The answer depends on whether F is known.¹³³

If F is known (the seller knows that demand shocks are, say, normally distributed with known variance), the EMLP algorithm (Epoch-based Maximum Likelihood Pricing) achieves regret $O(d \log T)$ (Theorem 3).¹³⁴ After 10,000 customers with $d = 5$ features, the revenue loss is roughly 50 customers' worth, an improvement over the $\sqrt{T} \approx 100$ customers' worth that Kleinberg's lower bound imposes without structural knowledge.

If F is unknown (even if only the variance of a Gaussian is unknown, with everything else known), the regret is at least $\Omega(\sqrt{T})$ (Theorem 12).¹³⁵ The seller is back to the Kleinberg baseline, with no algorithm able to achieve sublinear improvement regardless of how many features are available.

The gap between $O(d \log T)$ and $\Omega(\sqrt{T})$ is super-polynomial in T , not merely a constant factor. This separation quantifies, in regret units, the value of structural assumptions that econometricians routinely make. Knowing F is the pricing analogue of specifying the structural model; not knowing F is a more reduced-form agnostic position. When an economist specifies a logit or probit demand model, the assumed noise distribution is not merely a convenience for estimation; it purchases a qualitative improvement in the rate at which the algorithm

¹³¹The first sum is the leading term, scaling as $O(\log T)$. The second sum is a constant that does not grow with T . Replacing p_k with 1 recovers the standard UCB1 bound, which is looser.

¹³²For multi-product settings, Mueller et al. (2019) impose low-rank structure on the price-sensitivity matrix, achieving regret $O(T^{3/4} \sqrt{d})$ that scales with the latent demand dimension d rather than the number of products. Badanidiyuru et al. (2013) extend the bandit framework to handle inventory constraints ("bandits with knapsacks"), relevant when the seller faces limited stock alongside the pricing decision.

¹³³The regret benchmark here differs from Section 10.1.1. Kleinberg and Leighton (2003) and Broder and Rusmevichientong (2012) measure regret against the best fixed price; Xu and Wang (2021) and Liu et al. (2024) measure regret against the clairvoyant contextual policy that sets the optimal price p_t^* for each customer's features x_t . The contextual benchmark is harder, which makes the $O(d \log T)$ rate more impressive.

¹³⁴EMLP runs in doubling epochs of length $\tau_k = 2^{k-1}$. At each epoch boundary, the seller fits a maximum likelihood estimate $\hat{\theta}_k$ using data from the previous epoch, then prices greedily at $p_t = J(x_t^\top \hat{\theta}_k)$ throughout the epoch, where $J(u) = \arg \max_v v[1 - F(v - u)]$ is the revenue-maximizing price function. The key technical insight is that the negative log-likelihood is strongly convex (Lemma 7 of Xu and Wang (2021)), so MLE concentrates at rate $O(d/\tau_k)$. Since regret is quadratic in the parameter estimation error (Lemma 5) and there are $O(\log T)$ epochs, the total regret is $O(d \log T)$.

¹³⁵The lower bound constructs two noise variances $\sigma_1 = 1$ and $\sigma_2 = 1 - T^{-1/4}$. Any algorithm that performs well under both must spend $\Omega(\sqrt{T})$ revenue distinguishing the two cases. This extends the "uninformative price" phenomenon of Broder and Rusmevichientong (2012): when the seller does not know F , there exist prices at which observed purchase behavior is nearly identical under different demand parameters.

learns. Conversely, semiparametric approaches that leave the error distribution unspecified pay a concrete cost in learning speed, reverting from logarithmic to polynomial regret.¹³⁶

10.4 Strategic Buyers

Liu et al. (2024) introduce buyer manipulation into contextual dynamic pricing. At time t , a buyer arrives with true covariates $x_t^0 \in \mathbb{R}^d$ and valuation $v_t = \theta_0^\top x_t^0 + z_t$. The seller announces a pricing rule $p_t = g(\hat{\theta}_k^\top x_t)$, where $\hat{\theta}_k$ is the current parameter estimate. Crucially, the buyer observes this rule and can distort her reported features. She solves a cost-minimization problem:

$$\min_{\tilde{x}} (p - v_t) + \frac{1}{2}(\tilde{x} - x_t^0)^\top A(\tilde{x} - x_t^0) \quad (83)$$

where A is a positive definite matrix governing manipulation costs.¹³⁷ The first-order condition yields a systematic bias: the buyer shifts her features to make the seller’s model predict a lower valuation, securing a lower price. The seller observes only the distorted features $\tilde{x}_t$, not the true x_t^0 .

Theorem 5 (Theorem 1 of Liu et al. (2024)). *Under standard regularity conditions, any pricing policy that treats reported features as truthful accumulates regret $\Omega(T)$.*

This is not $\sqrt{T}$ or $\log T$; it is linear in T . The revenue gap grows without bound and never stabilizes. Every standard dynamic pricing algorithm, including EMLP and RMLP, falls into this category because none accounts for strategic manipulation. The non-strategic policy is systematically biased. It underprices because it bases decisions on manipulated features, and the bias does not shrink with more data because the manipulation is endogenous to the pricing rule.

The fix is to jointly estimate demand parameters and manipulation behavior. Liu et al. (2024) propose an episodic algorithm with two phases per episode. During the exploration phase, the seller posts uniform random prices that do not depend on features. Since the price is independent of $\tilde{x}_t$, buyers have no incentive to manipulate, and the seller observes true features x_t^0 . During the exploitation phase, the seller uses the corrected pricing rule that anticipates the manipulation:

$$p_t = g\left(\hat{\theta}_k^\top x_t + \hat{\beta}_k^\top A^{-1}\hat{\beta}_k \cdot g'(\hat{\theta}_k^\top x_t)\right) \quad (84)$$

where $\hat{\beta}_k$ is the estimated coefficient on the manipulable features and g is the optimal pricing function. This correction adds a markup that offsets the anticipated feature distortion.

Theorem 6 (Theorem 2 of Liu et al. (2024)). *With known manipulation cost matrix A , the strategic pricing algorithm achieves regret $O(d\sqrt{T})$.*

When A is unknown, the seller can still recover $O(d\sqrt{T/\tau})$ regret by tracking repeat buyers across exploration and exploitation phases, where τ is the fraction of buyers who appear in both phases (Theorem 3). Higher repeat rates mean more matched pairs for estimating manipulation behavior.

Ignoring strategic behavior in online pricing is quite suboptimal. Incentive compatibility matters even in settings where the seller is “just” learning demand.¹³⁸

¹³⁶Tullii et al. (2024) establish the tightest known bound under minimal distributional assumptions: if the noise distribution (c.d.f.) is merely Lipschitz continuous, the minimax regret is $\Theta(T^{2/3})$, strictly between the $\log T$ rate with known F and the $\sqrt{T}$ rate with unknown F . Fan et al. (2024) consider a semiparametric setting where the noise density is smooth and connect the pricing problem to the econometrics of semiparametric estimation.

¹³⁷The matrix A captures how costly it is for the buyer to distort each feature dimension. High eigenvalues of A mean manipulation is expensive. This is the standard model of strategic classification (Hardt et al., 2016), adapted to pricing.

¹³⁸Agrawal and Tang (2024) document a related phenomenon in pricing with reference effects. If consumers

10.5 Comparison of Regret Rates

Table 18 collects the regret rates across the papers discussed above. The table is ordered from weakest to strongest assumptions on demand, making visible three patterns. First, stronger assumptions yield faster learning, from $\sqrt{T}$ with no structure to $\log T$ with parametric or revealed-preference structure to $d \log T$ with known noise in high dimensions. Second, not knowing the noise distribution is expensive; moving from known to unknown F costs a super-polynomial factor in the regret rate. Third, strategic behavior destroys standard algorithms, producing linear regret that no amount of data can overcome without an explicit correction. Figure 9 plots each rate as a function of T , making the two-orders-of-magnitude gap between the strongest and weakest assumptions visible on a log-log scale.

Table 18: Regret rates in dynamic pricing under progressively stronger assumptions. T is the number of customers, d the feature dimension, s_0 the sparsity level. The last column translates asymptotic rates into concrete terms for $T = 10,000$ with $d = 5$, setting constants to 1.

Paper	Demand	Noise	Regret	Per 10K ($d=5$)
Kleinberg and Leighton (2003)	none	none	$\sqrt{T}$	~ 100 lost
Broder and Rusmevichientong (2012)	parametric	known family	$\sqrt{T}$	~ 100 lost
Broder and Rusmevichientong (2012)	param., well-sep.	known family	$\log T$	~ 9 lost
Javanmard and Nazerzadeh (2019)	linear, s_0 -sparse	known, log-conc.	$s_0 \log d \log T$	fast if s_0 small
Xu and Wang (2021)	linear, contextual	known	$d \log T$	~ 46 lost
Xu and Wang (2021)	linear, contextual	unknown var.	$\geq \sqrt{T}$	~ 100 lost
Tullii et al. (2024)	linear, contextual	Lipschitz only	$T^{2/3}$	~ 464 lost
Misra et al. (2019)	WARP	–	$\log T$	~ 9 lost
Liu et al. (2024)	linear + strategic	known, naïve	T	never improves
Liu et al. (2024)	linear + strategic	known, corrected	$d\sqrt{T}$	~ 500 lost

10.6 Applications

10.6.1 Joint Assortment and Pricing at Scale

Cai et al. (2023) tackle the joint assortment-pricing problem, where a retailer must simultaneously choose which products to display and at what prices. With a large catalog and limited shelf space, the number of possible assortments is combinatorially vast; for a Chinese instant noodle producer with 176 products and 30 display slots, there are $\binom{176}{30} \approx 6.4 \times 10^{33}$ possible assortments before prices are even set. Customer demand also depends on market context such as region and season. In each period t , the retailer selects an assortment-pricing vector $a_t \in \mathbb{R}^{d_a}$ (encoding which products to display and at what prices), observes a context vector $x_t \in \mathbb{R}^{d_x}$ (encoding customer demographics and market conditions), and earns revenue Y_t . Cai et al. model expected revenue as $\mathbb{E}[Y_t | x_t, a_t] = a_t^\top \Theta^* x_t$, where $\Theta^* \in \mathbb{R}^{d_a \times d_x}$ is an unknown matrix assumed to have low rank $r \ll \min\{d_a, d_x\}$. The low-rank assumption is the demand-side analogue of factor models in asset pricing; a small number of latent dimensions (flavor preferences, seasonal effects, price sensitivity) explain most of the variation in purchasing behavior.

Their Hi-CCAB algorithm estimates Θ^* via penalized least squares, where the penalty is the nuclear norm (the sum of singular values) of Θ . This penalty encourages low-rank solutions, playing the same role for matrices that the LASSO penalty plays for sparse vectors. The time-averaged regret is $\tilde{O}(T^{-1/6})$ with dimension dependence $r(d_a + d_x)$ rather than $d_a \cdot d_x$, so the

anchor on past prices, a static pricing policy that ignores reference dependence accumulates linear regret $\Omega(T)$. Chen et al. (2025) show that imposing fairness constraints (requiring similar prices for similar customers) raises the regret floor to $\Theta(T^{2/3})$, a social cost of equitable treatment.

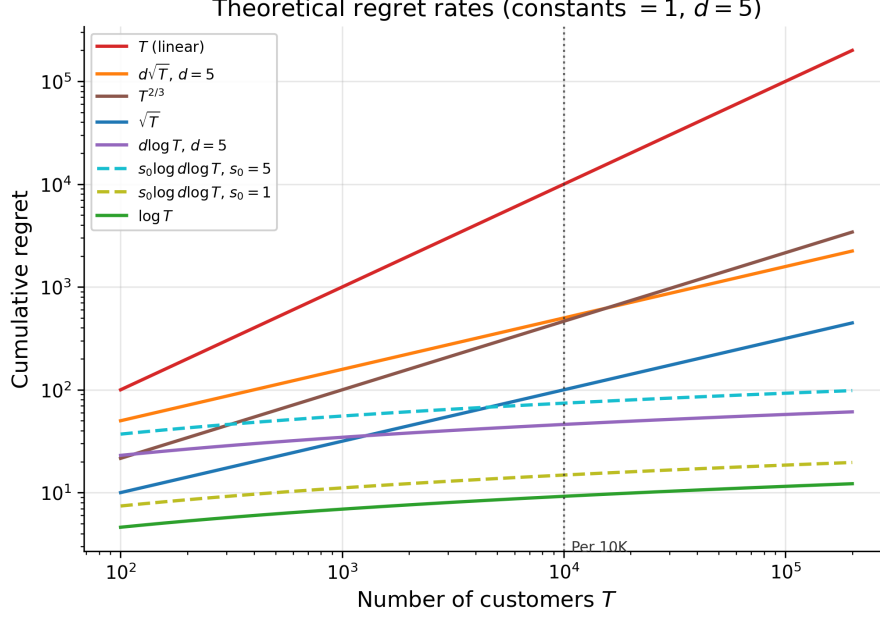


Figure 9: Theoretical regret rate functions at constants equal to 1, $d = 5$. Two cases for the $s_0 \log d \log T$ rate are shown: $s_0 = 1$ (very sparse, ≈ 15 lost at $T = 10,000$) and $s_0 = 5$ (moderate sparsity, ≈ 74 lost). The vertical line marks $T = 10,000$, matching the “Per 10K” column of Table 18.

effective parameter count scales with the number of latent factors, not the full product-by-context matrix. In simulation, Hi-CCAB achieves nearly four times the cumulative sales of the noodle producer’s historical assortment strategies, averaged over 100 replications.

Ganti et al. (2018) deploy Thompson Sampling for dynamic pricing at Walmart.com. Their MAX-REV-TS algorithm models demand for each item i on day t via a constant-elasticity function $d_{i,t}(p) = f_{i,t}(p/p_{i,t-1})^{\gamma_i^*}$, where $d_{i,t}(p)$ is unit sales at price p , $f_{i,t}$ is a baseline demand forecast at the previous day’s price $p_{i,t-1}$, and $\gamma_i^* < -1$ is the unknown price elasticity. The structural assumption, that demand responds to price through a single elasticity parameter per item, reduces the learning problem from estimating a full demand curve to estimating one scalar per item. MAX-REV-TS places a Gaussian prior over the elasticity vector γ^* and draws posterior samples at each period to solve a constrained revenue maximization problem. In a five-week field experiment on a basket of roughly 5,000 items, Thompson Sampling produced a statistically significant increase in per-item revenue relative to the passive pricing baseline.

10.7 Simulation Study: The Knowledge Ladder

I run six algorithms on the Misra et al. (2019) demand environment to trace how cumulative regret responds to increasing structural knowledge. The environment has $K = 100$ prices on a grid from \$0.01 to \$1.00, $S = 1,000$ consumer segments with equal weights, within-segment heterogeneity $\delta = 0.1$, and segment midpoints $v_s \sim \text{Uniform}(0.1, 0.9)$. A consumer purchases if and only if $v_i \geq p$ (WARP). The six algorithms, ordered by the structural knowledge they exploit, are: (0) ϵ -greedy with $\epsilon = 0.1$, which never adapts its exploration rate; (1) Learn-Then-Earn (LTE), which explores uniformly for the first 5% of rounds and then commits to the empirical best price; (2) UCB1 (Auer et al., 2002), which adapts exploration via confidence bounds but ignores demand structure; (3) Thompson Sampling (Thompson, 1933), which maintains a Bayesian posterior over purchase rates; (4) UCB-PI (Misra et al., 2019), which uses WARP to eliminate dominated prices and scales the exploration bonus by the price level; and (5) UCB-PI-tuned, which adds a variance-based refinement to the exploration bonus. Each algorithm

runs across 10 seeds with $T = 200,000$ rounds.

Table 19 and Figure 10 report cumulative regret at four checkpoints. The results confirm the theoretical progression from Table 18. ε -greedy regret grows linearly (R/T stabilizes near 0.011). LTE pays its exploration cost upfront, with regret of 1,005 at $T = 10,000$ (entirely during the random phase), but flattens as the committed price earns near-optimal profit; $R/T^{2/3}$ stabilizes near 0.5. UCB1 and Thompson Sampling both grow as $\sqrt{T}$, with Thompson achieving a five-fold better constant (2.5 vs. 15.1 in $R/\sqrt{T}$ at $T = 200,000$). UCB-PI-tuned achieves the lowest regret at every checkpoint past $T = 10,000$, reaching 780 at $T = 200,000$, a 31% reduction over Thompson Sampling and a 83% reduction over UCB-PI without variance tuning. The untuned UCB-PI variant overexplores; its regret of 4,503 scales as $\sqrt{T}$ rather than the $\log T$ that the theoretical bound predicts with correct tuning, placing it closer to UCB1 (6,734) than to the tuned variant (780).

Table 19: Cumulative regret for six pricing algorithms on the Misra et al. (2019) demand environment ($K = 100$, $S = 1,000$, 10 seeds).

Level	Algorithm	$T=10K$	$T=50K$	$T=100K$	$T=200K$	Rate
0	ε -greedy	160	624	1,158	2,263	$\Theta(T)$
1	LTE (5%)	1,005	1,165	1,363	1,771	$O(T^{2/3})$
2	UCB1	719	2,609	4,263	6,734	$O(\sqrt{KT})$
3	Thompson	290	628	844	1,136	$O(\sqrt{KT})$
4	UCB-PI	789	2,266	3,244	4,503	$O(\log T)$
5	UCB-PI-tuned	338	541	640	780	$O(\log T)$

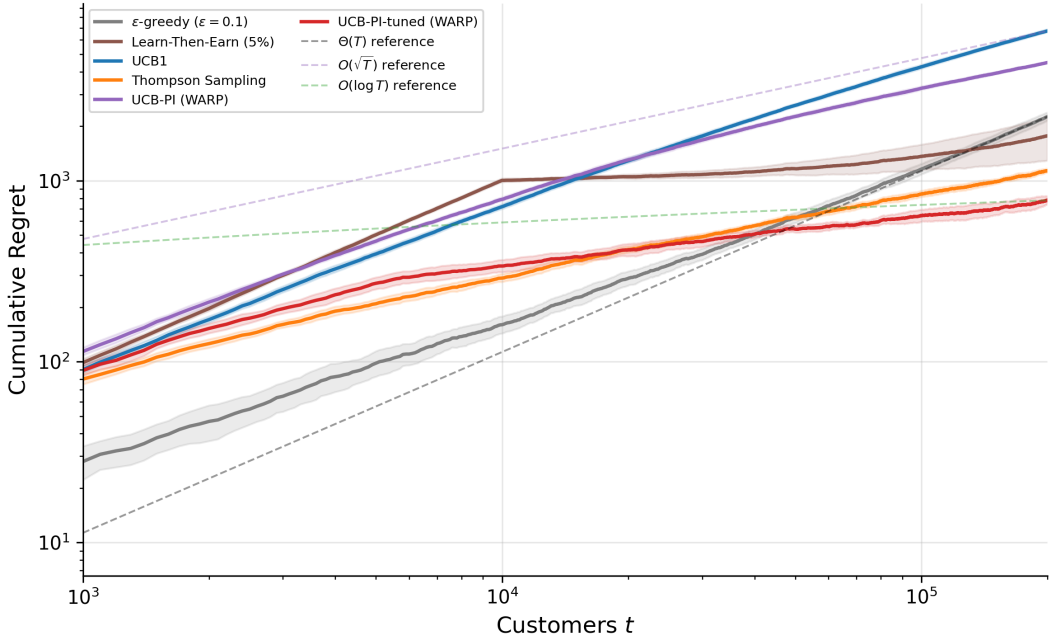


Figure 10: Cumulative regret on log-log axes. Dashed lines show $\Theta(T)$, $O(\sqrt{T})$, and $O(\log T)$ reference rates. Shaded regions are ± 2 standard errors across 10 seeds.

11 Reinforcement Learning from Human Feedback

Every chapter so far assumes access to a scalar reward signal. Dynamic programming (Chapter 3) requires the full reward function $r(s, a)$. Model-free RL (Chapter 2) relaxes the model requirement but still observes a numerical reward r_t after each transition. Q-learning, temporal difference methods, policy gradient, all update toward a scalar signal. The Bellman optimality equation $V^*(s) = \max_a \{r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s')\}$ presupposes that $r(s, a)$ is either known or observable. When it is neither, the entire DP/RL machinery cannot be applied directly.

In many domains, scalar rewards are unavailable but ordinal preferences over trajectories are easy to elicit. Text generation is the canonical example: a human evaluator cannot assign a meaningful numerical score to a paragraph of text, but can reliably say “response A is better than response B” when shown two candidate completions. Similarly, in robotics, a human can compare two demonstrated behaviors without specifying a cost function. The raw data in RLHF is a set of trajectory pairs (or trajectory segment pairs) with binary preference labels indicating which trajectory the evaluator preferred. RLHF uses these ordinal comparisons to learn a proxy reward function r_θ , which then serves as the scalar signal for standard RL optimization. This is not an inverse problem in the IRL sense; the goal is not to rationalize observed behavior or recover the agent’s own reward, but rather a two-stage forward problem in which the analyst first learns a reward from human judgments and then solves the resulting MDP using the RL tools from earlier chapters.

When no one can write down the reward function, human preferences provide an alternative training signal. Reinforcement Learning from Human Feedback (RLHF) learns a reward function by eliciting pairwise preference comparisons from a human evaluator over machine-generated outputs, rather than inverting an observed policy as in structural estimation or IRL. Christiano et al. (2017) demonstrated that this approach could train agents without an explicit reward function. RLHF has since become the predominant method for aligning large language models.

11.1 Learning Rewards from Preferences

The canonical RLHF framework is built on a formal model of human preference. The observed data consists of tuples (s, y_w, y_l) , where s is a context, and y_w and y_l are two outputs, with y_w being the “winner” preferred by a human. Assuming preferences follow a latent utility model, the Bradley-Terry model (Bradley and Terry, 1952) gives the probability that y_w is preferred: $P(y_w \succ y_l | s) = \sigma(r_\theta(s, y_w) - r_\theta(s, y_l))$, where $r_\theta : \mathcal{S} \times \mathcal{Y} \rightarrow \mathbb{R}$ is a learned *reward model* parameterized by θ , trained to approximate human preferences (not the ground-truth reward, which is unobserved), and $\sigma(\cdot)$ is the logistic function. This formulation is a direct application of discrete choice theory and is equivalent to a binary logit model, a special case of the multinomial logit framework (McFadden, 1974b) that underpins DDC estimation. The reward model parameters θ are estimated by minimizing the negative log-likelihood of the observed human choices:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\theta(s, y_w) - r_\theta(s, y_l))], \quad (85)$$

In the LLM setting, the “outputs” y_w and y_l are token sequences, i.e., trajectories of the autoregressive policy, so preferences are over trajectories rather than single actions. The preference loss in Equation (85) is MLE of a choice model where the “alternatives” are trajectory segments and the “choice” is the human-preferred one.

11.2 The RLHF Pipeline and Direct Optimization

The learned r_θ then serves as a proxy objective for policy optimization. Building on the reward-learning and RL fine-tuning framework of Ziegler et al. (2019) and Stiennon et al. (2020), the canonical three-stage pipeline was formalized by Ouyang et al. (2022). First, a base pretrained model is initialized via supervised fine-tuning (SFT) on a small dataset of high-quality demonstrations, yielding an initial policy π^{SFT} . This grounds the model in the desired style and format. The second step is the reward model training as described, using preference data generated from this π^{SFT} .

In the third and final step, the policy is optimized using reinforcement learning. The π^{SFT} policy, now denoted π_ϕ , is fine-tuned to maximize the expected reward signal from the frozen reward model r_θ . In practice, this optimization uses PPO (Chapter 2, Section 5), the same clipped policy gradient method that stabilizes updates via a trust region. The standard policy gradient machinery from Chapter 2 applies directly, but with one complication. The reward function is itself an estimate, introducing approximation error on top of the usual variance in gradient estimates. A critical component of this optimization is the inclusion of a regularization term that penalizes the Kullback-Leibler (KL) divergence between the learned policy π_ϕ and the initial reference policy π^{SFT} . The objective function for this final stage is

$$J(\phi) = \mathbb{E}_{s \sim \mathcal{D}, y \sim \pi_\phi(\cdot|s)}[r_\theta(s, y)] - \lambda_{KL} \mathbb{E}_{s \sim \mathcal{D}}[D_{KL}(\pi_\phi(\cdot|s) \parallel \pi^{SFT}(\cdot|s))], \quad (86)$$

where D_{KL} is the Kullback-Leibler divergence and λ_{KL} is a hyperparameter controlling the penalty strength.¹³⁹ This KL-divergence term is essential because it prevents the policy π_ϕ from deviating too far from the distribution of text on which the reward model is reliable, thus mitigating “reward hacking” where the policy exploits inaccuracies in r_θ to achieve a high score with degenerate behavior. The KL regularization serves a function analogous to the stabilization techniques discussed in Chapter 3’s treatment of the deadly triad: it constrains the policy to regions where the learned reward model r_θ is reliable, preventing the RLHF analogue of divergence under function approximation.

The KL-regularized objective in Equation (86) admits a Bayesian interpretation (Korbak et al., 2022). In this view, the reference policy π^{SFT} acts as a prior distribution over plausible responses. The reward model r_θ provides evidence, specifying which responses are more desirable. The goal of alignment is to find the posterior distribution π^* that optimally combines the prior with this evidence. This ideal posterior policy takes the form of Equation (87):

$$\pi^*(y|s) \propto \pi^{SFT}(y|s) \exp\left(\frac{r_\theta(s, y)}{\lambda_{KL}}\right). \quad (87)$$

This formulation directly parallels Bayes’ rule, where the posterior is proportional to the prior times the likelihood. The reward function, scaled by λ_{KL} , defines the log-likelihood of the preference evidence. From this perspective, the KL-regularized RL objective $J(\phi)$ is no longer arbitrary. It is mathematically equivalent, up to an additive constant, to the Evidence Lower Bound (ELBO) for this Bayesian inference problem. Consequently, maximizing the RLHF objective via PPO is a form of variational inference, where the aim is to find the policy π_ϕ that minimizes the KL divergence to the ideal posterior π^* . This insight reframes RLHF from a reward-maximization task into a probabilistic inference problem, providing a theoretical justification for the KL penalty as a structural component of the inference problem rather than an ad-hoc regularizer.

Despite this closed-form characterization, the three-stage RLHF pipeline is complex to implement, requiring training multiple large models and a computationally expensive reinforcement learning loop that involves sampling from the policy. Building on the analytical relationship

¹³⁹ λ_{KL} denotes the KL penalty weight, reserving β for model parameters and γ for discount factors. The standard RLHF literature, including Rafailov et al. (2023), uses β for this parameter.

between the optimal policy and the reward function, Direct Preference Optimization (DPO) was introduced by Rafailov et al. (2023) as a simpler, RL-free method. DPO eliminates the RL optimization loop entirely, bypassing the policy gradient and value function machinery of Chapters 2 and 3. Where standard RLHF decomposes the problem into “learn reward from trajectory preferences” then “solve the resulting MDP,” DPO solves for the policy directly from the same preference data over trajectory pairs, collapsing two stages into one supervised learning objective. DPO bypasses the explicit reward modeling step by reparameterizing the reward function directly in terms of the optimal policy π^* and the reference policy π^{SFT} , as in Equation (88):

$$r(s, y) = \lambda_{KL} \log \left(\frac{\pi^*(y|s)}{\pi^{SFT}(y|s)} \right) + \lambda_{KL} \log Z(s). \quad (88)$$

When this analytical expression for the reward is substituted into the Bradley-Terry preference loss from Equation (85), the unknown partition function $Z(s)$ cancels out. This yields a loss function that depends only on the policy π_ϕ being optimized and the fixed reference policy π^{SFT} . The DPO objective, given in Equation (89), is thus a simple binary cross-entropy loss over policy likelihoods:

$$\mathcal{L}_{DPO}(\phi; \pi^{SFT}) = -\mathbb{E}_{(s, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\lambda_{KL} \log \frac{\pi_\phi(y_w|s)}{\pi^{SFT}(y_w|s)} - \lambda_{KL} \log \frac{\pi_\phi(y_l|s)}{\pi^{SFT}(y_l|s)} \right) \right]. \quad (89)$$

This objective can be optimized directly on the policy parameters ϕ using standard supervised learning techniques with a static preference dataset. The gradient of this loss intuitively increases the likelihood of preferred responses y_w while decreasing the likelihood of dispreferred responses y_l , relative to the reference policy. By recasting the constrained optimization problem as a simple likelihood maximization, DPO offers a more stable and computationally efficient alternative to the multi-stage RLHF pipeline, eliminating the need for an explicitly trained reward model and the complexities of RL-based policy optimization.

11.3 Recent Developments

A subsequent innovation leverages DPO to create an iterative self-improvement loop, reducing reliance on static, human-annotated data (Yuan et al., 2024). In this paradigm, dubbed Self-Rewarding Language Models, a single language model acts as both the instruction-following agent and its own reward model. The process begins with a base model fine-tuned to have both instruction-following and evaluation capabilities (“LLM-as-a-Judge”). In each subsequent iteration, the current model generates a new preference dataset for itself by producing multiple responses to a set of prompts and then evaluating its own outputs to assign scores. These scores are used to construct preference pairs (y_w, y_l) , which form an AI-generated feedback dataset. The model is then fine-tuned on this new data using the DPO loss. This iterative process creates a feedback loop where enhancements in instruction-following ability lead to better reward modeling, which in turn fuels the next round of policy optimization.

While RLHF and its successors have proven effective at aligning model behavior with human preferences, several open issues remain. The framework does not aim to solve the identification problem in the strict econometric sense; the learned reward model r_θ (whether explicit or implicit in DPO) is a proxy for preference, not necessarily the uniquely identified “true” utility function required for welfare analysis.¹⁴⁰ Furthermore, the notion of “human feedback” is a significant simplification, as the preferences being optimized are those of a small, non-representative group of paid labelers, raising important questions about whose values are being embedded in these systems. Finally, the fine-tuning process can lead to degraded performance on standard academic benchmarks, a phenomenon called the *alignment tax* (Ouyang et al.,

¹⁴⁰The identification problem in preference learning mirrors that in discrete choice. The reward function is identified only up to an additive constant and requires a location normalization.

2022).¹⁴¹ Mitigating these challenges while scaling alignment beyond the limits of human data collection remains a key frontier for the field.

11.4 Simulation Study: Learning from Preferences on a Gridworld

The gridworld is a 10×10 stochastic grid with slip probability $p = 0.1$ and discount factor $\gamma = 0.99$, structurally identical to the environments in Chapters 3a and 4. The agent starts at $(0,0)$ and must reach the goal at $(9,9)$, receiving $r = -0.1$ per step, $r = +10$ for reaching the goal, $r = -2$ for entering hazard cells at $(3,3)$ and $(6,7)$, and $r = -5$ for entering a trap at $(5,2)$. The true reward function is linear in four indicator features, $r(s, a, s') = \phi_0 + \phi_1 \mathbb{1}\{s' = \text{goal}\} + \phi_2 \mathbb{1}\{s' \in \text{hazards}\} + \phi_3 \mathbb{1}\{s' \in \text{traps}\}$, with $\phi = (-0.1, 10.1, -1.9, -4.9)$. Dynamic programming gives $V^*(s_0) = 6.55$; the optimal policy avoids all hazards and traps. Figure 11 shows the environment with $V^*(s)$ and π^* .

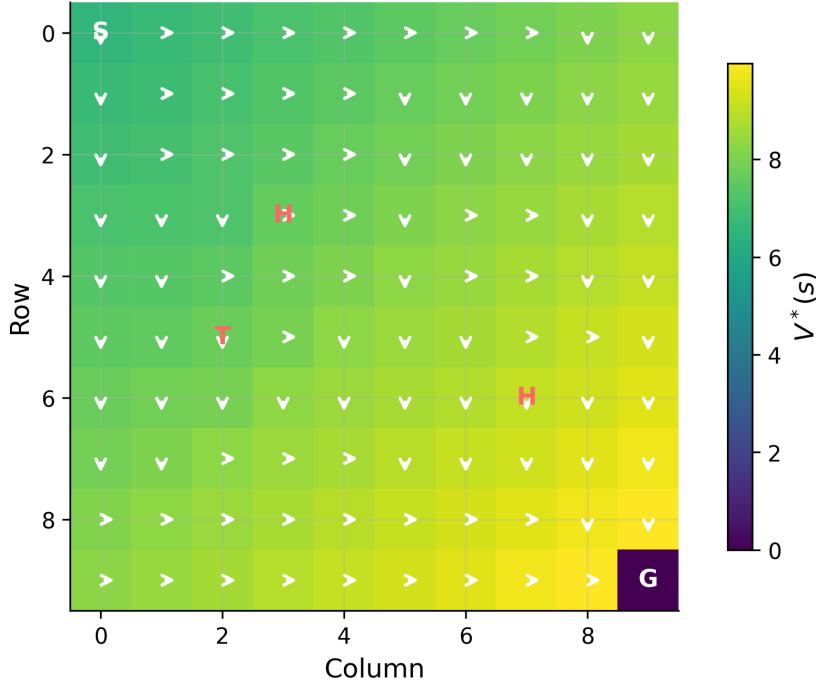


Figure 11: The 10×10 stochastic gridworld. Cell shading shows $V^*(s)$ (viridis scale); arrows indicate the optimal policy π^* . S marks the start, G the goal, H the hazard cells, and T the trap. The optimal policy routes right and down toward the goal while steering around hazards and traps.

Preference data is generated by rolling out a uniform random policy (each action equally likely) from random non-goal start states. Each rollout produces a trajectory segment of $L = 25$ steps. For each of K comparisons, two independent segments are generated; the segment with higher cumulative reward under the true (unobserved) reward function is labeled as preferred via the Bradley-Terry model. This produces K binary comparisons. All six methods receive identical preference data per seed; the agent never observes per-transition rewards, only the binary preference label per pair.

Five methods are compared across $K \in \{25, 50, 100, 200, 500, 1000, 2000, 5000\}$, averaged over 30 seeds. The neural network RLHF reward model is a two-layer MLP (8 inputs encoding normalized row and column positions plus one-hot action, 64 hidden units, $\sim 4,800$ parameters)

¹⁴¹The alignment tax, as described by Ouyang et al. (2022), refers to performance regressions on public NLP benchmarks (SQuAD, DROP, HellaSwag) that result from RLHF fine-tuning.

trained by Bradley-Terry MLE on the K segment pairs. For each segment, the model computes per-transition rewards $r_\theta(s_t, a_t, s_{t+1})$ and sums them to obtain a segment score; the preferred segment should score higher under the logistic loss from Equation (85). After 200 training epochs (learning rate 10^{-3} , batch size 64), the 100×4 reward table is extracted and the resulting MDP solved by value iteration. The correctly specified structural model parameterizes reward as $r = \phi_0 + \phi_1 \mathbb{1}\{\text{goal}\} + \phi_2 \mathbb{1}\{\text{hazard}\} + \phi_3 \mathbb{1}\{\text{trap}\}$ (4 parameters matching the true form); ϕ is fit by L-BFGS-B maximization of the same Bradley-Terry log-likelihood, and the resulting MDP solved by value iteration. The misspecified variant uses only ϕ_0 and ϕ_1 , ignoring hazards and traps. Tabular Q-learning (20,000 episodes, $\varepsilon = 0.1$, $\alpha = 0.1$) and exact DP provide scalar-reward baselines that observe per-transition rewards.

Table 20: Sample complexity of preference-based policy learning on a stochastic 10×10 gridworld (30 seeds). The neural network reward model reaches 99% of DP-optimal at $K = 1,000$. The correctly specified structural model reaches 100% at $K = 5,000$ but has high variance at intermediate K . The misspecified model plateaus near 98% while routing through hazard cells.

Method	K	Return	% of DP
DP Oracle	—	6.52 ± 0.02	100.0
Q-Learning	—	6.50 ± 0.01	99.7
NN RLHF	25	-6.28 ± 1.29	-96.3
Correct	25	4.89 ± 0.90	75.0
Misspecified	25	4.79 ± 0.90	73.5
NN RLHF	50	-3.37 ± 1.43	-51.6
Correct	50	6.00 ± 0.54	92.0
Misspecified	50	5.89 ± 0.54	90.2
NN RLHF	100	3.06 ± 1.19	47.0
Correct	100	4.35 ± 1.02	66.6
Misspecified	100	4.81 ± 0.90	73.8
NN RLHF	200	4.75 ± 0.90	72.8
Correct	200	4.90 ± 0.90	75.2
Misspecified	200	4.82 ± 0.90	73.8
NN RLHF	500	6.21 ± 0.27	95.1
Correct	500	4.89 ± 0.90	75.0
Misspecified	500	4.78 ± 0.90	73.3
NN RLHF	1000	6.45 ± 0.04	98.9
Correct	1000	5.46 ± 0.75	83.6
Misspecified	1000	5.89 ± 0.54	90.3
NN RLHF	2000	6.52 ± 0.01	99.9
Correct	2000	6.01 ± 0.54	92.1
Misspecified	2000	6.44 ± 0.02	98.7
NN RLHF	5000	6.48 ± 0.01	99.4
Correct	5000	6.55 ± 0.00	100.4
Misspecified	5000	6.42 ± 0.02	98.4

Table 20 and Figure 12 report the main results. The neural network reward model converges fastest, rising from negative returns at $K = 25$ to 95% of DP-optimal at $K = 500$ and 99% at $K = 1,000$. The correctly specified structural model dominates at very low K (75% at $K = 25$ versus -96% for the neural net) but exhibits high variance at intermediate K (standard errors of ± 0.9), because hazard and trap cells constitute only 3 of 99 non-terminal states and random segments rarely visit them, leaving ϕ_2 and ϕ_3 poorly identified.¹⁴² At $K = 5,000$, the

¹⁴²At $K = 100$, the estimated trap penalty $\hat{\phi}_3 = -18.6$ overshoots the true value of -4.9 by a factor of four,

structural model achieves 100.4% of DP return with parameter estimates converging toward their true values. The misspecified model converges to 98.4%, a seemingly small gap, but its policy consistently routes through hazard cells (2.1 visits on average), confirming qualitative misspecification.¹⁴³ Q-learning converges to 99.7% of DP using 458,000 scalar reward observations, three orders of magnitude more signal than the neural net requires at $K = 1,000$. The flexible neural net outperforms the correctly specified structural model at intermediate sample sizes ($K = 500$ through $K = 2,000$) because its 8-dimensional input representation extracts reward information from every transition, not just the rare visits to 3 special cells. Even mild misspecification (omitting 2 of 4 reward features) produces qualitatively wrong policies despite quantitatively similar returns.

Table 21: Diagnostics at $K = 5,000$ (single seed). Policy agreement measures the fraction of non-goal states where each method’s greedy policy matches π^* . Reward MAE is the mean absolute error of the estimated expected reward table against the true table. V^π correlation is the Pearson correlation between the true value of the learned policy (evaluated under the true reward) and V^* . Hazard visits counts states where the deterministic policy routes into hazard or trap cells.

Method	Policy agreement (%)	Reward MAE	V^π corr. with V^*	Hazard visits
NN RLHF	77.8	0.766	0.836	1
Correct	100.0	0.081	1.000	0
Misspecified	77.8	0.190	0.809	2

Table 21 provides a different view of the $K = 5,000$ results, showing what each method actually learned rather than just policy returns. The correctly specified structural model achieves 100% policy agreement with π^* and value function correlation 1.000, recovering the true reward to within MAE 0.081. The neural network model, despite achieving 99% of DP return in the main experiment, agrees with π^* on only 78% of states; the disagreements occur at states far from the typical trajectory where the precise action choice has little impact on expected return. The misspecified model matches the neural net in policy agreement (78%) but for a different reason; it routes through hazard cells that the optimal policy avoids, producing a qualitatively distinct trajectory distribution with value function correlation 0.809.

Table 22: Online versus offline data collection for the neural network RLHF model at $K = 1,000$ (20 seeds). Online collection is unstable; catastrophic failures on approximately half of seeds produce a negative mean return.

Collection	Return	p -value
Online (4 rounds)	-2.59 ± 1.81	
Offline	5.82 ± 0.66	0.9999

Table 22 reports the online versus offline ablation for the neural network model at $K = 1,000$. The online procedure divides K into four rounds of 250 comparisons each, collecting subsequent rounds from the current learned policy rather than the random policy. Offline collection (5.82 ± 0.66) substantially outperforms online (-2.59 ± 1.81 , $p > 0.999$). A poor first-round reward model, trained on only 250 comparisons, produces a degenerate policy whose concentrated state visitation degrades subsequent rounds through a compounding feedback loop. In this gridworld, the random policy already provides near-uniform state coverage that targeted collection can only narrow. The finding contrasts with Christiano et al. (2017), where

causing the policy to take extreme detours.

¹⁴³The modest return gap arises because hazard penalties (-2 per visit, ~ 2 visits per episode) are small relative to total return (~ 6.5). Larger penalties or more hazards would widen it.

online collection helped in high-dimensional Atari and MuJoCo environments because random exploration fails to cover the relevant state space; in small, well-covered environments, the instability risk of online collection outweighs its potential benefit.

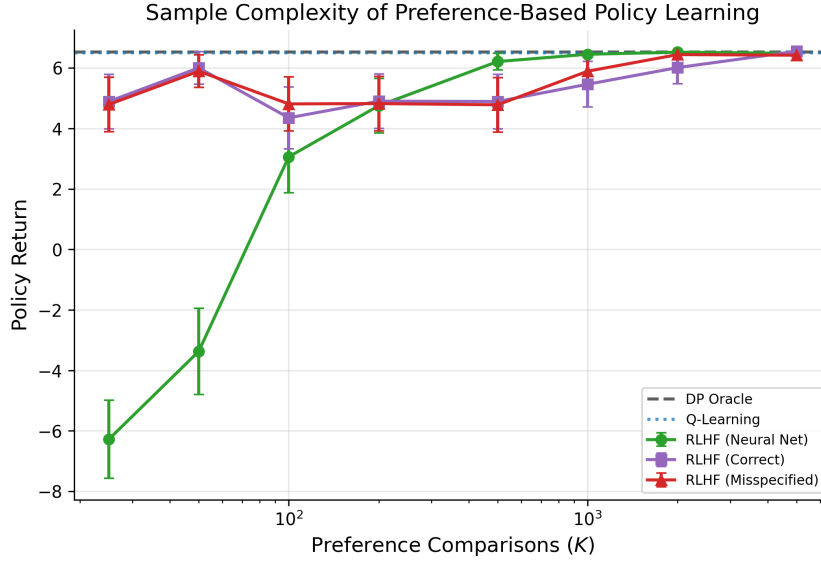


Figure 12: Policy return versus number of preference comparisons. The neural network RLHF model ($\sim 4,800$ parameters) converges fastest to near-optimal return. The structural model (4 parameters) has high variance at intermediate K but achieves exact recovery at $K = 5,000$.

12 Reinforcement Learning and Causal Inference

Reinforcement learning algorithms solve Markov decision processes by estimating value functions or optimizing policies from sampled transitions. Two assumptions underlie the standard formulation. First, the agent’s action at time t is determined solely by the observed state s_t and the agent’s policy $\pi(a \mid s_t)$; there is no unobserved variable (confounder) simultaneously influencing both the action and the reward or state transition.¹⁴⁴ Second, the observed state s_t is sufficient for prediction, so that conditioning on s_t renders future states independent of past history. Both assumptions are the Markov property restated in causal language. Both fail routinely in economic applications.

The preceding chapters operated under a third assumption that was so natural it required no mention: the analyst controls data collection. In the tabular algorithms of Section 3 and the gridworld study of Section 6, each method generated its own trajectories by executing actions in a simulator and observing the consequences. The bandit algorithms of Section 10 chose arms and observed payoffs in real time. Even when exploration was limited, the data-generating process was known because the agent’s own policy produced it. This chapter drops that assumption entirely. The analyst receives a fixed log of decisions made by someone else, a *behavioral policy* μ whose functional form may be unknown and whose action choices may depend on variables the analyst cannot observe. The data are observational in the econometric

¹⁴⁴In causal inference, “outcome” refers specifically to the response variable Y (or the potential outcome $Y(d)$ in the Rubin framework). The RL/MDP framework has no single corresponding concept called “outcome”; the relevant quantities are the reward R_t , the next state S_{t+1} , the return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$, or the value function $V^\pi(s)$. When economists refer to the “outcome model” $\mathbb{E}[Y \mid X, D]$, the RL analogue is the action-value function $Q(s, a)$. Throughout this chapter, I reserve “outcome” for its causal inference meaning and use the specific RL quantity (reward, state, return, value) elsewhere.

sense: the analyst had no role in generating them and cannot rerun the experiment under a different policy. Identification, not optimization, becomes the central problem.

Unobserved demand shocks that affect both a retailer’s pricing algorithm and consumer demand create endogeneity. The observed correlation between price and demand conflates the causal effect with the confounding effect. An RL agent trained on such observational data converges to a biased policy that systematically overestimates the revenue from high prices.

This chapter formalizes the *confounded MDP*, develops four identification strategies for recovering interventional quantities from observational data (backdoor adjustment, front-door adjustment, instrumental variables, and proximal causal inference), and demonstrates their practical consequences through a unified simulation study. For comprehensive surveys of the rapidly growing causal RL literature, including causal representation learning, counterfactual policy optimization, transfer, and fairness, I refer readers to [Deng et al. \(2023\)](#) and [da Costa Cunha et al. \(2025\)](#).

12.1 From Partial Observability to Causal Structure

Before formalizing confounded MDPs, it is useful to distinguish partial observability from confounding, since both involve hidden variables but create fundamentally different challenges. A partially observable MDP (POMDP) augments the standard MDP with an observation function. The POMDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, r, \gamma)$, where $\mathcal{O}$ is a finite observation space and O is the observation function.

$$O(o \mid s', a) = P(O_t = o \mid S_t = s', A_{t-1} = a). \quad (90)$$

The agent does not observe s_t directly but instead receives $o_t \sim O(\cdot \mid s_t, a_{t-1})$ and maintains a belief state $b_t \in \Delta(\mathcal{S})$

$$b_t(s) = P(S_t = s \mid o_1, a_1, \dots, o_t), \quad (91)$$

which is updated via Bayesian filtering at each step. The belief MDP, whose state space is $\Delta(\mathcal{S})$, is itself a fully observable (continuous-state) MDP, so standard value iteration applies in principle, though computation is intractable in general.

[da Costa Cunha et al. \(2025\)](#) organize sequential decision problems with hidden variables into a hierarchy of four settings. The standard MDP assumes full observability and no confounding. The POMDP relaxes observability: the agent knows its observation is incomplete and models its own ignorance through the belief state, but no unobserved variable distorts the data-generating process beyond the agent’s model. The confounded MDP (formalized in the next subsection) assumes full state observability but introduces unobserved confounders that distort the behavioral policy without the analyst’s knowledge. The causal POMDP combines both: partial observability and confounding act simultaneously, requiring both belief-state reasoning and causal identification.

The key distinction is epistemic versus identificational. In a POMDP, the hidden state is a modeling challenge. The agent acknowledges incomplete information and plans accordingly, using the belief state as a sufficient statistic for optimal decision-making. The econometric analogues are hidden state estimation methods such as the Kalman filter for linear-Gaussian dynamics and the Hamilton filter for Markov-switching models. In a confounded MDP, the hidden variable is an identification challenge. The analyst cannot tell from the data alone that the behavioral policy depended on an unobserved variable, and standard estimators silently produce biased results. The econometric analogues are endogeneity and omitted variable bias, addressed by instrumental variables, control functions, and the other identification strategies developed in this chapter.

12.2 The Confounded MDP

Standard MDPs assume that the state s_t is fully observed and that the agent’s policy depends only on s_t . In the online algorithms of earlier chapters, these assumptions held by construction: the agent executed its own policy in a known environment, so no unobserved variable could simultaneously influence action selection and transitions. When unobserved confounders influence both the behavioral policy and the transitions or rewards, the MDP is confounded. This formalization, developed by [Zhang and Bareinboim \(2019\)](#), [Zhang and Bareinboim \(2020\)](#), and [Kallus and Zhou \(2020\)](#), provides the foundation for causal reasoning in sequential decision problems. The key tool is Pearl’s do-operator. The interventional distribution $P(Y \mid \text{do}(X = x))$ is the distribution that arises when X is set externally rather than observed passively, severing all incoming causal influences on X while leaving the remaining data-generating process intact.¹⁴⁵

Definition D2 (Confounded MDP ([Zhang and Bareinboim, 2020](#))). *A confounded MDP is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{U}, P, r, \gamma)$ where $\mathcal{S}$ is a finite state space, $\mathcal{A}$ is a finite action space, $\mathcal{U}$ is a space of unobserved confounders, $\gamma \in [0, 1)$ is a discount factor, and the dynamics are governed by structural equations*

$$U_t \sim P_U(\cdot \mid S_t), \quad (92)$$

$$A_t \sim \mu(\cdot \mid S_t, U_t), \quad (93)$$

$$R_t = f_R(S_t, A_t, U_t) + \epsilon_t, \quad (94)$$

$$S_{t+1} \sim f_S(\cdot \mid S_t, A_t, U_t). \quad (95)$$

The behavioral (logging) policy μ depends on the unobserved confounder U_t through Equation (93). An evaluation policy $\pi(a \mid s)$ depends only on the observed state.

The separation between the behavioral policy μ and the evaluation policy π is what creates the identification problem. In the online algorithms of Sections 3–6, the behavior and target policies were identical (on-policy methods) or related by a known exploration mechanism (ϵ -greedy, softmax); here, μ is an unknown function of unobserved variables. The confounding problem arises whenever the analyst evaluates a policy using data generated by a different, unknown behavioral policy, whether the setting is offline (fixed logged data) or online with an unknown data-collection mechanism.

The key distinction is between observational and interventional transition dynamics. Under the behavioral policy μ , which depends on U_t , the conditional distribution $P(s' \mid s, a)$ obtained by conditioning on the observed action does not equal the interventional distribution obtained by setting the action externally.

$$P(s' \mid s, a) \neq P(s' \mid s, \text{do}(a)). \quad (96)$$

The left-hand side conditions on the event $\{A_t = a\}$, which carries information about U_t through the behavioral policy μ . The right-hand side intervenes on A_t , severing the dependence between action and confounder. When U_t also affects S_{t+1} through Equation (95), the conditioning introduces a spurious association between a and s' .

The Bellman equation for policy evaluation must use interventional, not observational, transition probabilities. Define the causal Bellman operator for a *target policy* π :

¹⁴⁵The do-operator is formalized within the structural causal model (SCM) framework of [Pearl \(2009\)](#). An SCM specifies endogenous variables $\mathbf{V}$, exogenous variables $\mathbf{U}$, structural equations $V_i = f_i(\text{pa}(V_i), U_i)$, and a distribution $P(\mathbf{U})$. The intervention $\text{do}(X = x)$ replaces the structural equation for X with a constant, producing the interventional distribution. See [Pearl \(2009\)](#) for the complete framework, including the causal hierarchy (association, intervention, counterfactual) and general identification theory.

Definition D3 (Causal Bellman Operator (Zhang and Bareinboim, 2020)). *The causal Bellman operator $\mathcal{T}_c^\pi$ for policy π in a confounded MDP is*

$$(\mathcal{T}_c^\pi V)(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{s' \in \mathcal{S}} P(s' | s, \text{do}(a)) [r(s, a) + \gamma V(s')], \quad (97)$$

where $r(s, a) = \mathbb{E}[R_t | S_t = s, \text{do}(A_t = a)]$ is the interventional expected reward.

When an analyst ignores confounding and uses the observational transition probabilities $P(s' | s, a)$ in place of $P(s' | s, \text{do}(a))$, the resulting value estimates are biased.

Lemma L1 (Bias of Naive Off-Policy Evaluation (Kallus and Zhou, 2020)). *Let $\hat{V}_{naive}^\pi$ denote the value function obtained by solving the Bellman equation with observational transitions $P(s' | s, a)$, and let V^π denote the true value function under interventional transitions $P(s' | s, \text{do}(a))$. In a confounded MDP where $P(s' | s, a) \neq P(s' | s, \text{do}(a))$ for some (s, a, s') , the naive estimator is biased.*

$$\hat{V}_{naive}^\pi(s) \neq V^\pi(s). \quad (98)$$

The importance-sampling estimator $\hat{V}_{IS}^\pi = \frac{1}{N} \sum_{i=1}^N \prod_{t=0}^T \frac{\pi(a_t^{(i)} | s_t^{(i)})}{\mu(a_t^{(i)} | s_t^{(i)})} G^{(i)}$, where $G^{(i)} = \sum_{t=0}^T \gamma^t R_t^{(i)}$ is the discounted return of trajectory i and T is the trajectory length, is also biased because the propensity $\mu(a | s)$ is not the true behavioral propensity $\mu(a | s, u)$.

The bias arises because the importance-sampling ratio $\pi(a | s)/\mu(a | s)$ does not correctly reweight the data. The true behavioral policy is $\mu(a | s, u)$, but since u is unobserved, the analyst can only estimate the marginal propensity $\mu(a | s) = \mathbb{E}_U[\mu(a | s, U) | s]$. Reweighting by the marginal propensity does not eliminate the confounding bias.¹⁴⁶

The backdoor criterion (Pearl, 2009) provides a path to identification. Zhang and Bareinboim (2020) apply it to the confounded MDP setting.

Theorem 7 (Backdoor Identification in Confounded MDPs (Pearl, 2009; Zhang and Bareinboim, 2020)). *Suppose a set of observed variables $\mathbf{Z}_t$ satisfies the backdoor criterion relative to (A_t, S_{t+1}) in the causal graph of the confounded MDP, meaning that $\mathbf{Z}_t$ blocks all backdoor paths from A_t to S_{t+1} and no element of $\mathbf{Z}_t$ is a descendant of A_t .¹⁴⁷ Then the interventional transition probability is identified:*

$$P(s' | s, \text{do}(a)) = \sum_{\mathbf{z}} P(s' | s, a, \mathbf{z}) P(\mathbf{z} | s). \quad (99)$$

Substituting Equation (99) into the causal Bellman operator (Equation 97) yields an identified, consistent estimator of V^π .

12.3 Backdoor-Adjusted Off-Policy Evaluation

Off-policy evaluation under confounding is precisely an average treatment effect (ATE) estimation problem in a dynamic setting (Bannon et al., 2020). The behavioral policy μ corresponds to the treatment assignment mechanism, the target policy π to the counterfactual treatment regime, and the policy value V^π to the expected potential outcome $\mathbb{E}[Y(d)]$ under the Rubin framework. Importance sampling in OPE corresponds to inverse probability weighting (IPW);

¹⁴⁶This is the sequential analogue of the omitted variable bias in linear regression. In the static case, regressing Y on X without controlling for a confounder U yields a biased coefficient. In the sequential case, the bias propagates through the Bellman recursion and can amplify over the horizon.

¹⁴⁷A backdoor path from A_t to S_{t+1} is any path in the causal graph that begins with an arrow into A_t (i.e., a non-causal path). In the confounded MDP, $A_t \leftarrow U_t \rightarrow S_{t+1}$ is a backdoor path: U_t causes both A_t and S_{t+1} , creating a spurious association. Blocking all such paths by conditioning on appropriate variables eliminates the confounding bias. See Pearl (2009), Chapter 3.

doubly robust OPE corresponds to the augmented IPW (AIPW) estimator of [Robins et al. \(1994\)](#). The key challenge, distribution shift between μ and π , is the analogue of the overlap (positivity) condition in causal inference.

In the online setting of earlier chapters, distribution shift did not arise, since the data distribution tracked the current policy throughout training, so the state-action frequencies under which value estimates were formed matched the frequencies under which the policy would be deployed.

Theorem 7 yields a concrete estimation procedure. Given logged data $\{(s_t, a_t, z_t, r_t, s_{t+1})\}_{t=1}^N$ collected under behavioral policy μ , where z_t is an observed proxy for the confounder.

1. Estimate the conditional transition model $\hat{P}(s' | s, a, z)$ and the proxy distribution $\hat{P}(z | s)$ from the logged data.
2. Compute interventional transitions via the backdoor adjustment:

$$\hat{P}(s' | s, \text{do}(a)) = \sum_z \hat{P}(s' | s, a, z) \hat{P}(z | s). \quad (100)$$

3. Solve the causal Bellman equation (Definition D3) using the estimated interventional transitions to obtain $\hat{V}^\pi$.

The doubly robust variant combines the fitted action-value function $\hat{Q}(s, a)$ (the RL analogue of the econometric outcome model) with backdoor-adjusted propensities, achieving consistency if either $\hat{Q}$ or the propensity model is correctly specified.

12.4 Alternative Identification Strategies

The backdoor criterion requires observing a variable $\mathbf{Z}_t$ that blocks all confounding paths. When no such variable is available, three alternative identification strategies apply, each with a direct econometric analogue. Figure 13 displays the causal graph for each strategy. In all three panels, the unobserved confounder U_t (gray, dashed) creates a spurious association between the action A_t and the next state S_{t+1} . The strategies differ in what additional structure they exploit. The front-door criterion (panel a) requires an observed mediator M_t through which the entire causal effect of A_t on S_{t+1} is channeled. The instrumental variables approach (panel b) requires an exogenous variable Z_t that shifts the action but has no direct effect on the next state. Proximal causal inference (panel c) requires two proxy variables $W_t^{(1)}$ and $W_t^{(2)}$ that are noisy measurements of the latent confounder, enabling identification through bridge functions without ever observing U_t directly.

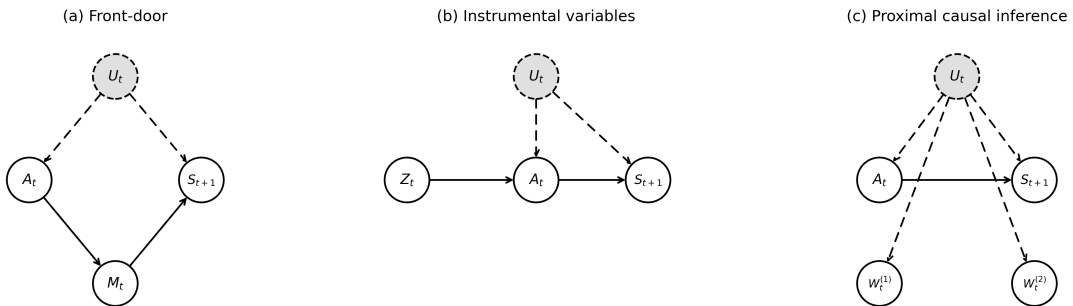


Figure 13: Causal graphs for three identification strategies in confounded MDPs. Gray dashed nodes are unobserved; dashed edges involve unobserved variables. (a) Front-door: A_t affects S_{t+1} only through mediator M_t . (b) Instrumental variables: exogenous Z_t shifts A_t without directly affecting S_{t+1} . (c) Proximal: two proxies $W_t^{(1)}, W_t^{(2)}$ of the latent U_t enable bridge function identification.

12.4.1 Front-Door Criterion

When the backdoor criterion fails because confounders are unobserved, the front-door criterion offers an alternative if the action A_t affects the next state S_{t+1} only through an observed mediator M_t . The front-door criterion requires three conditions: (i) M_t intercepts all directed paths from A_t to S_{t+1} , (ii) no unblocked backdoor path exists from A_t to M_t , and (iii) A_t blocks all backdoor paths from M_t to S_{t+1} . When satisfied, the interventional distribution is identified by a two-stage formula that first estimates how A_t affects M_t , then how M_t affects S_{t+1} while adjusting for A_t (Pearl, 2009):

$$P(s' \mid s, \text{do}(a)) = \sum_m P(m \mid a) \sum_{a'} P(s' \mid s, m, a') P(a' \mid s). \quad (101)$$

The first factor $P(m \mid a)$ is unconfounded because U_t does not block the $A_t \rightarrow M_t$ path. The inner sum adjusts for confounding on the $M_t \rightarrow S_{t+1}$ link by averaging over the observational action distribution, which serves as a backdoor adjustment for M_t . This decomposition is the sequential analogue of mediation analysis in econometrics (da Costa Cunha et al., 2025).

The causal RL survey of da Costa Cunha et al. (2025) illustrates the front-door criterion with a mobile wellness intervention. A health app (action A_t) aims to reduce patient cortisol levels (outcome S_{t+1}), but unobserved health consciousness U_t confounds the relationship because health-conscious individuals are both more likely to adopt the app and more likely to have low cortisol regardless. The app affects cortisol only through an observed mediator, supplement adherence M_t . The front-door criterion exploits this structure in two stages. First, the effect of app usage on supplement adherence is unconfounded because health consciousness does not block the causal path from A_t to M_t . Second, the effect of supplement adherence on cortisol is identified by adjusting for app usage, which satisfies the backdoor criterion for the $M_t \rightarrow S_{t+1}$ link. Combining these two estimates recovers the causal effect of the app on cortisol without ever observing health consciousness.

12.4.2 Instrumental Variables

When neither backdoor nor front-door variables are available, instrumental variable methods can identify causal effects. An instrument Z_t must satisfy two conditions: it affects the action A_t (relevance) and its effect on S_{t+1} is channeled entirely through A_t (exclusion restriction). Liao et al. (2024) formalize this as a Confounded MDP with Instrumental Variables (CMDP-IV), where transitions take the form $S_{t+1} = F^*(S_t, A_t) + \epsilon_t$ with unobserved confounders ϵ_t affecting both the behavior policy and transitions. The transition function F^* is recovered from a conditional moment restriction:

$$\mathbb{E}[S_{t+1} - F^*(S_t, A_t) \mid Z_t, S_t] = 0. \quad (102)$$

For a binary action and binary instrument, the Wald estimator provides a closed-form solution. Let β denote the causal effect of promoting (action $a = 1$) on the transition probability.

$$\beta = \frac{P(s' \mid s, Z=1) - P(s' \mid s, Z=0)}{P(A=1 \mid s, Z=1) - P(A=1 \mid s, Z=0)}, \quad (103)$$

where the numerator is the reduced-form effect of the instrument on the transition and the denominator is the first-stage effect on treatment uptake. The interventional transition is then recovered from any instrument value z via $P(s' \mid s, \text{do}(a=1)) = P(s' \mid s, Z=z) + \beta \cdot (1 - P(A=1 \mid s, Z=z))$. Their IV-aided Value Iteration algorithm applies this moment restriction at each state to estimate F^* , then runs standard value iteration on the estimated model.

Liao et al. (2024) illustrate with neonatal intensive care unit (NICU) assignment. A hospital must decide whether to admit each newborn to the NICU (action A_t), and this decision is

confounded by unobserved severity indicators that affect both the admission decision and the infant’s health trajectory. Differential travel time from the hospital to a specialty care provider serves as an instrument Z_t . Relevance holds because longer travel times discourage NICU referrals, shifting the admission probability. The exclusion restriction holds because travel time affects infant health outcomes only through the admission decision, not directly. The conditional moment restriction (Equation 102) uses variation in travel time across hospitals to trace out the causal effect of NICU admission on health transitions, recovering the transition function F^* that standard regression conflates with the unobserved confounder.

12.4.3 Proximal Causal Inference

When confounders are truly latent but proxy variables, noisy correlates of the confounder, are available, proximal causal inference provides identification. [Bennett and Kallus \(2021\)](#) adapt this to sequential settings. The analyst observes two proxies: a treatment-side proxy $W_t^{(1)}$ and an outcome-side proxy $W_t^{(2)}$, both conditionally independent given the latent confounder U_t . Identification proceeds through a bridge function h that solves a conditional moment equation linking the two proxies to the interventional quantity:

$$\mathbb{E}[V(S_{t+1}) \mid W_t^{(1)}, S_t, A_t] = \mathbb{E}[h(W_t^{(2)}, S_t, A_t) \mid W_t^{(1)}, S_t, A_t]. \quad (104)$$

The bridge function h is estimated by solving this integral equation (which reduces to a linear system in the discrete case), and the causal effect is recovered by marginalizing over the outcome proxy distribution.

$$P(s' \mid s, \text{do}(a)) = \sum_{w_2} h(w_2, s, a) P(W_t^{(2)}=w_2 \mid s). \quad (105)$$

Two bridge functions, analogous to inverse propensity scores and Q-functions, yield a doubly robust estimator of the policy value that is $\sqrt{n}$ -consistent without ever observing U_t . [Bennett and Kallus \(2021\)](#) demonstrate with a sepsis management simulator in which physicians choose among fluids, vasopressors, and antibiotics at each decision point. The patient’s diabetes status is the latent confounder, partially censored in the medical record so that 20% of diabetic patients appear as non-diabetic in the data. Because the physician observes the true diabetes status when prescribing but the analyst’s dataset contains the censored version, the behavioral policy depends on a variable the analyst cannot fully recover. Previous clinical observations O_{t-1} (prior lab values and vitals) serve as the treatment-side proxy $W_t^{(1)}$ because they correlate with diabetes status and influence the physician’s current prescribing. Current clinical observations O_t serve as the outcome-side proxy $W_t^{(2)}$ because they reflect diabetes status and predict future health transitions. In their experiments, the proximal estimator correctly identified which evaluation policy improved over the behavioral policy in 82–100% of test cases, while naive estimators that ignored confounding and standard MDP estimators that assumed full observability both achieved 0% accuracy.

12.5 The Broader Causal RL Landscape

The confounded MDP and the identification strategies above address one class of problems, but the intersection of causality and reinforcement learning extends considerably further. Three active research directions illustrate the scope.

Causal representation learning for RL seeks state representations $\phi(s)$ that capture causal mechanisms rather than spurious correlations. When superficial statistics change across environments but the underlying causal structure is preserved, policies learned on invariant causal features generalize where standard representations fail ([Schölkopf et al., 2021](#)). [da Costa Cunha et al. \(2025\)](#) formalize this as invariant policy optimization within a multi-environment MDP framework. Sim-to-real robotics illustrates the problem. A robot arm trained in a simulator

with blue backgrounds and simple textures may learn to associate visual quirks with correct actions rather than learning the actual object positions that matter. When deployed on a real factory floor with different lighting, the policy fails. Training across multiple visually distinct simulators, same physics but different textures and lighting, forces the state representation to discard renderer-specific features and retain only causally relevant ones like object pose and joint angles (da Costa Cunha et al., 2025).

Counterfactual policy optimization uses structural causal models to generate “what-if” trajectories for credit assignment and data augmentation. Buesing et al. (2019) introduce Gumbel-Max SCMs that produce counterfactual rollouts from a single observed trajectory, enabling off-policy evaluation without importance weights. This follows Pearl’s Abduction-Action-Prediction procedure (Pearl, 2009): first infer the exogenous noise U that explains the observed trajectory (abduction), then replace the logged action with the counterfactual action (action), and finally propagate the modified action through the original structural equations to predict the counterfactual outcome (prediction). Oberst and Sontag (2019) extend this to Gumbel-Max SCM-based counterfactual off-policy estimation in healthcare settings, while Forney et al. (2017) demonstrate that counterfactual data-fusion can augment online RL exploration by generating synthetic transitions consistent with the learned causal model. In Atari Breakout, moving the paddle determines whether a brick breaks many timesteps later, but which of the dozens of intervening actions actually mattered? Buesing et al. (2019) answer this by replaying an observed trajectory with one action changed while holding the environment’s random outcomes fixed. Because the real and counterfactual trajectories share the same randomness, any difference in reward is caused by that single action change, solving the long-horizon credit assignment problem without the compounding errors of model rollouts or the high variance of importance sampling.

Causal transfer in RL applies the transportability theory of Pearl and Bareinboim (2014) and Bareinboim and Pearl (2016) to policy transfer across domains. Causal invariance identifies which components of a learned policy remain valid when the environment changes, enabling transfer in settings where standard domain adaptation, which relies on distributional similarity, fails because the intervention targets differ. Autonomous driving illustrates the advantage. A self-driving policy trained in simulation has access to millions of miles of data, but the simulator’s graphics differ from real roads. Rather than treating this as one large distribution shift, selection diagrams identify which mechanisms are shared (physical dynamics of steering and braking) and which differ (visual rendering, weather effects) (Bareinboim and Pearl, 2016). The shared mechanisms transfer directly; only the differing components require recalibration with scarce real-world data.

For comprehensive surveys covering these directions and others, including fairness, safety, and partial observability, see Deng et al. (2023) and da Costa Cunha et al. (2025).

12.6 Simulation Study: Confounded Retail Pricing MDP

Unlike the simulations in Sections 6–7, where each algorithm generated its own training data through interaction, the estimators here receive a fixed dataset collected under the behavioral policy μ . I construct a 5-state engagement funnel $\mathcal{S} = \{0, 1, 2, 3, 4\}$ with two actions (promote, hold price) and an absorbing conversion state at $s = 4$. A retailer manages customers through engagement stages, deciding whether to offer a promotional discount. The data-generating process embeds four distinct sources of causal variation, enabling simultaneous validation of all four identification strategies from a single DGP.

Figure 14 displays the complete causal graph. The causal structure at each step involves six variables. Market conditions $Z_t \sim \text{Bernoulli}(0.5)$ are observed and affect both the latent confounder and transitions. Consumer sentiment U_t is an unobserved confounder with $P(U_t=1 \mid Z_t=1) = 0.9$ and $P(U_t=1 \mid Z_t=0) = 0.1$. An independent cost shock $IV_t \sim \text{Bernoulli}(0.5)$ serves as an instrument. The behavioral pricing policy depends on both U_t and

IV_t: $\mu(\text{promote} \mid s, U_t, \text{IV}_t) = 0.55 + \rho \cdot 0.25 \cdot (2U_t - 1) + 0.15 \cdot (\text{IV}_t - 0.5)$, where $\rho \in \{0, 0.2, \dots, 1.0\}$ controls confounding strength. Promotions trigger marketing follow-ups: marketing effort $M_t \sim \text{Bernoulli}(0.8)$ when the retailer promotes and $\text{Bernoulli}(0.2)$ otherwise, serving as a mediator. Two noisy proxies of U_t are available: a CRM score $W_t^{(1)} \sim \text{Bernoulli}(0.85 \cdot U_t + 0.15 \cdot (1 - U_t))$ and browsing behavior $W_t^{(2)} \sim \text{Bernoulli}(0.75 \cdot U_t + 0.25 \cdot (1 - U_t))$.

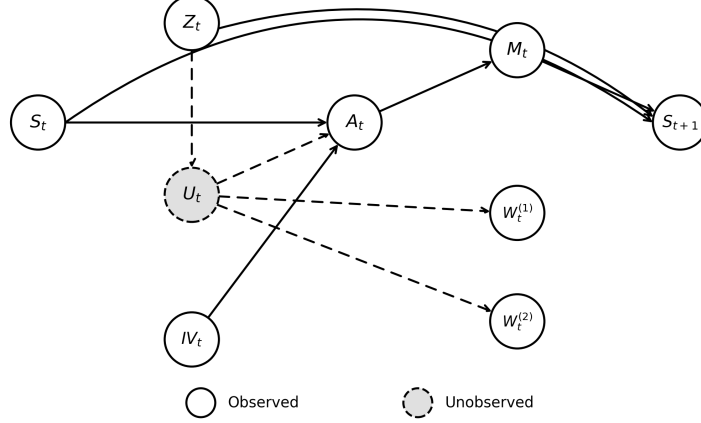


Figure 14: Complete causal graph of the simulation DGP. Gray dashed node (U_t) is unobserved; dashed edges involve U_t . The action A_t affects the next state S_{t+1} only through the mediator M_t . Market conditions Z_t cause both U_t and (with M_t) determine transitions. The cost shock IV_t is an independent instrument. Proxies $W_t^{(1)}$ and $W_t^{(2)}$ are noisy measurements of U_t .

The critical design property is that the action affects the next state only through the mediator. Specifically, $P(s+1 \mid s, M_t, Z_t)$ depends on M_t and Z_t but not on A_t or U_t directly. This enables all four identification strategies simultaneously. The backdoor criterion is satisfied by Z_t , which blocks the confounding path $A_t \leftarrow U_t \leftarrow Z_t \rightarrow S_{t+1}$. The front-door criterion is satisfied by M_t , since $A_t \rightarrow M_t \rightarrow S_{t+1}$ is the only directed path from action to next state. The cost shock IV_t serves as an instrument, shifting the promotion probability (relevance) but does not affect transitions directly (exclusion). The proxies $W_t^{(1)}$ and $W_t^{(2)}$ are conditionally independent given U_t and satisfy the treatment-proxy and outcome-proxy conditions for proximal identification. Rewards are a deterministic step cost $r(s, a) = -1$ for $s < 4$, with $\gamma = 0.9$. The target policy always promotes. The true interventional transition probability is $P(s+1 \mid s, \text{do}(\text{promote})) = 0.615$.

I compare six estimators: the oracle (analytical Bellman solution with true interventional transitions); the naive estimator (Bellman solution with observational transitions $\hat{P}(s' \mid s, a)$, biased per Lemma L1); the backdoor-adjusted estimator (Equation 100); the front-door estimator (Equation 101); the Wald IV estimator (Equation 103); and the proximal bridge function estimator (Equations 104–105). Each configuration uses 2,000 trajectories averaged over 20 seeds.

Table 23 reports bias and root mean squared error across confounding strengths. At $\rho = 0$, all estimators have near-zero bias, confirming correct implementation. As ρ increases, the naive estimator’s bias grows monotonically to +0.32 at $\rho = 1$, because the promote action is more likely when $U_t = 1$ (positive sentiment), which correlates with $Z_t = 1$ (favorable conditions) and high marketing effort, so observational transitions overestimate the promotion success probabil-

ity ($\hat{P}_{\text{obs}} \approx 0.68$ versus the true 0.615). This per-step bias compounds through the Bellman recursion across the four non-absorbing states. The backdoor and front-door estimators eliminate this bias (RMSE < 0.03 at all ρ), directly validating the identification formulas of Theorem 7 and Equation (101). The IV estimator maintains low bias but exhibits higher variance (RMSE ≈ 0.13) because the Wald ratio divides by the first-stage coefficient, amplifying sampling noise when the instrument is moderate. Panel (b) of Figure 15 isolates this effect: at $\rho = 1$ with a weak instrument (coefficient 0.05, first-stage ≈ 0.05), the IV estimator’s bias distribution spans $(-5, +1)$ with enormous variance; increasing the instrument strength to 0.30 (first-stage ≈ 0.30) shrinks the interquartile range by an order of magnitude, illustrating the classic bias-variance tradeoff of the Wald estimator. The proximal estimator achieves low bias (<0.02 at all ρ) with moderate variance, confirming that the bridge function correctly recovers the causal effect from noisy proxies of the latent confounder. Panel (a) of Figure 15 displays the bias trajectories.

Table 23: Unified identification in a confounded retail pricing MDP: bias and RMSE of five estimators across confounding strengths ρ (20 seeds, 2,000 trajectories per seed). All four identification strategies (backdoor, front-door, IV, proximal) recover approximately unbiased value estimates, while the naive estimator’s bias grows monotonically with confounding.

ρ	Naive		Backdoor		Front-door		IV		Proximal	
	Bias (SE)	RMSE	Bias (SE)	RMSE	Bias (SE)	RMSE	Bias (SE)	RMSE	Bias (SE)	RMSE
0.0	−0.010 (0.005)	0.024	−0.008 (0.005)	0.024	−0.001 (0.005)	0.023	−0.006 (0.028)	0.122	−0.011 (0.006)	0.028
0.2	+0.060 (0.005)	0.064	−0.007 (0.006)	0.025	+0.003 (0.006)	0.025	+0.012 (0.028)	0.125	−0.012 (0.006)	0.027
0.4	+0.127 (0.005)	0.129	−0.008 (0.006)	0.026	+0.001 (0.006)	0.027	+0.011 (0.029)	0.127	−0.015 (0.006)	0.028
0.6	+0.194 (0.005)	0.195	−0.007 (0.006)	0.028	+0.001 (0.006)	0.028	+0.020 (0.030)	0.131	−0.014 (0.007)	0.033
0.8	+0.258 (0.005)	0.259	−0.007 (0.006)	0.029	−0.004 (0.007)	0.030	+0.032 (0.030)	0.136	−0.015 (0.007)	0.034
1.0	+0.321 (0.005)	0.322	−0.005 (0.006)	0.028	−0.004 (0.007)	0.030	+0.055 (0.025)	0.123	−0.015 (0.008)	0.037

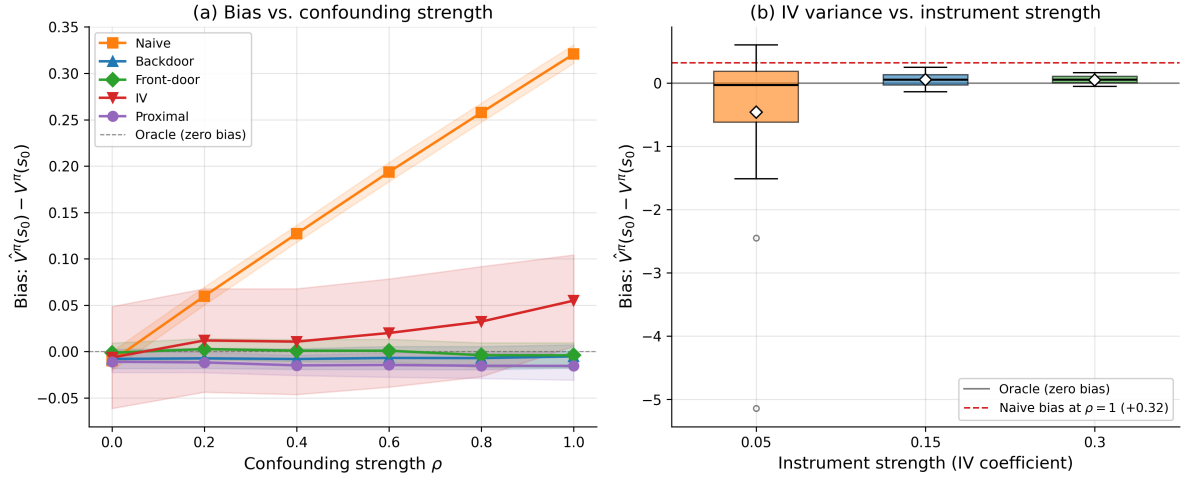


Figure 15: (a) Bias of five OPE estimators relative to the oracle as a function of confounding strength ρ . Naive bias grows monotonically; all four identification strategies remain approximately unbiased. (b) IV estimator bias distribution as a function of instrument strength at $\rho = 1$ (maximum confounding). A weak instrument (coefficient 0.05, first-stage ≈ 0.05) produces extreme variance because the Wald ratio (Equation 103) divides by a small first-stage coefficient, amplifying sampling noise. Increasing the instrument strength to 0.30 reduces the interquartile range by an order of magnitude. The dashed red line marks the naive estimator’s bias for reference.

13 Conclusion

Economics has long relied on dynamic programming to formalize and solve problems of sequential decision-making. Yet, for decades, the complexity of our models has been dictated less by the economic phenomena we wish to study and more by the computational limits of the curse of dimensionality. Reinforcement learning offers one pragmatic path forward. By replacing exhaustive state-space enumeration with sampling and function approximation, RL trades the absolute convergence guarantees of classical dynamic programming for the capacity to operate in high-dimensional, continuous, and complex strategic environments.

As this survey has sought to illustrate, RL is not a departure from dynamic programming, but a scalable and natural extension of it. It does come with its own issues. Deep reinforcement learning, in particular, remains brittle, sensitive to hyperparameters, and vulnerable to the structural instabilities of the deadly triad. Furthermore, its success in structural estimation and policy design relies heavily on computational tricks and simplifying assumptions, the availability of highly accurate simulators, and clearly defined transition or reward functions.

However, the intersection of these two fields is deeply complementary. While RL equips economics with new tools, economics provides RL with much-needed structure. As demonstrated in applications ranging from dynamic pricing to causal inference, incorporating fundamental economic concepts, such as revealed preference, parametric assumptions, or causal identification, can tame RL’s sample complexity and prevent algorithms from converging on systematically biased policies.

Looking ahead, we should view reinforcement learning simply as another tool in the applied economist’s toolkit. As these algorithms mature and their theoretical foundations solidify, the novelty of using “AI” to solve economic models will likely fade. However, one can expect interesting cross fertilization between these computational fields, that gradually expands the frontier of the models we can solve, the mechanisms we can design, and the behaviors we can understand.

References

- S. Adusumilli, M. Eckardt, and G. Tate. Estimation of dynamic discrete choice models with differentiable temporal-difference learning. *arXiv preprint arXiv:2209.15174*, 2022.
- Alekh Agarwal, Sham Kakade, and Lin F. Yang. Model-based reinforcement learning with a generative model is minimax optimal. In *Conference on Learning Theory (COLT)*, 2020a.
- Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98), 2021a.
- Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. An optimistic perspective on offline reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020b.
- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, 2021b.
- Shipra Agrawal and Wei Tang. Dynamic pricing with reference price effects. *arXiv preprint arXiv:2301.02497*, 2024.
- Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–40, 2001.

- Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2): 251–276, 1998.
- Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Leonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- András Antos, Csaba Szepesvári, and Rémi Munos. Learning near-optimal policies with Bellman-residual minimization based fitted policy iteration and a single sample path. *Machine Learning*, 71(1):89–129, 2008. doi: 10.1007/s10994-007-5038-2.
- Peter Arcidiacono and Robert A. Miller. Conditional choice probability estimation of dynamic discrete choice models with unobserved heterogeneity. *Econometrica*, 79(6):1823–1867, 2011.
- John Asker, Chaim Fershtman, Jihye Jeon, and Ariel Pakes. A computational framework for analyzing dynamic auctions: The market impact of information sharing. *The RAND Journal of Economics*, 51(3):805–839, 2020.
- Tohid Atashbar and Rui Aruhan Shi. Deep reinforcement learning: Emerging trends in macroeconomics and future prospects. Working Paper 2022/259, International Monetary Fund, 2022.
- Tohid Atashbar and Shuping Shi. Solving macroeconomic models with deep reinforcement learning. *Journal of Economic Dynamics and Control*, 2023.
- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2):235–256, 2002.
- Mohammad Gheshlaghi Azar, Rémi Munos, and Hilbert J Kappen. Minimax pac bounds on the sample complexity of reinforcement learning with a generative model. *Machine Learning*, 91(1):7–32, 2013.
- Ashwinkumar Badanidiyuru, Robert Kleinberg, and Aleksandrs Slivkins. Bandits with knapsacks. *IEEE 54th Annual Symposium on Foundations of Computer Science*, pages 207–216, 2013.
- Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 30–37. Morgan Kaufmann, 1995.
- James Bannon, Brad Langlois, Raimundo Fernandez, and Danielle Maddix. Causality and batch reinforcement learning: Complementary approaches to planning in unknown domains. In *NeurIPS Workshop on Causal Discovery and Causality-Inspired Machine Learning*, 2020.
- Elias Bareinboim and Judea Pearl. Causal inference and the data-fusion problem. *Proceedings of the National Academy of Sciences*, 113(27):7345–7352, 2016.
- Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.
- Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- Andrew Bennett and Nathan Kallus. Proximal reinforcement learning: Efficient off-policy evaluation in partially observed Markov decision processes. *Operations Research*, 2021.

- Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*. Athena Scientific, 1996.
- Dimitri P. Bertsekas. Lessons from AlphaZero for optimal, model predictive, and adaptive control. *Athena Scientific Reports*, 2021.
- Dimitri P. Bertsekas. *Abstract Dynamic Programming*. Athena Scientific, Belmont, MA, 3rd edition, 2022a.
- Dimitri P. Bertsekas. Newton’s method for reinforcement learning and model predictive control. *Results in Control and Optimization*, 7:100121, 2022b.
- Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, Belmont, MA, 1996.
- Jalaj Bhandari, Daniel Russo, and Raghav Singal. A finite time analysis of temporal difference learning with linear function approximation. *Operations Research*, 69(3), 2021.
- David Blackwell. Discounted dynamic programming. *Annals of Mathematical Statistics*, 36(1): 226–235, 1965.
- Tilman Börgers and Rajiv Sarin. Learning through reinforcement and replicator dynamics. *Journal of Economic Theory*, 77(1):1–14, 1997.
- Vivek S. Borkar. Stochastic approximation with two time scales. *Systems & Control Letters*, 29(5):291–294, 1997.
- Vivek S. Borkar and Sean P. Meyn. The ODE method for convergence of stochastic approximation and reinforcement learning. *SIAM Journal on Control and Optimization*, 38(2):447–469, 2000.
- Michael Bowling and Manuela Veloso. Multiagent learning using a variable learning rate. *Artificial Intelligence*, 136(2):215–250, 2002.
- Michael Bowling, Neil Burch, Michael Johanson, and Oskari Tammelin. Heads-up limit hold’em poker is solved. *Science*, 347(6218):145–149, 2015.
- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- Gianluca Brero, Alon Eden, Matthias Gerstgrasser, David C. Parkes, and Duncan Rheingans-Yoo. Reinforcement learning of sequential price mechanisms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13662–13670, 2021.
- William A. Brock and Leonard J. Mirman. Optimal economic growth and uncertainty: The discounted case. *Journal of Economic Theory*, 4(3):479–513, 1972.
- Josef Broder and Paat Rusmevichientong. Dynamic pricing under a general parametric choice model. *Operations Research*, 60(4):965–980, 2012.
- George W. Brown. Iterative solution of games by fictitious play, 1951.
- Noam Brown and Tuomas Sandholm. Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.
- Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456): 885–890, 2019.

- Noam Brown, Adam Lerer, Sam Gross, and Tuomas Sandholm. Deep counterfactual regret minimization. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 793–802. PMLR, 2019.
- Lars Buesing, Theophane Weber, Yori Zwols, Sebastien Racaniere, Arthur Guez, Jean-Baptiste Lespiau, and Nicolas Heess. Woulda, coulda, shoulda: Counterfactually-guided policy search. In *International Conference on Learning Representations*, 2019.
- Jeremy I. Bulow. Durable-goods monopolists. *Journal of Political Economy*, 90(2):314–332, 1982.
- Junhui Cai, Ran Chen, Martin J. Wainwright, and Linda Zhao. Doubly high-dimensional contextual bandits: An interpretable model for joint assortment-pricing. *arXiv preprint arXiv:2309.07956*, 2023.
- Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.
- Shicong Cen, Chen Cheng, Yuxin Chen, Yuting Wei, and Yuejie Chi. Fast global convergence of natural policy gradient methods with entropy regularization. *Operations Research*, 70(4), 2022.
- Yiwei Chen, Guillermo Gallego, and Tao Li. A reinforcement learning approach for hotel revenue management. *Journal of Operations Management*, 2023.
- Yuxin Chen, Jieming Mao, and Rui Miao. Dynamic pricing with fairness constraints. *arXiv preprint arXiv:2402.07834*, 2025.
- Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Kamil Ciosek, Quan Vuong, Robert Lierowski, and Katja Hofmann. Better exploration with optimistic actor-critic. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Andrew J. Clark and Herbert Scarf. Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4):475–490, 1960.
- Pierre Clavier, Erwan Le Pennec, and Matthieu Geist. Towards minimax optimality of model-based robust reinforcement learning. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2024.
- Ronald H. Coase. Durability and monopoly. *Journal of Law and Economics*, 15(1):143–149, 1972.
- Matias Covarrubias. Dynamic oligopoly and monetary policy: A deep reinforcement learning approach. Job Market Paper, New York University, 2022.
- Cristiano da Costa Cunha, Wei Liu, Tim French, and Ajmal Mian. Unifying causal reinforcement learning: Survey, taxonomy, algorithms and applications. *arXiv preprint arXiv:2512.18135*, 2025.
- Constantinos Daskalakis, Paul W. Goldberg, and Christos H. Papadimitriou. The complexity of computing a nash equilibrium. *SIAM Journal on Computing*, 39(1):195–259, 2009.

- Peter Dayan. The convergence of TD(λ) for general λ . *Machine Learning*, 8(3–4):341–362, 1992.
- Eric V. Denardo. Contraction mappings in the theory underlying dynamic programming. *SIAM Review*, 9(2):165–177, 1967.
- Zhihong Deng, Jing Jiang, Guodong Long, and Chengqi Zhang. Causal reinforcement learning: A survey. *Transactions on Machine Learning Research*, 2023.
- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Mahmoud, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.
- Pierluca D’Oro, Max Schwarzer, Evgenii Nikishin, Pierre-Luc Bacon, Marc G. Bellemare, and Aaron Courville. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- Theresa Eimer, Marius Lindauer, and Roberta Raileanu. Hyperparameters in reinforcement learning and how to tune them. In *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2023.
- Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep RL: A case study on PPO and TRPO. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.
- Eyal Even-Dar and Yishay Mansour. Learning rates for q-learning. *Journal of Machine Learning Research*, 5:1–25, 2003.
- Jianqing Fan, Yongyi Guo, and Mengxin Yu. Semiparametric dynamic pricing. *arXiv preprint arXiv:2401.01136*, 2024.
- John Fearnley. Exponential lower bounds for policy iteration. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 551–562, 2010.
- William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Marc G. Bellemare. Revisiting fundamentals of experience replay. In *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2020.
- Mattie Fellows, Matthew J. A. Smith, and Shimon Whiteson. Why target networks stabilise temporal difference methods. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 9886–9909. PMLR, 2023.
- Jesús Fernández-Villaverde, Samuel Hurtado, and Galo Nuño. Financial frictions and the wealth distribution. *Econometrica*, 91(3):869–901, 2023.
- Jesús Fernández-Villaverde, Galo Nuño, and Jesse Perla. Taming the curse of dimensionality: Quantitative economics with deep learning. Working Paper 33117, National Bureau of Economic Research, 2024.

- Chaim Fershtman and Ariel Pakes. Dynamic games with asymmetric information: A framework for empirical work. *The Quarterly Journal of Economics*, 127(4):1611–1661, 2012.
- Andrew Forney, Judea Pearl, and Elias Bareinboim. Counterfactual data-fusion for online reinforcement learners. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1156–1164, 2017.
- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2018.
- Scott Fujimoto, David Meger, Doina Precup, Ofir Nachum, and Shixiang Shane Gu. Why should I trust you, Bellman? the Bellman error is a poor replacement for value error. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, 2022.
- Guillermo Gallego and Garrett van Ryzin. Optimal dynamic pricing of inventories with stochastic demand over finite horizons. *Management Science*, 40(8):999–1020, 1994.
- Ravi Ganti, Matyas Sustik, Quoc Tran, and Brian Seaman. Thompson sampling for dynamic pricing. *arXiv preprint arXiv:1802.03050*, 2018.
- Jim Gao. Machine learning applications for data center optimization. Technical report, Google, 2014.
- Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized Markov decision processes. In *International Conference on Machine Learning (ICML)*, 2019.
- Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on dual sourcing, lost sales, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022.
- Christian Gourieroux, Alain Monfort, and Eric Renault. Indirect inference. *Journal of Applied Econometrics*, 8(S1):S85–S118, 1993.
- Amy Greenwald and Keith Hall. Correlated q-learning. In *Proceedings of the 20th International Conference on Machine Learning*, pages 242–249, 2003.
- Faruk Gul, Hugo Sonnenschein, and Robert Wilson. Foundations of dynamic monopoly and the Coase conjecture. *Journal of Economic Theory*, 39(1):155–190, 1986.
- Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.
- Ben Hambly, Renyuan Xu, and Huining Yang. Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3):437–503, 2023.
- Xiao Han, Weijian Zhang, Jiaxin Wang, Fan Zhang, and Jieping Ye. A better match for drivers and riders: Reinforcement learning at Lyft. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2927–2936, 2022.

- Moritz Hardt, Nimrod Megiddo, Christos Papadimitriou, and Mary Wootters. Strategic classification. In *Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science (ITCS)*, pages 111–122, 2016.
- Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*, 2016.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- Brett Hollenbeck. Horizontal mergers and innovation in concentrated industries. *Quantitative Marketing and Economics*, 2019.
- V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.
- Ronald A. Howard. *Dynamic Programming and Markov Processes*. The Technology Press of M.I.T. and John Wiley and Sons, New York, NY, 1960.
- Junling Hu and Michael P Wellman. Nash q-learning for general-sum stochastic games. *Journal of Machine Learning Research*, 4(Nov):1039–1069, 2003.
- Yingyao Hu and Matthew Shum. Nonparametric identification of dynamic models with unobserved state variables. *Journal of Econometrics*, 171(1):32–44, 2012.
- Yingyao Hu and Fangzhu Yang. Estimation of dynamic discrete choice models with unobserved state variables using reinforcement learning. *Working Paper, Johns Hopkins University*, 2025.
- Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weng Wang. The 37 implementation details of proximal policy optimization. *ICLR Blog Track*, 2022.
- Mitsuru Igami. Artificial intelligence as structural estimation: Deep blue, bonanza, and alphago. *The Econometrics Journal*, 23(3):S1–S24, 2020.
- Andrew Ilyas, Logan Engstrom, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. A closer look at deep policy gradients. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems 6*, pages 703–710. Morgan Kaufmann, 1994.
- Adel Javanmard and Hamid Nazerzadeh. Dynamic pricing in high-dimensions. *Journal of Machine Learning Research*, 20(9):1–49, 2019.
- Sham M. Kakade. A natural policy gradient. *Advances in Neural Information Processing Systems*, 14, 2001.
- Sham M. Kakade. *A Natural Policy Gradient*. PhD thesis, University College London, 2002.
- Nathan Kallus and Angela Zhou. Confounding-robust policy evaluation in infinite-horizon reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

- Leon J. Kamin. Predictability, surprise, attention and conditioning. In Byron A. Campbell and Russell M. Church, editors, *Punishment and Aversive Behavior*, pages 279–296. Appleton-Century-Crofts, 1969.
- Michael Kearns, Yishay Mansour, and Andrew Y. Ng. A sparse sampling algorithm for near-optimal planning in large Markov decision processes. *Machine Learning*, 49(2–3):193–208, 2002.
- Robert Kleinberg and Tom Leighton. The value of knowing a demand curve: Bounds on regret for online posted-price auctions. In *Proceedings of the 44th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 594–605, 2003.
- David L. Kleinman. On an iterative technique for Riccati equation computations. *IEEE Transactions on Automatic Control*, 13(1):114–115, 1968.
- Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12*, pages 1008–1014. MIT Press, 2000.
- Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. *arXiv preprint arXiv:2205.11275*, 2022.
- Aviral Kumar, Rishabh Agarwal, Dibya Ghosh, and Sergey Levine. Implicit under-parameterization inhibits data-efficient deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- Harold J. Kushner and Dean S. Clark. *Stochastic Approximation Methods for Constrained and Unconstrained Systems*. Springer-Verlag, New York, 1978.
- Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985.
- Nevena Lazic, Tyler Lu, Craig Boutilier, Moonkyung Ryu, Eehern Wong, Binz Roy, and Greg Imwalle. Data center cooling using model-predictive control. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Gen Li, Yuting Wei, Yuejie Chi, and Yuxin Chen. Softmax policy gradient methods can take exponential time to converge. *Mathematical Programming*, 196:579–632, 2022.
- Gen Li, Laixi Shi, Yuxin Chen, Yuting Wei, and Yuejie Chi. Is q-learning minimax optimal? a tight sample complexity analysis. *Operations Research*, 72(1), 2024a.
- Gen Li, Yuting Wei, Yuejie Chi, Yuantao Gu, and Yuxin Chen. Breaking the sample size barrier in model-based reinforcement learning with a generative model. *Operations Research*, 72(1), 2024b.
- Minne Li, Zhiwei Qin, Yan Jiao, Yaodong Yang, Zheng Gong, Jun Wang, Changjie Wang, Gauge Wu, and Jieping Ye. Efficient ridesharing order dispatching with mean field multi-agent reinforcement learning. In *Proceedings of the 2019 World Wide Web Conference (WWW)*, pages 983–994, 2019.
- Luofeng Liao, Zuyue Fu, Zhuoran Yang, Yixin Wang, Dingli Ma, Mladen Kolar, and Zhaoran Wang. Instrumental variable value iteration for causal offline reinforcement learning. *Journal of Machine Learning Research*, 25(303):1–56, 2024.
- Han-Dong Lim and Donghwan Lee. Regularized q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

- Long-Ji Lin. *Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching*. PhD thesis, Carnegie Mellon University, 1992. Also published in *Machine Learning*, 8(3–4):293–321, 1992.
- Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine Learning Proceedings 1994*, pages 157–163. Morgan Kaufmann, 1994.
- Michael L. Littman. Friend-or-foe q-learning in general-sum games. In *Proceedings of the 18th International Conference on Machine Learning*, pages 322–328, 2001.
- Allen Liu, Jingwen Yang, Yining Wang, and Jianghao Sun. Contextual dynamic pricing with strategic buyers. *arXiv preprint arXiv:2307.04055*, 2024.
- Nikolay Lomys and Luca Magnolfi. Estimation of games under no regret: Structural econometrics for ai. Working Paper NET Institute Working Paper No. 24-05, Social Science Research Network (SSRN), 2024. Available at SSRN: <https://ssrn.com/abstract=4717195> or <http://dx.doi.org/10.2139/ssrn.4717195>.
- Clare Lyle, Mark Rowland, and Will Dabney. Understanding and preventing capacity loss in reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- Clare Lyle, Mark Rowland, Will Dabney, Marta Kwiatkowska, and Yarin Gal. Understanding plasticity in neural networks. In *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2023.
- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Disentangling causes of plasticity loss in neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Lilia Maliar, Serguei Maliar, and Pablo Winant. Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101, 2021.
- Alan S. Manne. Linear programming and sequential decisions. *Management Science*, 6(3): 259–267, 1960.
- Amir massoud Farahmand, Rémi Munos, and Csaba Szepesvári. Error propagation for approximate policy and value iteration. In *Advances in Neural Information Processing Systems 22 (NeurIPS)*, pages 568–576, 2010.
- Daniel McFadden. Conditional logit analysis of qualitative choice behavior. *Frontiers in Econometrics*, pages 105–142, 1974a.
- Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, 1974b. Working paper circulated 1973.
- Jincheng Mei, Chenjun Xiao, Csaba Szepesvari, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *International Conference on Machine Learning*, 2020.
- Kanishka Misra, Eric M. Schwartz, and Jacob Abernethy. Dynamic online pricing with incomplete information using multiarmed bandit experiments. *Marketing Science*, 38(2):226–252, 2019.

- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.
- Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 2023.
- Jonas Mueller, Vasilis Syrgkanis, and Matt Taddy. Low-rank bandit methods for high-dimensional dynamic pricing. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9(27):815–857, 2008.
- Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 673–680, 2006.
- Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2022.
- Michael Oberst and David Sontag. Counterfactual off-policy evaluation with Gumbel-Max structural causal models. In *Proceedings of the 36th International Conference on Machine Learning*, pages 4923–4932, 2019.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- Ariel Pakes and Paul McGuire. Computing Markov-perfect Nash equilibria: Numerical implications of a dynamic differentiated product model. *RAND Journal of Economics*, 25(4): 555–589, 1994.
- Fabio Pardo, Arash Tavakoli, Vitaly Levnik, and Petar Kormushev. Time limits in reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2018.
- Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. In *arXiv preprint arXiv:2304.01315*, 2024.
- Ivan P. Pavlov. *Conditioned Reflexes: An Investigation of the Physiological Activity of the Cerebral Cortex*. Oxford University Press, 1927.
- Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2nd edition, 2009.
- Judea Pearl and Elias Bareinboim. External validity: From do-calculus to transportability across populations. *Statistical Science*, 29(4):579–595, 2014.

- Moshe A. Pollatschek and Benjamin Avi-Itzhak. Algorithms for stochastic games with geometrical interpretation. *Management Science*, 15(7):399–415, 1969.
- Martin L. Puterman and Shelby L. Brumelle. On the convergence of policy iteration in stationary dynamic programming. *Mathematics of Operations Research*, 4(1):60–69, 1979.
- Marek Pycia and Peter Troyan. A theory of simplicity in games and mechanism design. *Econometrica*, 91(4):1495–1526, 2023.
- Liqun Qi and Jie Sun. A nonsmooth version of Newton’s method. *Mathematical Programming*, 58(1–3):353–367, 1993.
- Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. Ride-hailing order dispatching at DiDi via reinforcement learning. *INFORMS Journal on Applied Analytics*, 51(3):272–286, 2021.
- Guannan Qu and Adam Wierman. Finite-time analysis of asynchronous stochastic approximation and q-learning. *Journal of Machine Learning Research*, 21(1):1–28, 2020.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Sai Srivatsa Ravindranath, Zhe Feng, Di Wang, Manzil Zaheer, Aranyak Mehta, and David C. Parkes. Deep reinforcement learning for sequential combinatorial auctions. Submitted to ICLR 2025, 2024. Available at <https://openreview.net/forum?id=SVd9Ffcdp8>.
- Pranjal Rawat. Designing auctions when algorithms learn to bid. *Working Paper*, 2026.
- Robert A. Rescorla and Allan R. Wagner. A theory of Pavlovian conditioning: Variations in the effectiveness of reinforcement and nonreinforcement. In Abraham H. Black and William F. Prokasy, editors, *Classical Conditioning II: Current Research and Theory*, pages 64–99. Appleton-Century-Crofts, 1972.
- Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.
- James M. Robins, Andrea Rotnitzky, and Lue Ping Zhao. Estimation of regression coefficients when some regressors are not always observed. *Journal of the American Statistical Association*, 89(427):846–866, 1994.
- Michael Rothschild. A two-armed bandit theory of market pricing. *Journal of Economic Theory*, 9(2):185–202, 1974.
- G. A. Rummery and M. Niranjan. On-line q-learning using connectionist systems. Technical report, Cambridge University, 1994.
- John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.
- John Rust and Pranjal Rawat. Structural econometrics and inverse reinforcement learning: Inferring preferences and beliefs from human behavior. Working Paper, Georgetown University, 2026.
- Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.

- Manuel S. Santos and John Rust. Convergence properties of policy iteration. *SIAM Journal on Control and Optimization*, 42(6):2094–2115, 2004.
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- Bernhard Schölkopf, Francesco Locatello, Stefan Bauer, Nan Rosemary Ke, Nal Kalchbrenner, Anirudh Goyal, and Yoshua Bengio. Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634, 2021.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Lior Shani, Yonathan Efroni, and Shie Mannor. Adaptive trust region policy optimization: Global convergence and faster rates for regularized MDPs. In *AAAI Conference on Artificial Intelligence*, 2020.
- Claude E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41(314):256–275, 1950.
- Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.
- Lloyd S. Shapley. Some topics in two-person games. In M. Dresher, L. S. Shapley, and A. W. Tucker, editors, *Advances in Game Theory*, pages 1–28. Princeton University Press, 1964.
- Yoav Shoham, Rob Powers, and Trond Grenager. If multi-agent learning is the answer, what is the question? *Artificial Intelligence*, 171(7):365–377, 2007.
- Aaron Sidford, Mengdi Wang, Xian Wu, Lin F. Yang, and Yinyu Ye. Near-optimal time and sample complexities for solving Markov decision processes with a generative model. In *Advances in Neural Information Processing Systems*, 2018.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

- Satinder Singh, Tommi Jaakkola, Michael L Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine learning*, 38: 287–308, 2000.
- Satinder P. Singh and Richard C. Yee. An upper bound on the loss from approximate optimal-value functions. *Machine Learning*, 16(3):227–233, 1994.
- Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashenninikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The dormant neuron phenomenon in deep reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2023.
- Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021, 2020.
- Nancy L. Stokey. Rational expectations and durable goods pricing. *Bell Journal of Economics*, 12(1):112–128, 1981.
- Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1023/A:1022633531479.
- Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224. Morgan Kaufmann, 1990.
- Richard S. Sutton and Andrew G. Barto. Time-derivative models of Pavlovian reinforcement. In Michael Gabriel and John Moore, editors, *Learning and Computational Neuroscience: Foundations of Adaptive Networks*, pages 497–537. MIT Press, 1990.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.
- Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999.
- Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 2000.
- Richard S. Sutton, Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba Szepesvári, and Eric Wiewiora. Fast gradient-descent methods for temporal-difference learning with linear function approximation. In *Proceedings of the 26th International Conference on Machine Learning*, pages 993–1000. ACM, 2009.
- Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2010.
- Oskari Tammelin. Solving large imperfect information games using cfr+. *arXiv preprint arXiv:1407.5042*, 2014.

- Xiaocheng Tang, Zhiwei Qin, Fan Zhang, Zhaodong Wang, Zhe Xu, Yintai Ma, Hongtu Zhu, and Jieping Ye. A deep value-network based approach for multi-driver order dispatching. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1780–1790, 2019.
- Gerald Tesauro. Td-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.
- William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.
- Edward L. Thorndike. *Animal Intelligence: An Experimental Study of the Associative Processes in Animals*. Macmillan, 1898. Psychological Review Monograph Supplements, No. 8.
- Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the Fourth Connectionist Models Summer School*, 1993.
- Haoxing Tian, Ioannis Ch. Paschalidis, and Alex Olshevsky. Convergence of actor-critic methods with multi-layer neural networks. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- Mark Towers, Ariel Kwiatkowski, Jordan K. Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulao, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tze, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In *arXiv preprint arXiv:2407.17032*, 2024.
- John N. Tsitsiklis. Asynchronous stochastic approximation and q-learning. *Machine Learning*, 16(3):185–202, 1994. doi: 10.1007/bf00993306.
- John N. Tsitsiklis. On the convergence of optimistic policy iteration. *Journal of Machine Learning Research*, 3:59–72, 2002.
- John N. Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in Neural Information Processing Systems 9 (NIPS)*, 1997.
- Daniele Tullii, Adel Javanmard, Matteo Pirodda, and Pierre Lezard. Contextual dynamic pricing with strategic buyers under unknown valuations. *arXiv preprint arXiv:2307.04895*, 2024.
- Hado van Hasselt. Double q-learning. In *Advances in Neural Information Processing Systems*, volume 23, 2010.
- Hado van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems*, volume 29, 2016a.
- Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-Learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016b.
- Hado van Hasselt, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. Deep reinforcement learning and the deadly triad. In *arXiv preprint arXiv:1812.02648*, 2018.
- Martin J. Wainwright. Stochastic approximation with cone-contractive operators: Sharp ℓ_∞ -bounds for Q-learning. *Annals of Statistics*, 47(6):3168–3197, 2019.

- Yue Wang, Tomasz Żak, and Csaba Szepesvári. Near-optimal sample complexity for iterated CVaR reinforcement learning with a generative model. *arXiv preprint arXiv:2503.08934*, 2025.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- Di Wu, Xiujun Chen, Xun Yang, Hao Wang, Qing Tan, Xiaoxun Zhang, Jian Xu, and Kun Gai. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1443–1451, 2018.
- Yue Wu, Weitong Zhang, Pan Xu, and Quanquan Gu. A finite-time analysis of two time-scale actor-critic methods. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Lin Xiao. On the convergence rates of policy gradient methods. *Journal of Machine Learning Research*, 23, 2022.
- Jianyu Xu and Yu-Xiang Wang. Logarithmic regret in feature-based dynamic pricing. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Yinyu Ye. The simplex and policy-iteration methods are strongly polynomial for the Markov decision problem with a fixed discount rate. *Mathematics of Operations Research*, 36(4):593–603, 2011.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.
- Junzhe Zhang and Elias Bareinboim. Near-optimal reinforcement learning in dynamic treatment regimes. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Junzhe Zhang and Elias Bareinboim. Designing optimal dynamic treatment regimes: A causal reinforcement learning approach. In *Proceedings of the 37th International Conference on Machine Learning*, pages 11012–11022, 2020.
- Shangdong Zhang and Richard S. Sutton. A deeper look at experience replay. In *arXiv preprint arXiv:1712.01275*, 2017.
- Shangdong Zhang, Hengshuai Yao, and Shimon Whiteson. Breaking the deadly triad with a target network. In *International Conference on Machine Learning (ICML)*, 2021.
- Stephan Zheng, Alexander Trott, Sunil Srinivasa, David C. Parkes, and Richard Socher. The AI economist: Taxation policy design via two-level deep multiagent reinforcement learning. *Science Advances*, 8(18):eabk2607, 2022.
- Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.
- Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, volume 20, 2008.